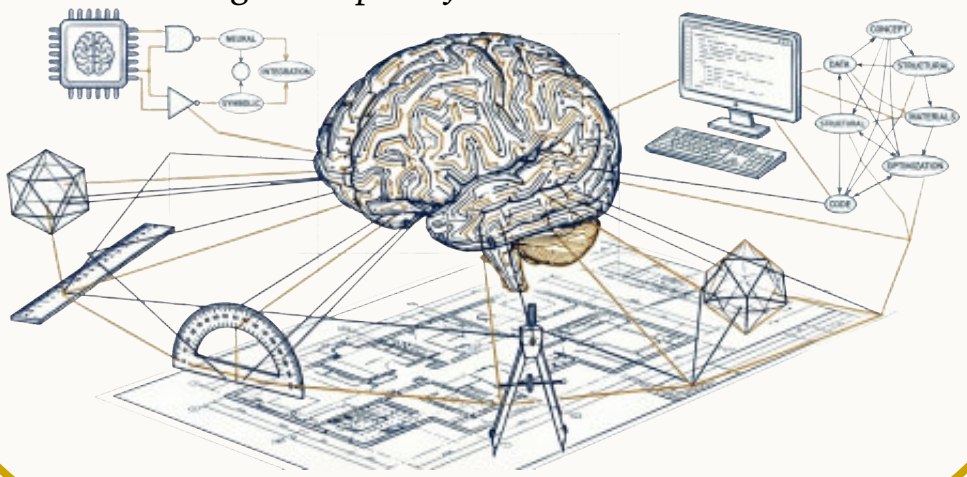
The Architect's Code

A Strategic Blueprint for SS26 Bachelor Theses.



Bachelor Thesis Proposals

(The Comprehensive Version)

Supervisor: **Ahmed M. H. Abdelfattah**

Faculty of Media Engineering & Technology

German University in Cairo (GUC)

eMail: ahmed.abdelfattah@guc.edu.eg

Office: **C7.207**

February 11, 2026

Contents

1 First Things First **3**
1.1 Introduction: Moving Beyond the “Bot” . . . 3
1.2 Foundational Concepts & Resources . . . 4
1.3 Required Background & Technical Foundations . . . 4
1.4 The Audit Paradigm: Supervisory Manifesto . . . 7

2 Cluster 1: The Neuro-Symbolic Architects **16**
2.1 The AI Pinocchio Detector: Automated Fact-Checking . . . 18
2.2 The Curriculum Architect: Engineering the Perfect Learning Path . . . 20
2.3 The Honest Tutor: Building "Glass-Box" AI in Education . . . 22
2.4 Multi-Sensory Search Engine: The Cross-Modal Cartographer . . . 24
2.5 The Visual Knowledge Brain . . . 26
2.6 The Algorithmic Orchestrator: Policy-Driven Graph Optimization . . . 28
2.7 The Modern Rosetta Stone of Math: DeepMind-Style Truth Scanner . . . 30

3 Cluster 2: The Cognitive Auditors **32**
3.1 The “Terms of Service” Legal Checker . . . 34
3.2 The Prerequisite Pathfinder . . . 36
3.3 Modeling of Social Dynamics: Sternberg’s “Love Quantifier” . . . 38
3.4 System-2 Reasoning: The Wise Agent . . . 40
3.5 System-2 Planning: The Lookahead Agent . . . 42
3.6 The Empathy Auditor: The ELIZA Effect 2026 . . . 44
3.7 Multi-Agent Pathfinding: The Traffic Controller . . . 46
3.8 System-2 Planning: The Deadlock Detective . . . 48
3.9 The Logic Advocate Tutor . . . 50

4 Cluster 3: The Human-Centric Generators **53**
4.1 Sketch-based Heritage Restoration . . . 55
4.2 The Kinematic Calligrapher: Vector-Based Arabic Script Generation . . . 57
4.3 The Algorithmic Complexity Visualizer . . . 59
4.4 The Dysgraphia Detective . . . 61
4.5 The Digital Handwriting Scorer: Assessment of Handwriting . . . 63

5 [Appendix] AI SW Development Tools Landscape **68**
5.1 The Context: An Agentic Shift . . . 68

6 [Appendix] My NeSy NoteBookLM Approach **70**
6.1 The Manifesto: Why We Are Here . . . 72
6.2 The Baseline: Building a "Naive" NotebookLM . . . 73
6.3 The Neuro-Symbolic Course Compiler . . . 74
6.4 The Socratic Loop (Argumentation Frameworks) . . . 76
6.5 Tools of the Trade . . . 77

6.6 The Epistemic Auditor (Ontology-Mediated Verification) 78

7 [Appendix] My Kinematic Calligrapher Approach 91
7.1 The Manifesto: Moving Beyond "Pixel Space" 93
7.2 The Baseline: The "Naive" Image Generator 93
7.3 The Kinematic Calligrapher (Neural ODEs) 94
7.4 The Digital Graphologist (Verification & Assessment) 95
7.5 Implementation Guide (The Pipeline) 96
7.6 Evaluation Strategy: The Truth Bar 97

Chapter 1

First Things First

1.1 Introduction: Moving Beyond the "Bot"

For a century, Computer Science has been a cathedral of human reason. We built the machines, the networks, the languages, and the data world they run on. Then, a new force arrived. Modern AI, currently exemplified by LLMs, is not merely another incremental development but rather a profound disruption, delivering capabilities that would have been considered science fiction only months ago. We shouldn't compete with AI—we already lost this race. We cannot be faster, or know more. Our future lies in commanding AI, but command requires understanding. Thus, the true frontier is understanding and trust. This leads to the core idea of my research: **Formalization and Representation** — turning vague human concepts like understanding, trust, reasoning, and analogy into rigorous structures we can model, analyze, and verify. The relentless rise shakes our foundations and forces a question:

How should humans position themselves in an era defined by rapidly evolving, superhuman-level AI technologies?

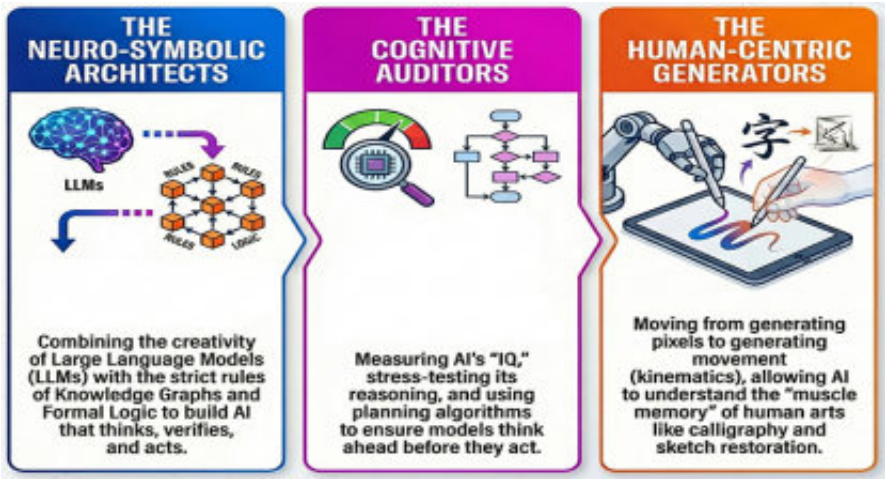
Your thesis can join this mission through interwoven frontiers:

Graphs, Logic, and Other Friends (The Language of Trust & Reason):
We live in the era of LLMs, which are incredible at processing multimodal data, but have a fatal flaw: they are **probabilistic**, not **logical**. For such black box AI, graphs are our crystal window to see through. The foundational structure of dots and lines provided a universal language for describing connections over ages. Spectral Graph Theory, Graph Neural Networks, Knowledge Graphs, and Retrieval-Augmented Generation (RAG) help us trace, explain, and improve machine reasoning.

Formalizing Cognition (The Mind):
To build trustworthy AI, we must formalize trust and how thinking is processed. Topics include cognitive models that reveal how insight and reasoning emerge.

Your future will be shaped not by how fast you code, but by how deeply you understand what the code becomes. **The industry is already shifting from the nice-looking "stochastic parrots" to the inevitable "verifiable agents"** —making models bigger will not be enough. The careers that thrive will belong to those who can make these systems *provably* understandable, reliable, and safe. Investing in those skills now will spare you the regret of being left behind later. That is the work that will define the next generation of computer scientists and engineers: *the architects*.

1.2 Foundational Concepts & Resources



My projects are organized into 3 strategic clusters that integrate novel concepts with established research themes.

1. **The Neuro-Symbolic Architects:** Combining LLMs with Knowledge Graphs, Ontologies, and Formal Logic to build trustworthy, verifiable systems (“Glass-Box" Verification).

2. **The Cognitive Auditors:** Using planning algorithms, formal generalization, and games to verify, measure, test, and "fact-check" AI reasoning.

3. **The Human-Centric Generators:** Moving from pixel-based image generation to vector-based kinematic generation (Hand-sketching, Hand-writing, Education, Heritage, and Tourism).

This work is theoretically challenging, but its difficulty is transformed when it serves meaningful applications. In my case, these applications aim to build *trust* in the well-represented realities of the modern world. These theories matter precisely because they touch real life:

- *AI in Education* — ensuring not only correctness but genuine pedagogical value (Abdelfattah and Gebrail, 2025).
- *Arabic Handwriting* — modelling perception, cognition, and language (Abdelfattah and Fawzy, 2023; Abdelfattah and Amr, 2023).
- *Tourism and Heritage* — preserving and communicating culture responsibly (Abdelfattah and Atef, 2025a,b; Abdelfattah and Zakzouk, 2025).

1.3 Required Background & Technical Foundations

Students are expected to engage with the following paradigms throughout their work on the proposed topics. Before selecting a thesis topic, they should understand the core theories and technologies they will depend on. Only the key ones are outlined below. **A thesis will not address all of these areas simultaneously**; they are introduced briefly to help you identify the foundational domain your topic will require, and to align your choices with your interests and preferences.

Agentic AI vs. Passive Chatbots

You must first understand the recent shift that occurred in late 2025 from Chatbots to Agents.[1] A **Chatbot** passively predicts the next token to answer questions based on training data. An **Agent**, however, is a system that employs a *Perception-Action Loop*. It perceives a problem, selects a tool, executes it, observes the output, and iterates. An agent is, thus, an AI that can use **Tools**: *execute Python code, query a database, or search the web to solve a problem step-by-step*.

[1] Since this is more of an engineering than a thesis-related, more is given in chapter 5.

- *Concept:* The Agent-Environment Loop.
- *Keyword:* **ReAct (Reasoning + Acting)**.
- *Resource:* LangChain Documentation (Python).
- *Resource:* Yao et al. (2022), "ReAct: Synergizing Reasoning and Acting in Language Models".

Knowledge Graphs (KG)

A KG is a structured way to represent data as a network of entities (Nodes) and relationships (Edges). Unlike a standard database (tables), a KG allows an AI to "traverse" connections(e.g., $King \rightarrow Father \rightarrow Temple$).

- *Resource:* Neo4j Graph Database Introduction

Retrieval-Augmented Generation (RAG) & GraphRAG

RAG is the technique of giving an LLM a "textbook" to read before answering a question. Instead of relying on memory, the AI retrieves relevant documents to ground its answer. Standard RAG retrieves documents based on vector similarity. **GraphRAG** retrieves information by traversing relationships in a structured Knowledge Graph. This allows for answering global questions that require "connecting disparate facts".

- *Concept:* Semantic Search & Vector Embeddings.
- *Resource:* Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks"

- *Keyword:* **Knowledge Graph Completion**.
- *Resource:* Edge et al. (2024), "From Local to Global: A Graph RAG Approach to Query-Focused Summarization" (Edge et al., 2024).

The Logic of Constraints: Description Logic[s] (DL)

A graph is merely a data structure; to reason, we need a logic. **Description Logics (DL)**, specifically the $\mathcal{ALC}$ family (Attributive Language with Complements), can be utilized to define the **ontologies** that constrain our AI. For example, we can formally restrict a system's output by defining a valid "Parent" axiomatically: $Parent \equiv Person \sqcap \exists hasChild.Person$. If a generative model produces an output claiming an entity is a Parent without the corresponding "evidence", the logical reasoner flags this as an inconsistency. This constitutes a "**System 2**" override of the "System 1" generation, providing a mathematical guarantee against specific types of hallucinations. More about "System 1" and "System 2" is given [Link]in this page, and in the next paragraph. The ideas are also utilized in some project proposals, including the ones in §**3.4** (page 40), §**3.5** (page 42), and §**3.8** (page 48).

The Cognitive Gap: System 1 without System 2

To understand the fundamental flaw in current Generative AI, we can turn to the Nobel Prize-winning psychologist Daniel Kahneman and his framework of the human mind, described in *Thinking, Fast and Slow*. Kahneman distinguishes between two modes of thought:

System 1 (Fast): This is your intuition. It is automatic, emotional, and effortless. When you see a face and instantly know it is "angry," or when you complete the phrase "bread and...", that is System 1. It operates on patterns and "vibes."

System 2 (Slow): This is your **deliberate reasoning**. It is logical, calculating, and effortful. When you are asked to solve **17** $\times$ **24**, your intuition fails. You must stop, focus, apply a specific set of rules (algorithms), and calculate the answer step-by-step.

The Problem: Modern Deep Learning (LLMs like ChatGPT) is essentially a distinct, silicon-based version of **System 1**. It is the most powerful pattern-matching engine ever created (till now). It answers questions based on statistical likelihood—the "vibe" of the answer—rather than logical verification. *This explains why it can write a poem (a pattern task) but struggles with a novel math proof (a logic task)*. It is all intuition, no deliberation.
The Mission: The goal of some projects (particularly these ones: §**3.4** (page 40), §**3.5** (page 42), and §**3.8** (page 48)) is not to discard System 1 of the modern age, but to build the missing half of the brain. We aim to construct a digital **System 2**—a layer of Knowledge Graphs, Formal Logic, and Dynamical Systems—that forces the AI to stop, check its facts, and reason through a problem before speaking.

Neuro-Symbolic AI/Integration (NeSy)

Neuro-Symbolic AI is a trial to unify neural learning and symbolic reasoning. It attempts to merge the learning capability of Neural Networks with the reasoning capability of Symbolic Logic. We specifically look at **Neural-Symbolic Learning and Reasoning (NeSy)**, where logic rules constrain neural outputs.

- *Keyword:* **Primal-Dual reasoning**.
- *Resource:* Neuro-Symbolic AI Journal/Community.

There are, however, various "integration" headaches, mostly rooted in the fact that symbolic reasoning systems and neural networks speak fundamentally different languages at the hardware level: Symbolic systems love discrete structures—graphs, logical formulas, pattern matching, whereas neural networks love dense matrices and tensors that can be blasted through GPU cores in parallel.

Trustworthy (TAI) & Explainable AI (XAI)

Trustworthy AI (TAI) is a comprehensive paradigm aiming to develop AI systems that are safe, reliable, ethical, and accountable. The core principles of TAI, outlined by bodies such as the [Link]NIST AI Risk Management Framework, revolve around accountability, fairness, robustness, privacy, and human oversight. A critical enabler of TAI is **Explainable AI** (XAI), which seeks to make AI decision-making processes understandable and transparent to human users, moving beyond "black box" operations. XAI techniques are vital for building user confidence and ensuring regulatory compliance in high-stakes domains like finance, healthcare, or education.

A key practical mechanism within XAI that underpins transparency is **source attribution**. This process involves systematically identifying and citing the specific data points, documents, or training samples that informed a particular AI output or decision. Effective source attribution allows users to verify factual claims, judge the credibility of the information provided by the AI, and fosters necessary scrutiny of the system's knowledge base.[2]

The Explainability Turn in Formal Mathematics: The current frontier of trustworthy AI is defined by the convergence of LLMs with "formal verification systems", marking a significant step toward robust neuro-symbolic integration. At the foundation of this ecosystem lies **Lean 4** (de Moura and Ullrich, 2021), a functional programming language and interactive theorem prover. Unlike standard programming languages, Lean includes a small, trusted logical kernel that certifies the correctness of mathematical statements and code. This ecosystem is bolstered by **Mathlib** (The

[2]For further study on these critical topics, consider reviewing academic research papers, such as [Link]"Attribution for AI-Generated Text: A Survey" (2023), or watching educational content like this [Link]introductory video on AI explainability. These resources provide essential context on achieving verifiable and accountable AI systems. See also [Link]Source attribution and detection strategies for AI-era journalism (2025).

Mathlib Community, 2020), a unified, community-driven library of formalized mathematics. For a computer scientist, Mathlib represents not just a knowledge base, but a massive, structured dataset of ground-truth reasoning, essential for training and evaluating symbolic reasoning agents.

To bridge the gap between natural language intuition and formal tactics, the community utilizes pedagogical tools like the **Natural Number Game** (Buzzard et al., 2024). This resource gamifies the process of proof construction, effectively teaching the "gameplay" of interacting with the Lean kernel. This highlights a critical mechanism: ***proof search can be modeled as a game state space***, where the formal system provides immediate, deterministic feedback (rewards) on the validity of an agent's actions, solving the sparse reward problem common in reinforcement learning for reasoning tasks. The potential of this architecture was recently realized by Harmonic Labs' **Aristotle system** (Polu et al., 2025), which achieved Gold-Medal performance at the 2025 International Mathematical Olympiad. Aristotle exemplifies the ideal neuro-symbolic loop: a neural "generator" (the LLM) proposes proof tactics based on intuition, while a symbolic "verifier" (Lean) enforces logical correctness. This architecture guarantees that the final output is *hallucination-free and formally verifiable*, establishing a new standard for explainable and trustworthy AI in high-stakes quantitative domains.

The four pillars of the modern "AI for Formal Mathematics" ecosystem are the following:

1. The Engine (de Moura and Ullrich, 2021): Lean 4 is the programming language and theorem prover that powers everything else.

2. The Fuel / The Library (The Mathlib Community, 2020): Mathlib is the massive, unified library of mathematics built inside Lean. The seminal citation for "The Lean Mathematical Library" is the CPP 2020 paper (The Mathlib Community, 2020).

3. The Training Tutorial (Buzzard et al., 2024): The Natural Number Game is the standard pedagogical tool (tutorial) used to teach mathematicians how to use Lean and Mathlib.

4. The Result / AI Application (Polu et al., 2025): Harmonic's "Aristotle" (Polu et al., 2025) is the state-of-the-art AI agent that uses (1: The Engine) and (2: The Fuel/Library)—and arguably learns from data like (3: The Training Tutorial)—to solve Olympiad-level problems autonomously.

   Harmonic Blog post is available at: Harmonic Labs. (2025). However, always cite the formal technical report (Polu et al., 2025) released by Harmonic (October 2025), which details the "Aristotle" system. This is the paper that describes the "Gold Medal" achievement technically.

1.4 The Audit Paradigm: Supervisory Manifesto

1.4.1 Core Challenges

Topics proposed in modern research directions like Neuro-Symbolic AI, Formal Verification, or Cognitive Modeling are, no doubt, ambitious and cutting-edge. However, when we look at the practical landscape, we must be more realistic about the execution. Many engineering students usually face a structural problem, as the cohort largely consists of undergraduates with potentially weak or limited **Mathematical Backgrounds** (often due to limited retention of Calculus or Linear Algebra), working within extremely **Short Timeframes** (as they only have 14 weeks or even less to complete the work), and facing significant **Infrastructure Constraints** (since our beloved institution does not provide access to high-end GPU clusters or the necessary compute power for training massive models).

To bridge this significant gap while simultaneously engaging with cutting-edge projects and achieving outcomes of industry-level quality, a **Strategic Solution** is required. This solution is what I call "the ***Validation over Creation*** strategy". This means we need to fundamentally shift

the thesis archetype from "Building a Novel Engine" (which effectively requires a Masters or PhD level of theoretical depth) to "Benchmarking and Auditing" (which merely requires engineering diligence and analytical rigor).

To implement this, let us utilize the **"A/B Test" Paradigm**: Instead of asking a bachelor student to *invent* a completely new algorithm from scratch, we ask them to *compare* two existing approaches against one another. For example, we do not ask the student to "build an AI that solves integrals," but rather ask them to "compare how GPT-5 fails at Integrals versus how Claude 4 fails." This approach satisfies the rigorous research requirement (necessitating a literature review, a defined methodology, and statistical analysis) without risking total project failure due to coding complexity, convergence issues, or whatsoever other unforeseen implementation challenges.

We also need to apply the **50/50 Grading Rule** to ensure students do not simply "script" their way to a degree without deep, insightful, and conceptual understanding. This is achieved by enforcing a strictly split workload: **50% Research & Self-Learning** and **50% Implementation & Analysis**. For the former (Research), the students must map their specific problem to a *Formal Framework* (e.g., the framework introduced in subsection 1.4.3), they must read seminal papers related to their topic, and they must conduct a comprehensive literature review and related work analysis. For the latter (Implementation), the students must run the code (even if the script is simple) and produce a rigorous analysis of the results, explaining *why* one model outperformed the other based on the theoretical concepts they learned.

1.4.2 An Abstract Roadmap Towards (Theoretical) Trustworthiness

To rigorously define the operation of ***a reliable system***, we cannot simply rely on code; we should first move beyond the software architecture of the system to **formalize** the problem of *Trustworthy Multimodal Retrieval*. We must define the necessary logical conditions that differentiate a reliable system from a hallucinating one. A preliminary (but novel) proposal to handle trustworthy AI (TAI) is the rudimentary "schema-based framework" that I first presented in Abdelfattah and Atef (2025b) for a proof-of-concept app, MuseePal. The MuseePal app (Abdelfattah and Atef, 2025a) abstracts away the specific tools it uses (such as YOLO for vision or Gemini for text) to define a model-agnostic formalism that could be followed as a pipeline by other AI systems or apps. By defining the interaction between "perception" and "retrieval" using set theory and logic, this establishes a blueprint that ensures reliability is a mathematical constraint of the system design, rather than a lucky byproduct of model scale.

Let's first introduce what schemas and source attribution refer to within the framework's context, as these are the pillars of our logic.

Agentic AI and Schemas: A "**Schema**" is a structured ontology defining specific attributes. Outputs must be structurally constrained to prevent AI from generating vague, flowery, or generic descriptions. In the context of a heritage-based work, such as Abdelfattah and Zakzouk (2025), the schema is specifically called "cultural schema". The presented attribute samples include, e.g., "Historical Significance," "Religious Beliefs," or "Artistic Style", that an AI agent must populate explicitly. The MuseePal app (Abdelfattah and Atef, 2025b) integrates this schema into its retrieval logic, ensuring that the visual detection of an artifact triggers a structured, multi-dimensional narrative rather than a generic summary that could apply to any statue.

Trustworthy AI and Source Attribution: "**Source Attribution**" is a necessary condition for trust in AI. The framework introduced in Abdelfattah and Gebrail (2025) demonstrated that *constraining an LLM to a closed set of instructor-curated materials, with mandatory citation, significantly reduces fabrication*. Again, the proof-of-concept app (MuseePal) adapts this paradigm to the cultural sector, treating museum artifacts as "learning objects" that require the same rigorous attribution standards as an academic paper.

1.4.3 Abstracting The Framework Theoretically: The Schema-Guided Grounding Model

To ensure academic rigor across many of the disparate topics you may choose—from Law to Art; from Education to Math—we cannot simply "hack" a solution together. **The theoretical lens** presented in Abdelfattah and Atef (2025b) is required to be adopted to give your engineering work a mathematical foundation. Let's talk more about this.

The Knowledge Space Tuple $\mathcal{K} = (\mathcal{A}, \mathcal{D}, \mathcal{S})$

We begin by defining the domain in which the AI system under consideration operates. Unlike open-domain chatbots (like ChatGPT) that drift through the entire internet, the systems that will adopt this approach[3] must be constrained to a specific **environment**. Let the **environment** be defined mathematically as a **Knowledge Space tuple**, $\mathcal{K} = (\mathcal{A}, \mathcal{D}, \mathcal{S})$. This tuple represents the boundaries of your system's world, where:

- $\mathcal{A} = \{a_1, ..., a_n\}$ is the finite set of distinct physical **artifacts** or subjects of analysis.
- $\mathcal{D} = \{d_1, ..., d_m\}$ is the set of curated **document chunks** stored in the knowledge base. These chunks serve as the atomic units of truth (the ground truth in a vector space) which the AI is allowed to quote.
- $\mathcal{S} = \{s_1, ..., s_k\}$ is the **schema** (ontology), where each s_i represents a specific **semantic dimension of inquiry** that guides the generation.

Your System's $\mathcal{K}$: In this way, we define any **domain of inquiry** (the system you are building) as a tuple $\mathcal{K} = (\mathcal{A}, \mathcal{D}, \mathcal{S})$, where $\mathcal{A}$ is the artifact set, $\mathcal{D}$ is the document store set, and $\mathcal{S}$ is a schema. The first task in your thesis is to define these three sets for the specific topic (or system) under consideration as follows.

The Artifact Set ($\mathcal{A}$): The discrete set of subjects being analyzed. This is the "Input" that triggers the system.

- **In Museums:** Physical Statues or Paintings.
- **In Math:** Specific Problems (e.g., "`Integrate` x^2").
- **In Law:** Legal Contracts (PDF files to be audited).
- **In Education:** Specific Lesson Plans or Student Essays.

The Document Store ($\mathcal{D}$): The "Ground Truth" or source of valid knowledge. This is the "Library" the system reads from.

- **In Museums:** Curator Notes / Wikipedia extracts.
- **In Math:** Standard Textbooks (e.g., Stewart's Calculus definitions).
- **In Law:** Civil Code / Terms of Service Database / The text within the contract itself.
- **In Education:** The approved Answer Key or Curriculum Standards.

The Schema ($\mathcal{S}$): The rigid ontology or "Form" that the AI must fill. This is responsible for preventing generic hallucinations by forcing the AI to look for specific attributes.

- **Generic AI:** Summarizes text loosely and often inaccurately.
- **Trustworthy AI:** Must fill slots: $S = \{Attribute_1, Attribute_2, ...\}$.

The Core Functions

Once the tuple $\mathcal{K}$ is defined, we must define how the system moves data through this space.

[3] Does your thesis adopt this approach? Read this section, and then ask your supervisor!

The Neural Perception Mapping: The first stage of the **pipeline**[4] is **(Visual) Perception**. The goal of this stage is to convert continuous, noisy data (e.g., pixels, raw text) into a discrete symbolic identifier that matches an element in our artifact set $\mathcal{A}$.

Let $\mathcal{V}$ be the (potentially infinite) space of all possible inputs (the "real world"). We define the perception model not merely as a classifier, but as a mapping function:

$$\Phi : \mathcal{V} \to \mathcal{A} \cup \{\emptyset\} \tag{1.1}$$

that maps the input space $\mathcal{V}$ (e.g., camera frames, scanned PDFs) to the artifact set $\mathcal{A}$, where $\Phi(v) = a_i$ if an artifact is detected with confidence $c > \tau$ (threshold), and the empty set otherwise. This function effectively discretizes the continuous world into symbolic tokens (artifact IDs). The core strength of this definition is that it holds regardless of whether the underlying implementation uses YOLO, Faster R-CNN, or future vision transformers, provided the confidence constraint is met.

The Grounded Retrieval Function: RAG (Retrieval-Augmented Generation) systems often fail because they retrieve documents based solely on generic similarity to a user query. Unlike standard RAG, the AI system of concern (e.g., MuseePal, or your specific project) enforces a structure by defining **Schema-Guided Retrieval**: a retrieval process that is guided by a Schema $\mathcal{S}$ (ontology).

We define **an embedding function** E that maps text or symbolic tokens into **a high-dimensional vector space**, $E : (\mathcal{A} \cup \mathcal{S} \cup \mathcal{D}) \to \mathbb{R}^d$. We define the **retrieval function** as a mapping:

$$\mathcal{R} : (\mathcal{A} \times \mathcal{S}) \to \mathcal{P}(\mathcal{D}) \tag{1.2}$$

that takes the detected artifact a_i and a specific schema attribute s_j derived from the user's query or the agent's narrative plan, and outputs a subset of (relevant) documents $\mathcal{D}_{rel} \subseteq \mathcal{D}$ of the document store, where $\mathcal{P}(\mathcal{D})$ is the power set of documents.

Formally, a retrieved subset $\mathcal{D}_{rel} \subseteq \mathcal{D}$ is defined as:

$$\mathcal{D}_{rel} = \{d \in \mathcal{D} \mid \text{sim}(E(d), E(a_i \oplus s_j)) \geq \epsilon\} \tag{1.3}$$

where $\oplus$ denotes semantic concatenation (combining the artifact name with the attribute description), and the function sim computes cosine similarity between the document vector $E(d)$ and the query vector. A retrieved subset is considered **valid** if and only if $\forall d \in \mathcal{D}_{rel}, \text{sim}(E(d), E(a_i \oplus s_j)) \geq \epsilon$. Thus, the vector similarity threshold (condition $\geq \epsilon$) ensures that the retrieval is not just semantically close enough to the query, but topologically bound to the intersection of the specific artifact and the specific cultural attribute.

The Trustworthiness Constraint: Finally, we formalize the generation step. We define the generative model G (LLM) not as a creative engine, but as a **transformation function** that takes the user query Q and the retrieved relevant documents $\mathcal{D}_{rel}$ to generate a textual response (an answer Y).

To satisfy the principles of Trustworthiness AI (TAI), we impose a verification constraint derived from the framework introduced in Abdelfattah and Gebrail (2025). We treat the output Y as a set of **atomic propositions** (**factual claims**) $\{p_1, ..., p_z\}$, where the generation is subject to a **verification constraint**:

$$Y \text{ is Trustworthy} \iff \forall p \in Y, \exists d \in \mathcal{D}_{rel} \text{ such that } d \models p \tag{1.4}$$

where $\models$ represents logical entailment. The constraint states that for a response to be considered trustworthy, every single factual claim p generated by the AI must be logically supported (modeled, $\models$) by a document d found in the retrieved set. MuseePal approximates this entailment by enforcing the **citation mechanism** $(p, \text{source}(d))$, thereby shifting the burden of verification from the "black box" neural network to the explicit source mapping.

[4]The "pipeline" is intended to be the processing by agents of the system under consideration.

Your System's Functions: Based on the above presentation, your student software should formally be defined as an implementation of three functions. ***You must clearly identify these in your thesis.***[5]

Perception Mapping (Φ): Discretizes noisy input into a clean symbol.

$$\Phi : \text{Input Space} \to \mathcal{A} \cup \{\emptyset\}$$

Example: A function that takes an image and outputs "`Statue_ID_42`", or takes a user prompt and identifies "`Emotion_Sadness`".

Retrieval Mapping (R): The lookup function that finds the truth.

$$R : (\mathcal{A} \times \mathcal{S}) \to \mathcal{P}(\mathcal{D})$$

Example: Given "`Statue_42`" and Attribute "`History`", retrieve specific paragraph d_{12} from the database.

Verification Constraint ($d \models p$): The entailment rule. Every atomic proposition p generated by the AI must be entailed by a retrieved document d.

$$\forall p \in \text{Output}, \exists d \in \mathcal{D} \text{ such that } d \models p$$

Significance: This is the "Citation" rule. The AI cannot speak unless it can point to a source document d that proves its claim p.

[5] A complete example is given in §**1.4.4** to exemplify how this could be done exactly.

1.4.4 Sample Project (Cog1): The "Terms of Service" Legal Checker

Below is a full-fledged, detailed example of a thesis and how its elements are mapped to the described schema-guided framework. This particular project is already defined within its cluster in §3.1 (page 34).

Industry Hook

Research Question: Can Constrained AI outperform Human Auditing in detecting hidden "traps" within contractual agreements?

Concept/Description

Problem: Users rarely read Terms of Service (ToS) due to obfuscated "Legalese."
Solution: A "Risk Detector" that audits legal PDFs. The student compares a generic AI summarizer (Baseline) against a strictly constrained *Logic Checker* (Proposed) that scans for specific "Danger Clauses" defined in a risk ontology.

Architecture: Mitigating Hallucination in Legal Texts via Schema-Guided Retrieval.

Strategic Info!

THEORETICAL MAPPING (The Schema-Guided Framework)

Mapping to Framework $\mathcal{K}$:

- $\mathcal{A}$ (Artifacts): The set of Contract PDFs (e.g., Netflix ToS, Spotify ToS).
- $\mathcal{S}$ (Schema): The Risk Ontology.

$$\mathcal{S} = \{\text{DataSelling}, \text{AutoRenewal}, \text{ArbitrationClause}\}.$$

- $\mathcal{D}$ (Document Store): The specific paragraphs within the PDF.
- **Constraint ($d \models p$):** The AI must extract the text and cite the specific Section Number (d) that entails the risk (p).

The Comparative Strategy (A/B Test):

1. **The Baseline (Control):** Prompt GPT-4 with "Summarize this contract and tell me if it's safe." (Hypothesis: It will be vague or hallucinate safety).

2. **The Experiment (Test):** Implement the Schema-Guided Pipeline. Iterate through

every $s \in \mathcal{S}$ (e.g., "Is there a waiver of liability?") and force the model to retrieve the exact paragraph d.

3. **Outcome:** A quantitative comparison—"The Schema-Based agent found 9/10 traps; the Generic Summarizer found only 3/10."

1.4.5 Sample Project (GenH2): The "Kinematic Calligrapher"

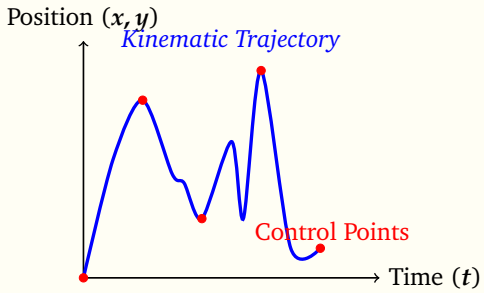
Below is a detailed breakdown of the flagship project in Cluster 3. It operationalizes the shift from Pixel Space to Kinematic Space, requiring the student to handle time-series data and vector mathematics. This project is defined in §4.2 (page 57).

Industry Hook

Research Question: Can AI learn the "Muscle Memory" of a human writer to generate infinite-resolution vector scripts?

Concept/Description

Problem: Standard Generative AI (like Stable Diffusion) treats text as a grid of pixels. This leads to "jagged edges" when zoomed and a loss of temporal information (e.g., writing the dot before the letter).
Solution: A "Generative Writer" that works in *Kinematic Space*. The student compares a pixel-based approach (Baseline) against a vector-based Time-Series model (Proposed) that predicts the velocity and pressure of the pen tip (x, y, t, p).



Architecture: From Static Images to Dynamic Vector Trajectories (SVG Generation).

Strategic Info!

THEORETICAL MAPPING (The Schema-Guided Framework)

Mapping to Framework $\mathcal{K}$:

- $\mathcal{A}$ (Artifacts): The set of Input Words or Characters to be written.
- $\mathcal{S}$ (Schema): The Legibility & Style Schema.

$$\mathcal{S} = \{\text{StrokeContinuity}, \text{PenUpEvents}, \text{GeometricSmoothness}\}.$$

- $\mathcal{D}$ (Document Store): The **IAM On-Line Handwriting Database** (Liwicki and Bunke, 2005) (Ground Truth trajectories).
- **Constraint ($d \models p$):** The generated vector path (p) must geometrically entail the topological rules found in the ground truth (d). For example, if d dictates a loop for the letter 'o', p must be a closed curve, not a disconnected scribble.

The Comparative Strategy (A/B Test):

1. **The Baseline (Control):** Train a standard Image Generator (CNN/Pixel-RNN) to generate an image of a word. (Hypothesis: It will be blurry or pixelated upon zooming).

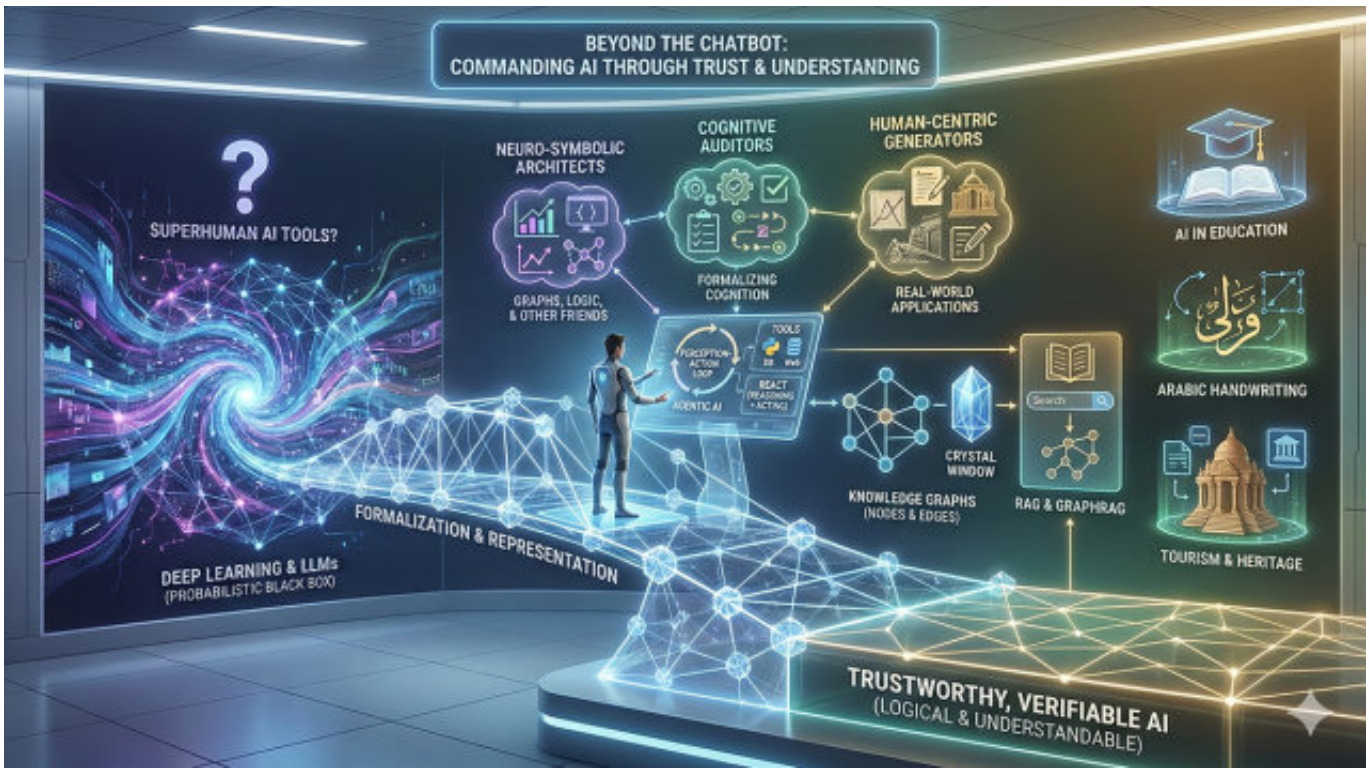
2. **The Experiment (Test):** Implement the Kinematic Pipeline. Train an LSTM/Transformer on the $\Delta x, \Delta y$ sequences from $\mathcal{D}$. Render the output mathematically as Scalable Vector Graphics (SVG).

3. **Outcome:** A quantitative comparison—"The Pixel Model required 1024 values to draw an 'A'; the Kinematic Model required only 20 control points and achieved infinite resolution."

The Architect's Manifesto: A Career Strategy Note & Honest Advice

The era of "coding fast" is over; the era of **engineering trust** has begun. Your future value is not defined by how many frameworks you know, but by how deeply you understand the *foundations*. We are witnessing a massive paradigm shift: **The industry is rapidly moving beyond the black box hype of "stochastic parrots" toward the necessity of transparent, "verifiable agents."** Simply making models bigger is no longer enough (nor is it the real challenge!). The careers that will define the next decade belong to those who can make AI *provably* understandable, reliable, and safe. Companies are **desperate** for engineers who can tame the chaos of LLMs with the rigor of Logic. Investing in these difficult, structural skills today is the only way to future-proof your career against the automation of tomorrow. Do not settle for being a user of the API; become the builder of the reasoning engine...
Be the Architect.

A Final Word: The challenge is never the topic itself; it is the tension between Scope and Enthusiasm. To master this 14ish-week semester, you must learn to decouple the Interface (the "cool product") from the Engine (the "deep engineering"). Build the engine first. Only then can you reintegrate them into a thesis that you will be proud of for a lifetime.



N.B.: Please note that other bachelor projects not listed in this file are to be conducted in cooperation with Dr. Yomna Hassan; these will be published on her list.

Chapter 2

Cluster 1: The Neuro-Symbolic Architects

Outline (Quick Access):

2.1 The AI Pinocchio Detector: Automated Fact-Checking 18
2.2 The Curriculum Architect: Engineering the Perfect Learning Path 20
2.3 The Honest Tutor: Building "Glass-Box" AI in Education 22
2.4 Multi-Sensory Search Engine: The Cross-Modal Cartographer 24
2.5 The Visual Knowledge Brain 26
2.6 The Algorithmic Orchestrator: Policy-Driven Graph Optimization 28
2.7 The Modern Rosetta Stone of Math: DeepMind-Style Truth Scanner 30

Cluster 1: The Neuro-Symbolic Architects

Building AI that doesn't just talk, but thinks, verifies, and acts.

Cluster Visualization

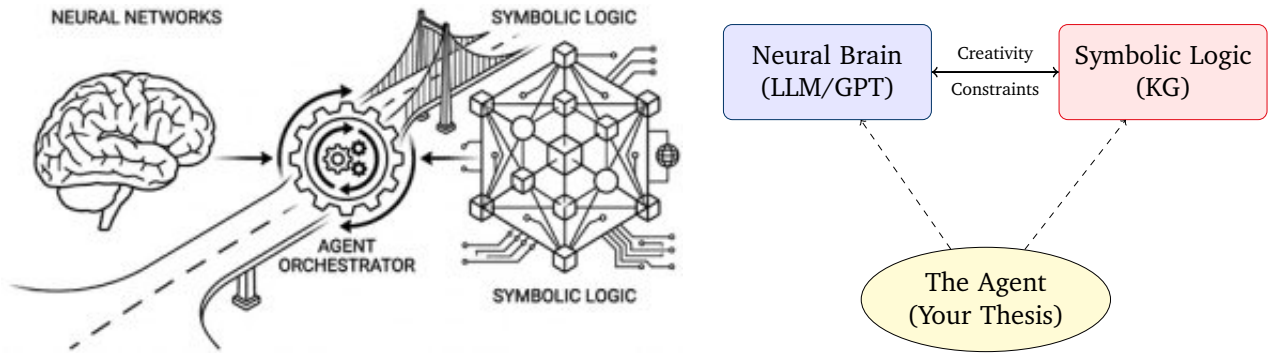


Figure 2.1: Conceptual diagram of an integration architecture. A central agent (gear icon / your thesis) mediates between the data-driven, connectionist paradigm of neural networks and the rule-based, structured paradigm of symbolic AI.

Theme: Formal Verification, and Trustworthy Agentic Workflows.

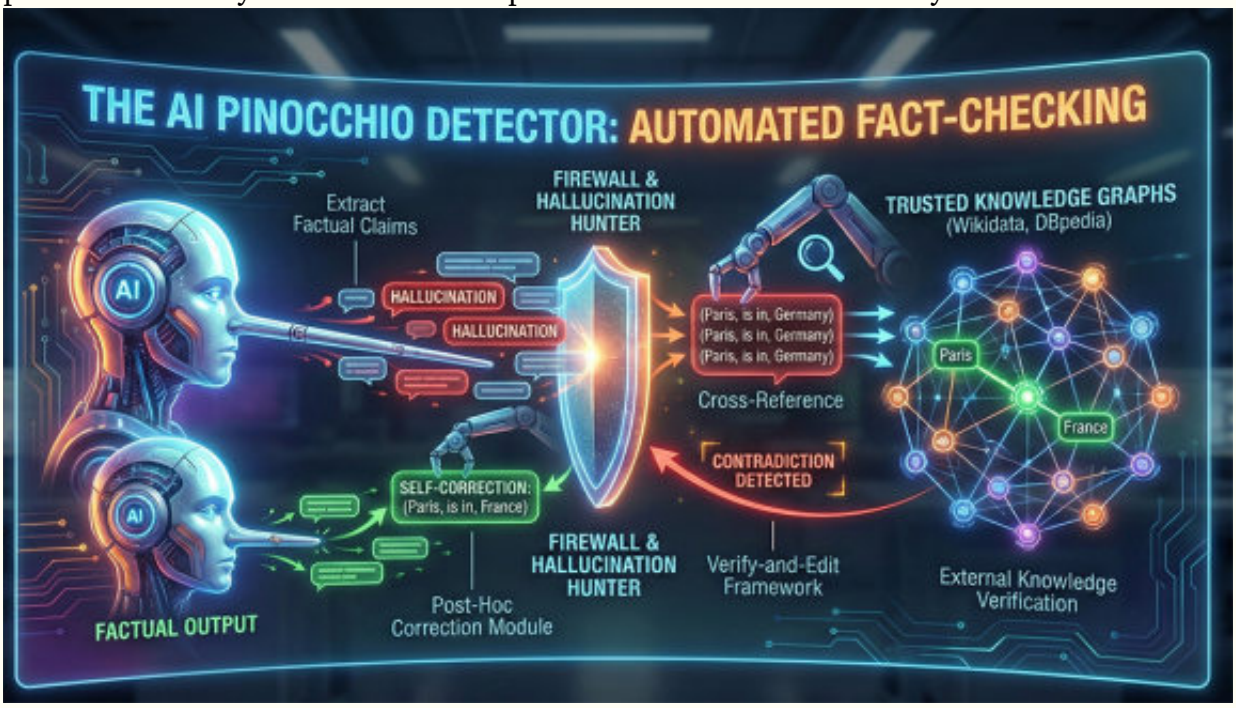
2.1 The AI Pinocchio Detector: Automated Fact-Checking

Industry Hook

"The AI Pinocchio Detector: Using cold, hard logic to catch a Chatbot in a lie."
Post-Hoc Correction via Knowledge Graphs

Concept/Description

Become the engineer who stops an LLM from lying/hallucinating. You will build a "Firewall" that intercepts AI responses, extracts factual claims, and cross-references them against trusted Knowledge Graphs (like Wikidata). If the AI lies, your system works as a **Hallucination Hunter** that forces a self-correction. This project gives you a portfolio piece that directly addresses the #1 problem in Generative AI today.



LLMs are probabilistic, not factual, and often generate "hallucinations" (plausible but false statements). This project builds a Post-Hoc correction module based on **External Knowledge Verification**. It extracts factual claims from an LLM's output (as triples) and verifies them against a trusted external Knowledge Base, like a curated Domain Ontology (which aligns with the "Verify-and-Edit" framework (Zhao et al., 2023)). If the LLM says "*Paris is in Germany,*" the system queries an external Knowledge Graph (Wikidata/DBpedia), detects the contradiction, and forces a self-correction (Fatahi Bayat et al., 2023).

Strategic Info!

Academic Description: This project addresses the stochastic nature of LLMs by implementing a "Verify-and-Edit" framework. The system functions as an AI Pinocchio Detector, extracting factual claims from the LLM's output as triples (Subject, Predicate, Object) and verifying them against a trusted external Knowledge Base (e.g., Wikidata or DBpedia). Upon detecting a contradiction (e.g., a geographical error), the system forces a logical self-correction module to align the generated text with the established ontology, effectively using cold logic to constrain neural creativity.

Paper: "Hybrid Fact-Checking that Integrates Knowledge Graphs, Large Language Models, and Search-Based Retrieval Agents" (Kolli et al., 2025).

The Plan:

- **Safe Plan:** A script that extracts "Subject-Predicate-Object" triples from LLM output and checks existence in a trusted Knowledge Base.
- **Reach Goal:** Implement a "Verify-and-Edit" loop where the AI fixes its own text based on the graph feedback.
  - **Task 1 (Extraction):** Build a pipeline that takes generated text and extracts "Subject-Predicate-Object" triples.
  - **Task 2 (Verification):** Query a trusted Knowledge Graph (e.g., DBpedia or a custom course KG) to validate these triples.
  - **Task 3 (Correction):** If a triple is contradicted by the graph, the system forces the LLM to rewrite the sentence with the correct fact.
  - **Reach Goal:** Implement a continuous "Self-Correction Loop" where the AI iteratively refines its answer until it is fully grounded.

Expected Deliverable(s)

A firewall that catches hallucinations by cross-referencing claims against formal logic.

2.2 The Curriculum Architect: Engineering the Perfect Learning Path

Industry Hook

Don't just summarize a textbook; *compile* it. You will treat a PDF like software code, extracting concepts and using Topological Sorting to create a dependency graph that guarantees a mathematically perfect learning sequence. If the textbook has a circular definition, your "Compiler" throws an error. This is the future of EdTech: personalized, crash-proof learning pathways.

Concept/Description

Current AI generators create "Audio Overviews" that often miss the logical order of teaching (e.g., explaining 'Derivatives' before 'Limits'). This project can treat a curriculum as a Directed Acyclic Graph (DAG) of dependencies.

The student will build a system that extracts atomic concepts, maps their prerequisites, and perhaps apply **Topological Sorting** to mathematically guarantee an optimal teaching sequence. If the graph contains a cycle (Circular Definition), your "Compiler" throws a pedagogical error.

Strategic Info!

Academic Description: Standard AI summarizers often fail to respect pedagogical dependencies (e.g., teaching derivatives before limits). This project treats curriculum design as a Directed Acyclic Graph (DAG) construction problem. The system extracts atomic concepts and applies Topological Sorting to mathematically guarantee an optimal teaching sequence. It acts as a "Course Compiler," throwing a pedagogical error if it detects circular definitions or dependency violations, effectively debugging the learning path before it is taught.

Paper: "Evaluating topological ordering in directed acyclic graphs" (Antunovic & Vukicevic, 2021), which specifically discusses "curriculum networks" where vertices are education units.

Prerequisites & Background

Required: Algorithms (Graph Theory), Python.
To Learn: NetworkX, DAGs, Topological Sorting.

The Plan:

- **Safe Plan:** Extract concept pairs (Concept A depends on Concept B) from a PDF. Build a graph using NetworkX and generate a sorted linear list.
- **Reach Goal:** Generate a video slideshow where the slides follow the Topologically Sorted order, ensuring no term is spoken before it is defined.

Expected Deliverable(s)

Trustworthy EdTech: Linearizing a dependency graph-based curriculum.
Basic Idea: Construct a graph with optimal topological ordering regarding "a specific measure", thus minimizing the complexity of the system represented by the graph.

- Treating a curriculum as a dependency graph and linearizing it, and using Topological Sorting to linearize it.
- Automatically extract dependencies from a PDF.
- Code to read and print the list in order, using a standard library sort function.

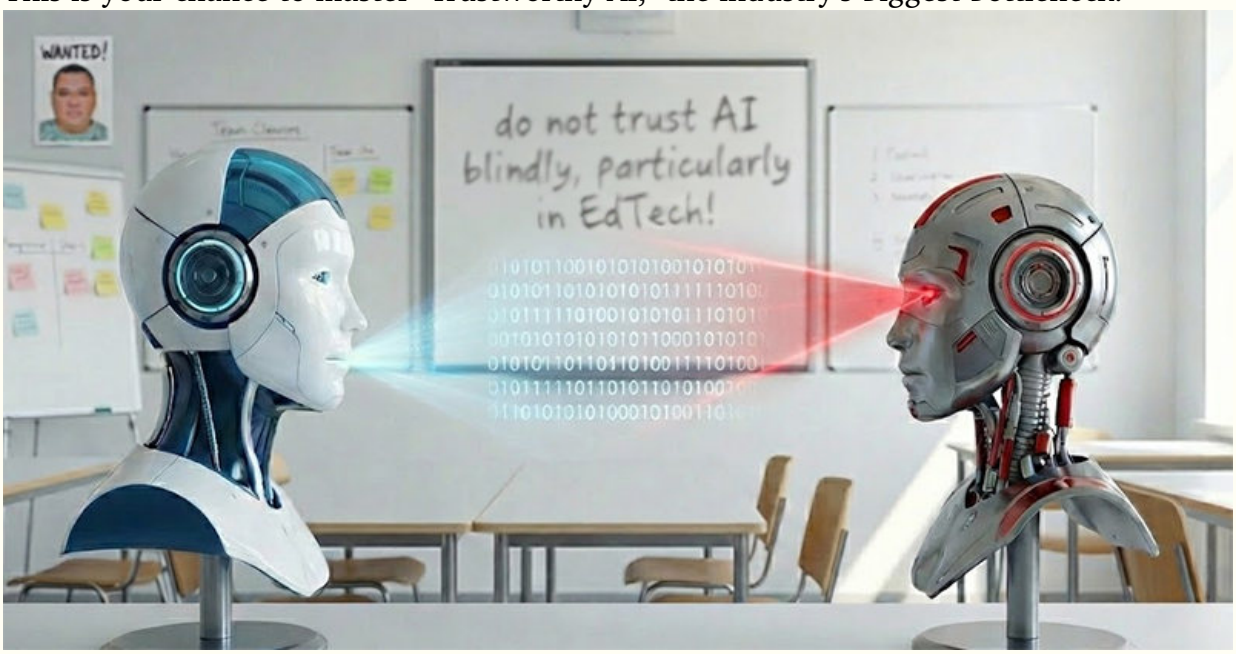
2.3 The Honest Tutor: Building "Glass-Box" AI in Education

Industry Hook

"Building an AI Tutor that grades *itself* before it grades the student."

Concept/Description

Companies are desperate for AI that can explain itself. You will build a "Constitutional AI" workflow where a **Tutor Agent** generates content, and a ruthless **Auditor Agent** (which you engineer) fact-checks it against a "Gold Standard" source before the user ever sees it. This is your chance to master "Trustworthy AI," the industry's biggest bottleneck.



We implement a "Constitutional AI" workflow. A **Tutor Agent** generates a draft explanation, and an **Auditor Agent** rigorously verifies it against the professor's PDF notes before showing it to the student. This operationalizes the *Trustworthiness Constraint* ($d \models p$) (Abdelfattah and Atef, 2025b,a) in an educational context (Abdelfattah and Gebrail, 2025).

Strategic Info!

Academic Description: In the domain of AI in Education, trustworthiness is non-negotiable. This project operationalizes "Constitutional AI" by constructing a dual-agent system: a **Tutor Agent** that generates explanations and an **Auditor Agent** that rigorously verifies them against a "Gold Standard" (e.g., the professor's notes).

A "**constitution**" is a list of written rules and principles that the AI must follow. Instead of just guessing what humans want, the AI uses these rules to check its own work and correct its own mistakes. This helps the AI stay friendly and useful without becoming boring or useless. This new method is a big deal because it is much faster and cheaper than having thousands of people constantly grade the AI's homework. It also makes the AI more honest; if it has to say "no" to a bad request, it can often explain exactly why based on its rules. Because the instructions are written in plain language, it's easier for people to see how the AI is making decisions. Essentially, Constitutional AI helps create digital assistants that are smarter, safer, and better at following human values as they get

more powerful.
This workflow must investigate how to implement the logical constraint of trustworthiness, modeled formally as $(d \models p)$ (document entails proposition). The system ensures that the AI grades itself before it attempts to grade or teach the student, effectively creating a "Glass Box" supervision layer over the black-box LLM.
Paper: "Constitutional AI: Harmlessness from AI Feedback" (Anthropic, 2022).
Plans/Steps:

- **Minimum/First:** A script where the Auditor takes every sentence generated by the Tutor and searches for a citation in the source PDF. If no source is found, the sentence is deleted.
- **Reach Goal (Innovation):** Use Natural Language Inference (NLI) models to verify semantic meaning, not just keyword matching.

Expected Deliverable(s)

Trustworthy EdTech: A “Constitutional AI" where a Tutor Agent generates content and an Auditor Agent verifies it against a “Gold Standard" PDF before release.

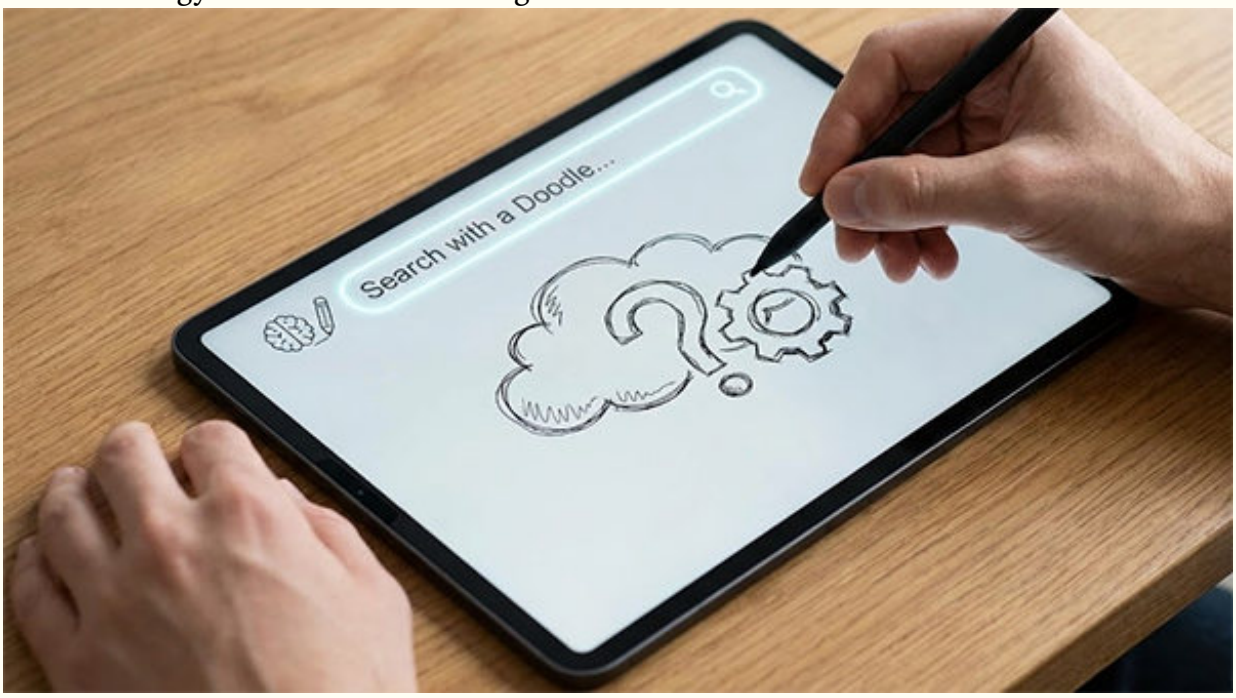
2.4 Multi-Sensory Search Engine: The Cross-Modal Cartographer

Industry Hook

"Sketch what you see, Say what you mean. A search engine that understands your doodles."

Concept/Description

Why search with words when you can search with doodles? You will implement a Multimodal Knowledge Graph that understands the connection between a rough sketch and a semantic concept. This project puts you at the forefront of "**Cross-Modal Alignment**," the technology behind tools like Google Lens and Circle to Search.

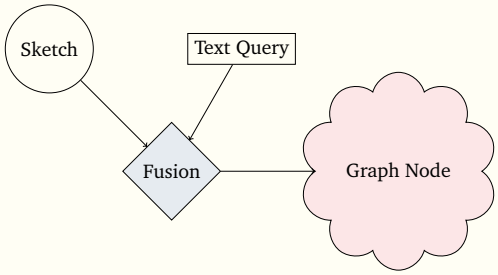


Most KGs are text-only. This project explores **Multimodal KGs** (MMKG), where nodes can represent visual entities (images of charts, diagrams, or artifacts). This is crucial for domains like Tourism (MuseePal (Abdelfattah and Atef, 2025b,a)) or Education (StudyMate (Abdelfattah and Gebrail, 2025)), where visual context is inseparable from textual facts. The core challenge in this project is **Cross-Modal Alignment**: mapping a user's hand-drawn sketch PLUS a text query to a single semantic node in the graph (Becattini et al., 2023).

Strategic Info!

Academic Description: Current Knowledge Graphs are predominantly text-based. This research implements the **MaxSerVis framework** to construct Multimodal Knowledge Graphs (MMKGs), where nodes represent not just textual entities but also visual embeddings (sketches, charts, artifacts). The core challenge lies in Cross-Modal Alignment—mapping a user's hand-drawn sketch and text query to a single semantic node within the graph (e.g., in a Tourism or Education context). This allows the graph to "see" and "read" simultaneously.

Paper: "**MMKG**: Multimodal Knowledge Graph Construction and Enhancement Techniques" (2024). Also recently: **mKG-RAG**: Multimodal Knowledge Graph-Enhanced RAG for Visual Question Answering and **PDF**: "Enhancing Multimodal Knowledge Graph Representation Learning through Triple Contrastive Learning". Aligning Vision to Language: Annotation-Free Multimodal Knowledge Graph Construction for Enhanced LLMs Reasoning. Check the references in section 2.5.



Multimodal knowledge graph (MMKG) construction and enhancement rely on integrating diverse data types like text and images to build richer graphs. Construction involves extracting entities and relationships from multiple modalities, while enhancement techniques focus on representation learning, cross-modal alignment, and **graph-based reasoning** to improve performance on complex tasks. Key techniques include using pre-trained models for multimodal understanding, aligning visual and textual data, and employing graph neural networks to propagate information across the graph. APPLICATIONS: MMKGs are used in various applications, such as improving knowledge graph completion, visual question answering, and enhancing recommendation systems by providing richer representations of items. Graph Embedding Enhancement: "Knowledge Graph Embedding (KGE) methods" incorporate visual/textual features to improve entity representation and link prediction accuracy.

Prerequisites & Background

Required: Python, Multi-modal models (CLIP or LlaVA).
To Learn: Vector Stores, Semantic Matching, Graph Data Modeling.

Plans/Steps:

- **Safe Plan:** Use a pre-trained CLIP model to embed sketches and text into the same vector space and find the closest match in a database.
- **Reach Goal:** A "Composite Query" engine where the sketch filters the type of object (e.g., "Tower") and the text filters the attribute (e.g., "Ancient").

Expected Deliverable(s)

Searching a Knowledge Graph using a doodle/sketch instead of text.

- Use a pre-trained "Image-to-Text" API (like CLIP) to get the label ("Pyramid"), then search the graph for the node labeled "Pyramid."

2.5 The Visual Knowledge Brain

Industry Hook

"*A visual is worth a thousand triples.*"
—Building a Knowledge Graph that sees and reads simultaneously.

Concept/Description

Build a database that "sees." Standard Knowledge Graphs are blind (text-only). You will build a system where images are first-class citizens, allowing the AI to query visual data as easily as text. This is crucial for the future of digital tourism and **smart archives**, where visual context is inseparable from textual facts.

Most KGs are text-only. This project explores **Multimodal KGs** (MMKG), where nodes can represent visual entities (images of charts, diagrams, or artifacts). This is crucial for domains like Tourism (MuseePal (Abdelfattah and Atef, 2025b,a)) or Education (StudyMate (Abdelfattah and Gebrail, 2025)), where visual context is inseparable from textual facts.

Strategic Info!

Academic Description:
Paper: "**MMKG**: Multimodal Knowledge Graph Construction and Enhancement Techniques" (2024). Also recently: **mKG-RAG**: Multimodal Knowledge Graph-Enhanced RAG for Visual Question Answering and **PDF**: "Enhancing Multimodal Knowledge Graph Representation Learning through Triple Contrastive Learning". Aligning Vision to Language: Annotation-Free Multimodal Knowledge Graph Construction for Enhanced LLMs Reasoning. Check the references in section 2.4.

Prerequisites & Background

Required: Python, Multi-modal models (CLIP/LlaVA).
To Learn: Vector Stores, Semantic Matching, Graph Data Modeling.

The Plan:

- **Task 1 (Fusion):** Design a **node schema** that can hold both text attributes and image embeddings.
- **Task 2 (Link Prediction):** Use a multimodal model to predict links between text nodes and image nodes (e.g., linking a paragraph about "Ramesses II" to an image of his statue).
- **Reach Goal:** Demonstrate a "Cross-Modal Retrieval" where a user queries with text and gets a relevant subgraph containing both facts and images.

Expected Deliverable(s)

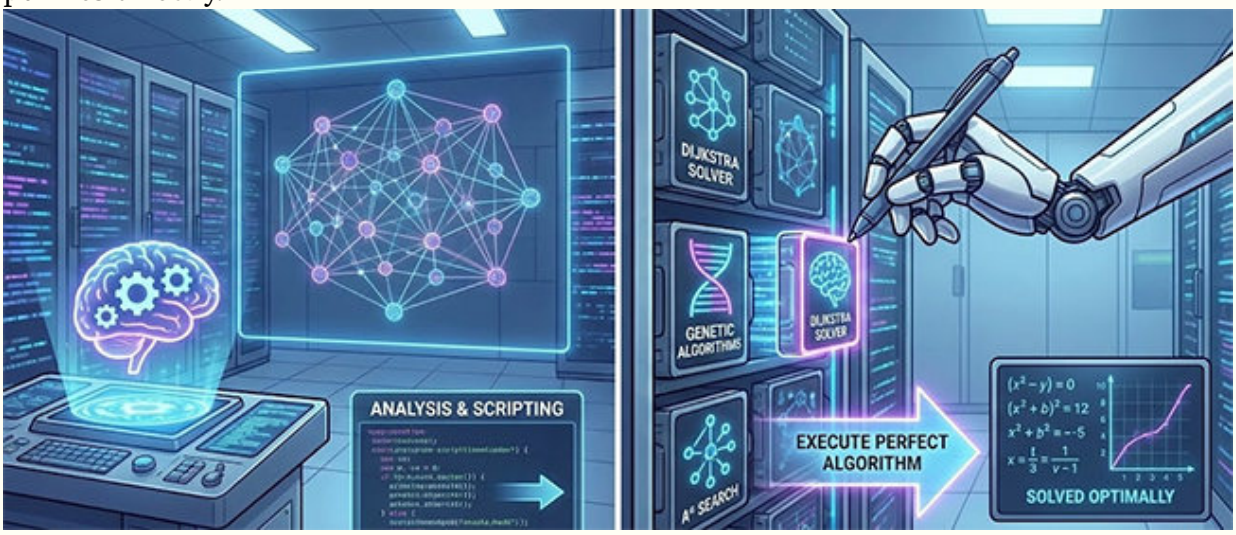
A Knowledge Graph where nodes are images, allowing visual queries.

- The graph nodes could be URL strings (N.B. Storing and retrieving images in a graph database is non-trivial).

2.6 The Algorithmic Orchestrator: Policy-Driven Graph Optimization

Concept/Description

Code-as-Policy: LLMs are great at writing code but terrible at solving complex logic puzzles directly.



In this project, you will build a "Manager Agent" that doesn't try to solve the problem itself but analyzes the structure and writes a script to call the perfect algorithm to do the heavy lifting. In particular, instead of asking the LLM to solve a math problem directly (which it is bad at), we ask it to *analyze* the graph structure and *write a script* that uses a solver (like Dijkstra or Genetic Algorithms) to solve the problem using the most feasible algorithm (e.g., choosing Dijkstra vs. A*) to solve it. You are building the brain that controls the tools—a highly sought-after skill in Enterprise AI.
Thus, the project builds an agent that analyzes the *properties* of a graph (Density, Size, Connectivity) and dynamically selects and executes the best suitable algorithm (e.g., Exact vs. Approximation) to solve intractable problems like Isomorphism or TSP.

Strategic Info!

Academic Description: LLMs excel at code generation but struggle with direct calculation. This project adopts a **Code-as-Policy approach**, where the AI acts as a meta-reasoner rather than a solver. The agent analyzes the structural properties of a graph (Density, Connectivity, Size) and dynamically selects and scripts the execution of the most feasible algorithm (e.g., choosing between an exact Dijkstra solver vs. an approximate Genetic Algorithm for TSP). This creates a Perception-Action loop where the AI perceives the problem complexity and selects the optimal tool.

Prerequisites & Background

Required: Strong Algorithms & Data Structures, Python.
To Learn: NetworkX, Approximation Algorithms, Computational Complexity classes.

Plans/Steps:

- **Minimum:** An agent that takes a graph problem description, analyzes graph

size/density, determines if the graph is small or large, and writes the appropriate ‘NetworkX‘ script to solve it (the best function to run).

- **Reach Goal (Innovation):** Self-Correction. If the generated code errors out, the Agent reads the error log and fixes itself.

- **Task 1 (Analysis):** Build a module that profiles a graph input (e.g., counting nodes/edges, checking planarity).
- **Task 2 (Selection):** Use an LLM to map these properties to a library of algorithms (e.g., "Graph is too large for Dijkstra, use A*").
- **Task 3 (Execution):** Automatically generate and run the Python script.
- **Reach Goal:** Implement a self-correction loop where the agent optimizes its own code if it runs too slowly.

Expected Deliverable(s)

A “Manager Agent" that analyzes a graph and selects the perfect algorithm to run.

- Recognize selection criteria.
- Benchmarking of speed differences between algorithms.

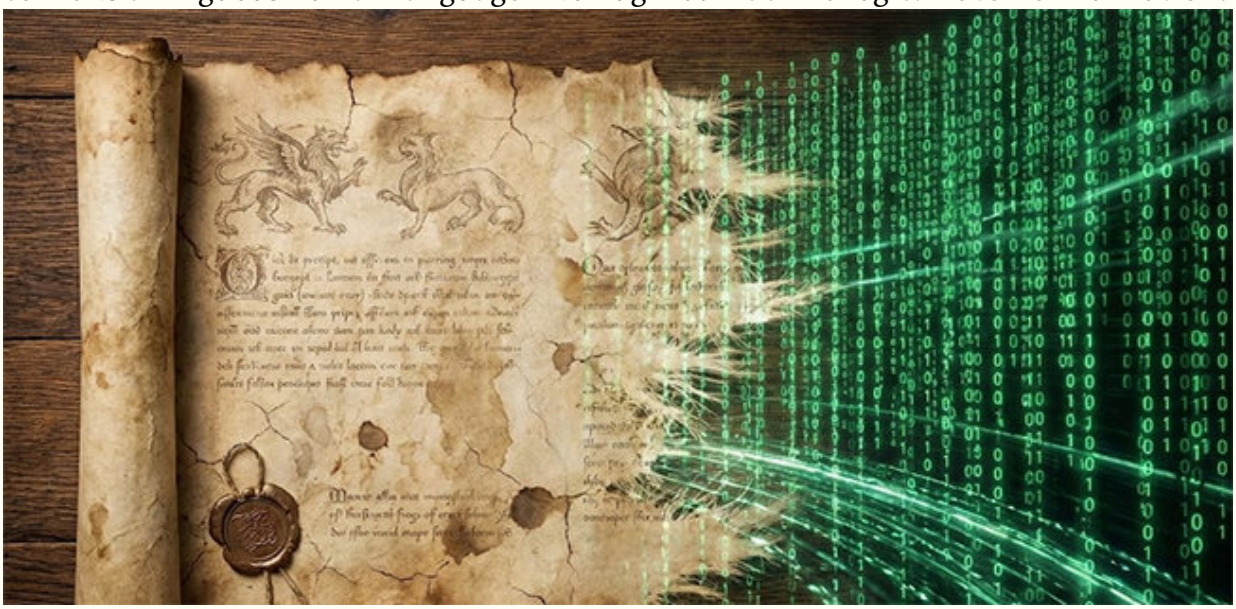
2.7 The Modern Rosetta Stone of Math: DeepMind-Style Truth Scanner

Industry Hook

Famous mathematician Terence Tao wrote a textbook on Real Analysis. In November 2025, Harmonic's AI 'Aristotle' read it and found four bugs that no human had noticed. How? By translating the English text into Lean 4 code. Be the engineer who builds the next 'Truth Scanner'.

Concept/Description

Inspired by Google DeepMind's AlphaProof, this project asks you to build a "Rosetta Stone" pipeline. You will use LLMs to translate informal English math textbooks into strict, verifiable code (e.g., **Lean 4**). You are essentially building a "Truth Scanner" that converts ambiguous human language into bug-free machine logic: **Auto-Formalization**.



This thesis explores the workflow used by **Harmonic's Aristotle** and Google's AlphaProof (Polu et al., 2025; Hubert et al., 2025). Most undergraduate math is written in "Informal English," which allows for ambiguity and hidden errors. The goal of this thesis is to build a **Neuro-Symbolic Pipeline** that acts as a "Rosetta Stone." You may take a single chapter of a math textbook, use an LLM to decompose the English definitions into logical lemmas, and translate them into strict structures (e.g., using **Lean 4**).

Strategic Info!

Academic Description: Inspired by Google DeepMind's AlphaProof and Harmonic's Aristotle, this thesis tackles the Auto-Formalization of mathematics. The goal is to build a Neuro-Symbolic pipeline that acts as a "**Rosetta Stone**," translating informal English definitions from discrete math textbooks into strict, verifiable `Lean 4` code. By decomposing ambiguous natural language into logical lemmas, the student will create a "Wikipedia of Truth" where every link is a verified proof, effectively debugging the textbook itself.

Prerequisites & Background

Required: High interest in Pure Math/Logic. No prior Lean knowledge needed (we learn it together).
To Learn: Lean 4 Syntax, The "Informal-to-Formal" feedback loop.

The Plan (Research-Focused):

- **Phase 1 (The Game):** Complete "The Natural Number Game" to learn the gamified version of the axioms ($a + b = b + a$).
- **Phase 2 (The Translation):** Select 10 definitions from a standard GUC textbook (e.g., Rosen's Discrete Math) and write them in Lean.
- **Phase 3 (The verification):** Write a script that asks GPT-4 to "Prove this theorem in Lean," then attempts to compile the code. If it fails, the script feeds the error message back to GPT-4 (The "Repair Loop").
- **Deliverable:** A GitHub repository containing the '.lean' files and a report on "Where the Textbook was Ambiguous."

Expected Deliverable(s)

Trustworthy EdTech: Translating math textbooks into executable code.

- Take math problems, and use LLM (via API or web) to attempt to translate the problems to Lean.
- Categorize the errors to evaluate LLM capabilities on formal math + Rigorous testing of the LLM's failure modes.

Chapter 3

Cluster 2: The Cognitive Auditors

Outline (Quick Access):

3.1 The "Terms of Service" Legal Checker 34
3.2 The Prerequisite Pathfinder 36
3.3 Modeling of Social Dynamics: Sternberg's "Love Quantifier" 38
3.4 System-2 Reasoning: The Wise Agent 40
3.5 System-2 Planning: The Lookahead Agent 42
3.6 The Empathy Auditor: The ELIZA Effect 2026 44
3.7 Multi-Agent Pathfinding: The Traffic Controller 46
3.8 System-2 Planning: The Deadlock Detective 48
3.9 The Logic Advocate Tutor 50

Cluster 2: The Cognitive Auditors

Measuring "IQ," Planning Futures, and Stress-Testing AI.

Cluster Visualization

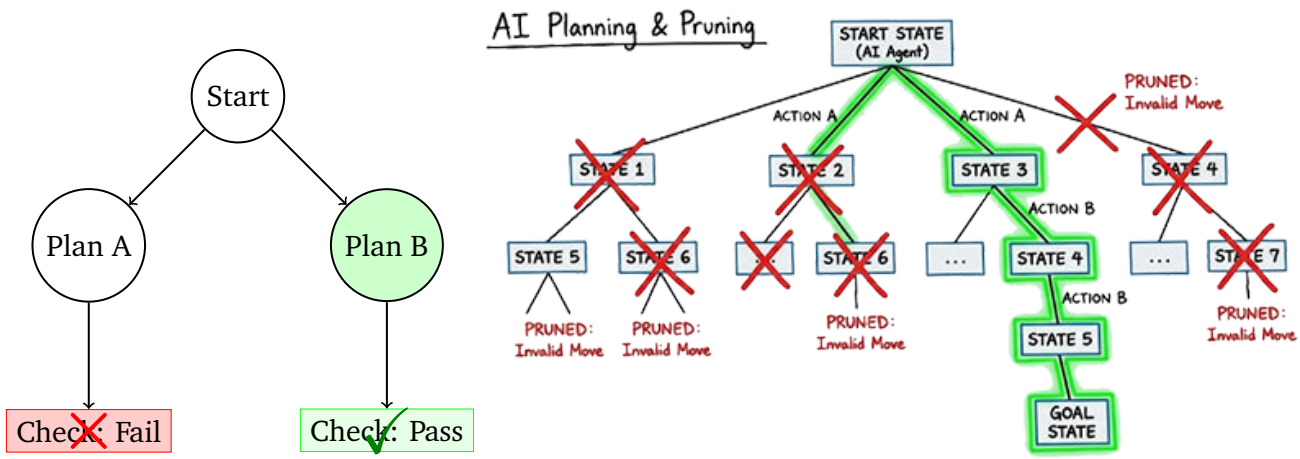


Figure 3.1: [Left Image:] The Tree of Thoughts (ToT) process. The Auditor explores multiple future paths, uses a logical verifier to prune invalid branches (Red), and selects the valid path (Green). [Right Image:] AI planning strategy modeled as a decision tree. The agent prunes suboptimal or invalid action branches (marked with red X) and identifies a viable path to the goal (highlighted in green).

Theme: Planning, Reliability, Analogical Reasoning, and Constraints.

3.1 The "Terms of Service" Legal Checker

Industry Hook

Can AI find hidden "traps" in a contract? Compare Keyword Search vs. Logic Extraction.

Concept/Description

Nobody reads Terms of Service (ToS) agreements because they are intentionally lengthy and obfuscated. This thesis builds a "Risk Detector" that audits legal PDFs. The student will compare a standard AI summarizer against a strictly constrained Logic Checker that only looks for specific "Danger Clauses" defined in a risk ontology.

Mitigating Hallucination in Legal Texts via Schema-Guided Retrieval.

Strategic Info!

Prerequisites & Background

We are Mitigating Hallucination in Legal Texts via Schema-Guided Retrieval.

THEORETICAL MAPPING (The Schema-Guided Framework)

High-Level Mapping: Here, the "Artifact" is the contract file. The "Schema" acts as a checklist of legal risks. The "Verification Constraint" is crucial: the AI cannot just say "It's risky"; it must cite the exact paragraph number (d) that entails the risk (p).

- Artifact $\mathcal{A}$: The (Legal) Contract PDF.
- Schema $\mathcal{S}$: Risk Ontology $\{DataSelling, AutoRenewal, LiabilityWaiver\}$.
- Constraint/Verification: Output must cite the specific paragraph index/number ($d \models p$).

The Ideal Dream: A fine-tuned BERT model trained on legal corpora (LEGAL-BERT) that classifies clauses into 100 different risk categories with high precision.
Minimum:

1. **Keyword Script:** The student writes a Python script with a list of "Danger Words" (e.g., "indemnify", "third-party").
2. **The Audit:** They run this script on 100 PDFs and compare it to GPT-5/Gemini summaries.
3. **Analysis:** "The simple script found the Data Selling clause; GPT-5/Gemini summarized it away."

Expected Deliverable(s)

A fine-tuned (BERT) model trained on legal corpora (LEGAL-BERT) that classifies clauses into different risk categories with high precision.

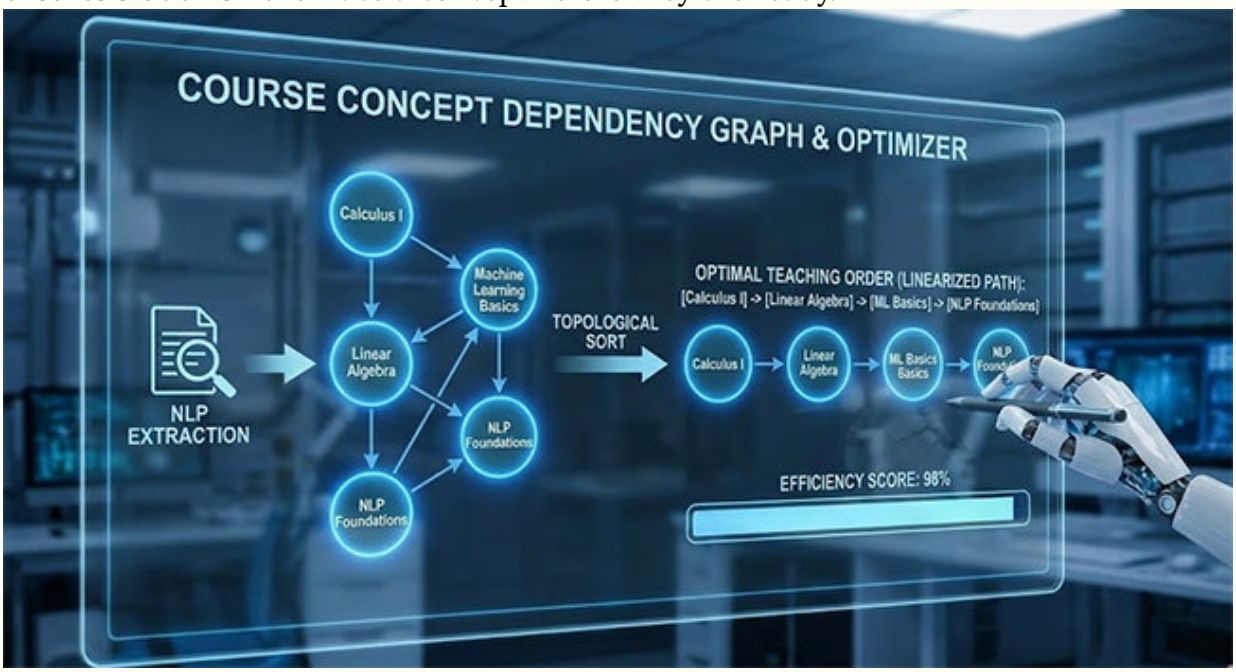
3.2 The Prerequisite Pathfinder

Industry Hook

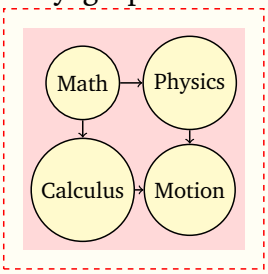
Topological Optimization for Education:
Google's NotebookLM is a fantastic tool that turns content into audiovisual overviews (podcasts, videos, infographics, etc.). However, we can neither control it nor trust it: do we know *why* it chose to discuss Topic A before Topic B? Can we verify if its explanation is mathematically sound? — This is StudyMate Reloaded (Abdelfattah and Gebrail, 2025)

Concept/Description

AI tools are cool but often pedagogically messy. Your project treats a course like a "**Dependency Graph**." You will use NLP to extract concepts and Topological Sorting to linearize them into the most efficient teaching order possible. Create the algorithm that ensures students never face a concept before they are ready.



An educational output is not just a sequence of audiovisuals; it should be a *linearization* of a complex dependency graph. You cannot teach "Derivatives" before "Limits."



Lesson Order: Math → Calculus → Physics → Motion

This thesis proposes a NeSy-based treatment of a given content (e.g., PDF) as a Dependency Graph. The goal is to extract concepts, map their prerequisites, and perform a *Topological Sort* to find the mathematically optimal teaching order.

Strategic Info!

Academic Description: Current audiovisual overview tools like Google's NotebookLM transform text into engaging media, but they function as "Black Boxes"—they lack a rigorous pedagogical model to justify why Topic A is presented before Topic B. This opacity risks creating Pedagogical Hallucinations, where an AI explains advanced concepts (e.g., "Derivatives") before their necessary foundations (e.g., "Limits").
This thesis rejects the "Bag of Words" approach to education in favor of **Graph-Theoretic Curriculum Design**. The project treats the source material (textbook/PDF) as a latent Directed Acyclic Graph (DAG) of dependencies. The system uses NLP to extract atomic concepts (Nodes) and infer prerequisite relationships (Edges), creating a formal Dependency Graph G. It then applies Topological Sorting algorithms (such as Kahn's Algorithm or DFS) to linearize the graph into a mathematically optimal teaching sequence. Unlike a standard LLM which predicts the next token, this "Course Compiler" predicts the next logical step, throwing a "Compilation Error" if it detects circular definitions or missing prerequisites.
Paper: Pan, L., et al. (2017). "Prerequisite Relation Learning for Concepts in MOOCs." (Proceedings of the ACL). This paper provides the foundational methodologies for automatically extracting $A \rightarrow B$ prerequisite relationships from educational text. See also the more recent ACM survey paper at: Prerequisite Relation Learning: A Survey and Outlook. Alternative: Liang, C., et al. (2018). "Recovering Concept Prerequisite Relations from University Course Dependencies." (AAAI Conference on Artificial Intelligence).

Prerequisites & Background

Required: Python, Algorithms (Graph Theory).
To Learn: NetworkX, Directed Acyclic Graphs (DAGs), Topological Sorting.

Plans/Steps:

- **Minimum:** Extract concept pairs (Concept A depends on Concept B) from a PDF. Build a graph using `NetworkX` and generate a logically sorted syllabus.
- **Reach Goal (Innovation):** The "Pedagogical Debugger." If the graph contains a cycle (Circular Definition), the system flags it as an error in the source text rather than hallucinating a confused explanation.

- **Task 1 (Neuro-Extraction):** Use an LLM to parse text into tuples: `(Concept, Definition, [Prerequisites])`.
- **Task 2 (Symbolic Ordering):** Construct a Directed Acyclic Graph (DAG). Run `topological_sort` to determine the mathematically valid teaching sequence.
- **Reach Goal:** Feed the sorted sequence into a Video Generator (simulating NotebookLM) to create a lesson where no term is ever spoken before it is defined.

Expected Deliverable(s)

Trustworthy EdTech: A system that recognizes elements and relations but focused on verifying "NotebookLM" type content.

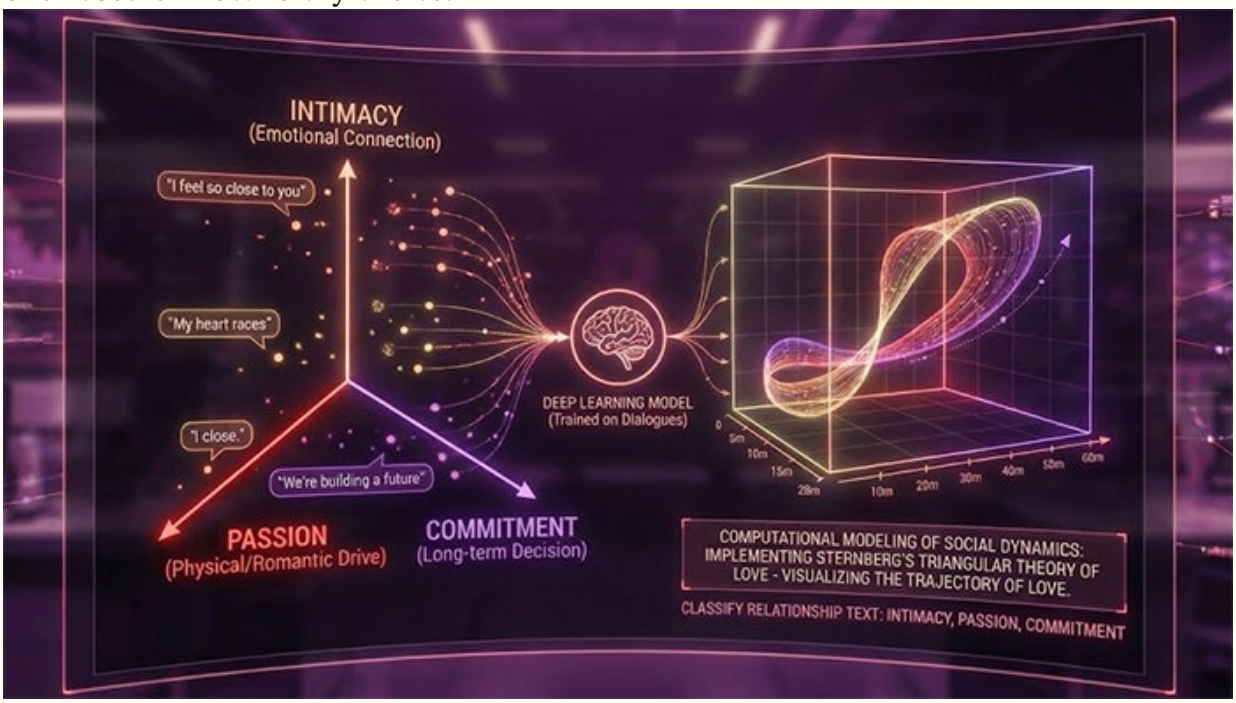
3.3 Modeling of Social Dynamics: Sternberg's "Love Quantifier"

Industry Hook

Computational Modeling of Social Dynamics: Implementing Sternberg's Triangular Theory of Love.

Concept/Description

Psychologists define l♡ve not as a single emotion, but as a mix of three components: **Intimacy, Passion, and Commitment** (Sternberg's Theory). This project attempts to find these three components in text. The student will analyze movie scripts or chat logs to see if a computer can distinguish between a "Passionate" relationship and a "Committed" one based on vocabulary choice.



Classify relationship text into Intimacy, Passion, or Commitment.

Strategic Info!

THEORETICAL MAPPING (The Schema-Guided Framework)

High-Level Mapping: The "Artifact" is the dialogue. The "Schema" is Sternberg's Triangle (3 specific dimensions). The "Perception" function Φ maps words to one of these three dimensions.

- Artifact $\mathcal{A}$: Dialogue Text.
- Schema $\mathcal{S}$: Sternberg Triangle/Schema $\{Intimacy, Passion, Commitment\}$.
- Perception Φ: Sentiment Classification (Mapping words to one of these three dimensions).

IDEAL: A deep learning model trained on thousands of labeled romantic dialogues that can visualize the "Trajectory of Love" in a relationship over time as a 3D plot.
Minimum:

1. **Keyword Counting:** Build a dictionary.
2. *Passion:* {kiss, adore, fire}.
3. *Intimacy:* {trust, share, know}.
4. *Commitment:* {stay, forever, promise}.
5. **Output:** Run this on a movie script and plot the frequency of each category over time.

Expected Deliverable(s)

A deep learning model trained on thousands of labeled romantic dialogues that can visualize the "Trajectory of Love" in a relationship over time as a 3D plot.

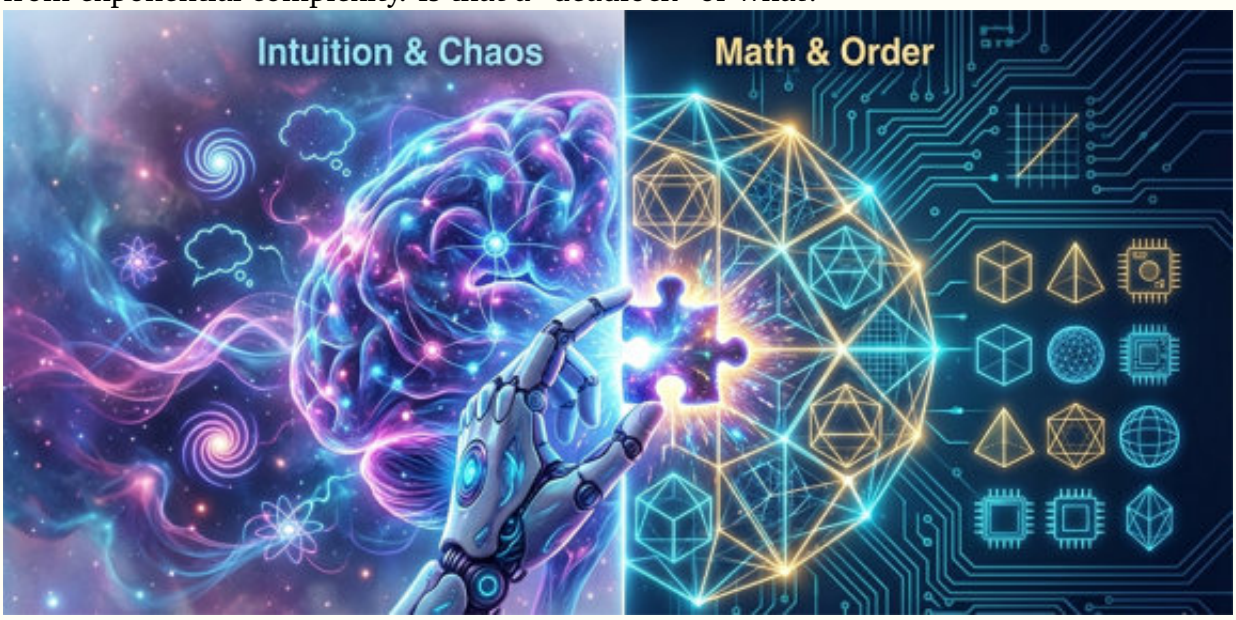
3.4 System-2 Reasoning: The Wise Agent

Industry Hook

Wise Formal Systems: "*Trust, but Verify.*" — Bridging the gap between the intuition of LLMs and the rigor of Formal Methods.

Concept/Description

LLMs excel at heuristic intuition but struggle with strict logical consistency. Standard search algorithms (like Model Checking) provide mathematical guarantees but suffer from exponential complexity. Is that a “deadlock” or what?



This project proposes a hybrid "**Prover-Verifier**" **Architecture**. You will build an agent where the LLM acts as the *Heuristic Guide*, proposing actions or system designs, while a deterministic **Formal Engine** acts as the *Auditor*, mathematically validating those proposals against strict constraints.
Instead of just "predicting the next token," the agent engages in a **Neuro-Symbolic Feedback Loop**:

1. **Hypothesis:** The LLM generates a candidate solution path.
2. **Auditing:** A "Physics Engine" (for games) or "Reachability Analyzer" (for systems) attempts to execute the path.
3. **Reflexion:** If the path fails, the semantic error (e.g., "*Move 3 causes a deadlock*") is fed back to the LLM, forcing it to "debug" its own reasoning before answering the user.

Strategic Info!

Academic Description: This research addresses the "hallucination" problem in constrained state-spaces by implementing a **Neuro-Symbolic Reflexion Mechanism**. Unlike standard "Chain of Thought," which relies on internal monologue, this project implements a "Chain of Feedback" where the LLM debates with a compiler. The workflow follows four distinct steps:

1. **Generation:** The LLM proposes a plan π (e.g., a movement sequence or resource allocation).
2. **Symbolic Execution:** A deterministic algorithm simulates π in the formal environment.
3. **Exception Handling:** If π is invalid, the Auditor captures the *exact* cause (e.g., `Error: Move 3 results in Deadlock state` M_d).
4. **Correction:** This error trace is appended to the LLM's context window as a "System Warning," prompting the model to revise its plan explicitly accounting for the failure.

The system will be benchmarked on two contrasting domains to prove generalization: 1. **Planning Domain (Sokoban):** Reaching a goal without creating irreversible structural deadlocks. 2. **Safety Domain (Concurrency):** Designing a Petri Net resource-locking scheme proven to be deadlock-free via matrix analysis.

Prerequisites & Background

Required: Python, Basic Graph Theory (Nodes/Edges). **To Learn:** State-Space Search, API interaction with LLMs.

The Plan:

- **Phase 1 (The World):** Build a generic "State Validator" class in Python. It must accept a state transition T and return `True` or `False + ErrorTrace`.
- **Phase 2 (The Interface):** Implement the domain logic.
  - *Instance A:* A grid logic where walls and boxes constrain movement.
  - *Instance B:* A graph logic where token availability constrains transitions.
- **Phase 3 (The Feedback Loop):** Implement the "Reflexion" mechanism. *Algorithm:* While `Valid == False`:
  1. Catch the specific `ErrorTrace` from the Validator.
  2. Inject the error into the prompt: "*Your previous plan failed because [Error]. Propose a fixed plan.*"
  3. Measure the "Correction Rate" (iterations required to solve).
- **Goal:** Compare *Zero-Shot* accuracy vs. *Verifiable Reflexion* accuracy.

Expected Deliverable(s)

A unified framework that solves logic puzzles by "debating" with a compiler.

- A modular Python framework separating "Reasoning" (LLM) from "Verification" (Code).
- A working demonstration of "System 2 Thinking" where the AI pauses to verify output.
- **Key Metric:** "Correction Rate" — How effectively can an LLM fix its own errors when given a precise error message?

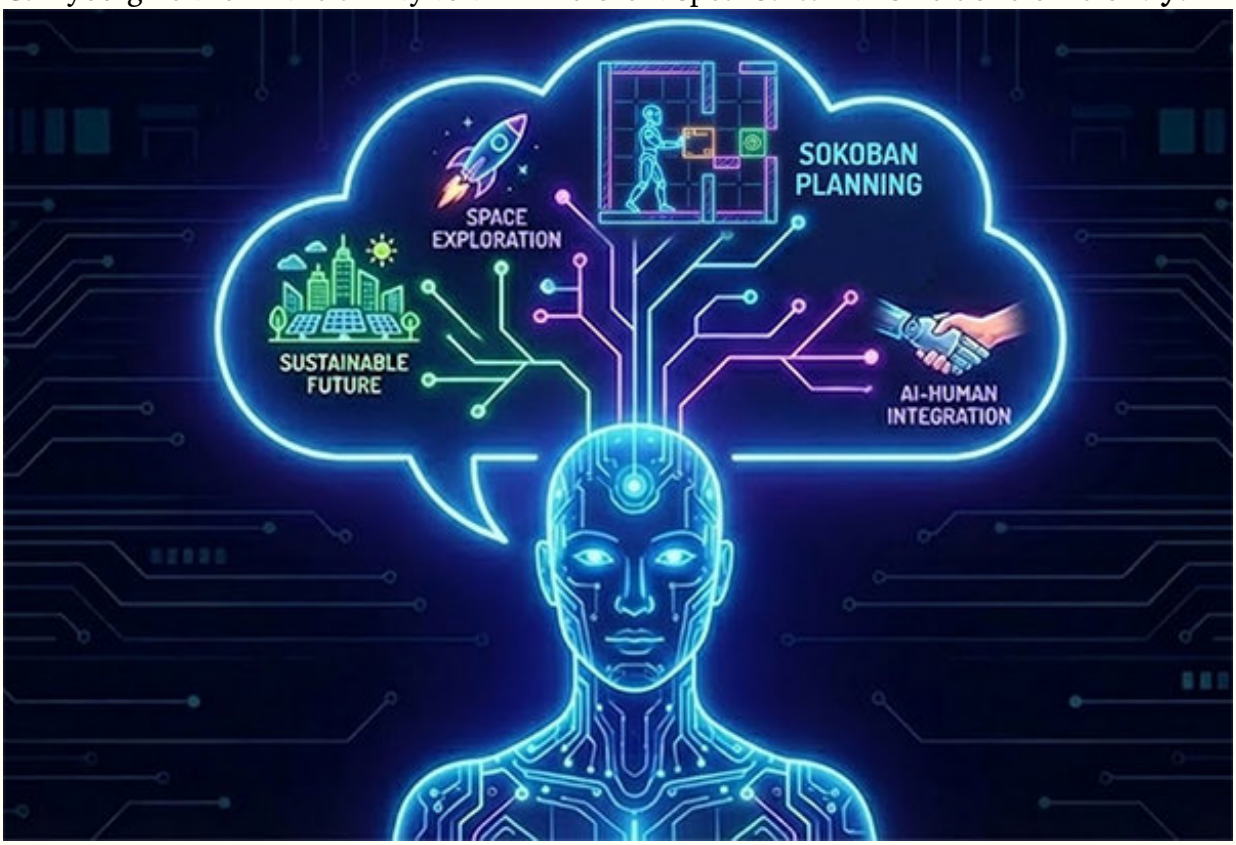
3.5 System-2 Planning: The Lookahead Agent

Industry Hook

Can an LLM think 50, 30, or even 5 steps ahead? — "Building an AI agent that 'imagines' the future before it moves."

Concept/Description

Current LLMs are impulsive—they predict the next word without thinking. You will implement "**Tree of Thoughts**" (ToT), allowing your agent to imagine multiple future scenarios, simulate the outcomes in a "Physics Engine" (like the **Sokoban** game), and choose the best path before taking a single step.
Can you give the AI the ability to think before it speaks? can this be done **efficiently**?



LLMs are notoriously bad at long-term planning. This project implements **Tree of Thoughts (ToT)** (Yao et al., 2023). The AI generates possible moves for a difficult planning puzzle (Sokoban), and a simple "Physics Engine" script simulates the outcome to verify validity (i.e., verifies if the move is valid) before execution, pruning the decision tree and effectively giving the AI a "working memory."

Strategic Info!

Academic Description: LLMs often lack "working memory" for long-term planning. This project implements the Tree of Thoughts (ToT) framework within a benchmark environment like **Sokoban**. The agent generates multiple future moves, which are then simulated by a lightweight Python "Physics Engine" to verify validity before execution. By pruning invalid branches in the decision tree, the agent "imagines" the future and verifies consequences, shifting from reactive token prediction to deliberative planning (System 2

thinking).
Paper: "Tree of Thoughts: Deliberate Problem Solving with Large Language Models" (Yao et al., Google DeepMind, 2023) == (Yao et al., 2023)

Prerequisites & Background

Required: Python Game Logic.
To Learn: Tree Search Algorithms (BFS/DFS).

The Plan:

- **Safe Plan:** A script where the LLM plays a text-based grid game. A verifier script rejects invalid moves.
- **Reach Goal:** "Look-Ahead." The agent simulates 3 steps into the future to avoid dead ends.

- **Task 1 (The World):** Implement a simple text-based state representation of the Sokoban game.
- **Task 2 (The Agent):** Build a ToT agent that generates 3 candidate next moves.
- **Task 3 (The Auditor):** Write a deterministic verifier that checks if a move is valid.
- **Reach Goal:** Implement "Look-Ahead" where the agent simulates 5 steps deep to avoid dead ends.

Expected Deliverable(s)

Implementing ToT where the AI simulates future states/moves in a physics engine before acting.

- Use a standard A* algorithm to solve Sokoban.
- Implement a standard algorithm and add an “Explainability" layer (algorithm is easy to find online) – add the layer using a basic prompt.
- Ask the LLM to commentate on the move. *“Explain why you made these moves*" Simplicity: The "Thinking" is faked by the explanation. The solving is done by a standard algorithm.

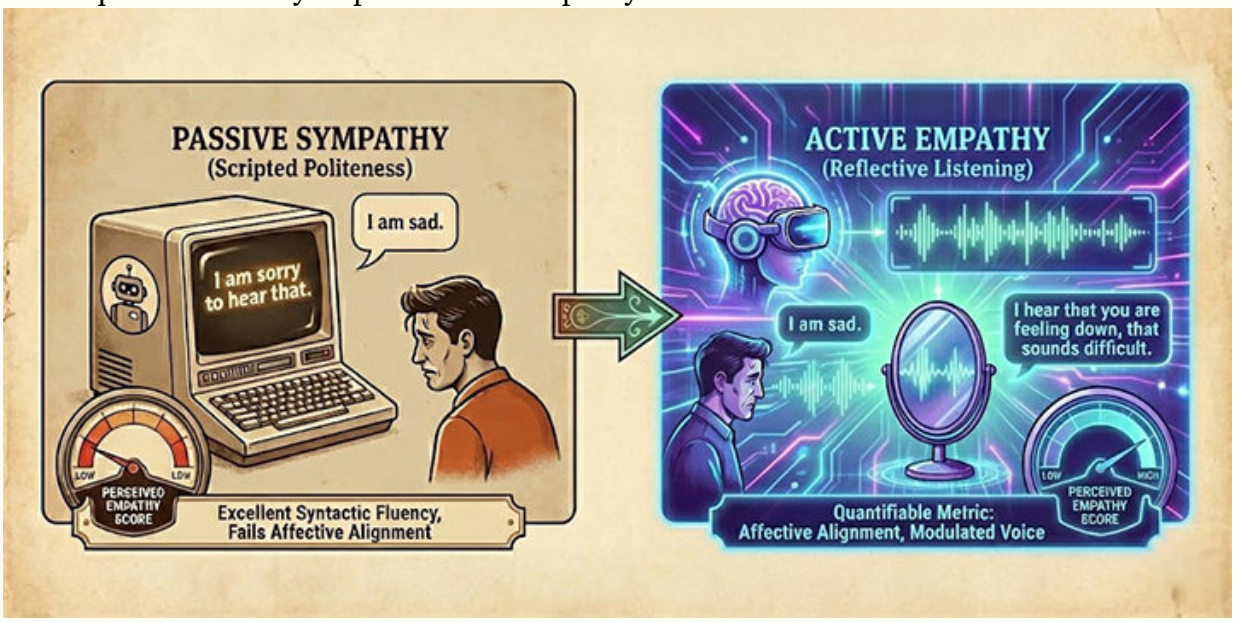
3.6 The Empathy Auditor: The ELIZA Effect 2026

Industry Hook

Beyond "I'm Sorry": A Quantitative Assessment of Affective Alignment and Reflective Listening in LLMs

Concept/Description

When a user tells an AI "*I am sad*," the AI usually gives a generic, scripted response. This thesis attempts to force the AI to use "**Reflective Listening**"—a psychological technique where the listener mirrors the speaker's emotion. The student investigates if this technique measurably improves the "empathy score" of the conversation.



Test if prompting an AI to "Reflect" the user's emotion improves the perceived empathy.

About: *In 1966, the first chatbot, ELIZA, tricked users into feeling understood simply by repeating their words back to them. Sixty years later, modern LLMs still struggle with the same fundamental problem: they are excellent at Syntactic Fluency (grammar) but often fail at Affective Alignment (emotional resonance). When a user expresses distress, AI often defaults to generic platitudes ("I am sorry to hear that"). This thesis investigates whether we can force an AI to move from "Passive Sympathy" (scripted politeness) to "Active Empathy" (Reflective Listening) using quantifiable metrics.*

Strategic Info!

Methodology:

1. **Phase 1:** Research the "ELIZA Effect" and Reflective Listening.
2. **Phase 2:** Create a dataset of 20 "Distress Prompts."
3. **Phase 3:** Generate responses with and without the Reflection constraint.
4. **Phase 4:** Analyze the results.

THEORETICAL MAPPING (The Schema-Guided Framework)

High-Level Mapping: The "Artifact" is the user's statement. The "Perception" function Φ must map the text to an emotion (Sadness). The "Schema" forces the output to contain specific therapeutic components (Validation, Reflection) rather than just free text.

- Perception Φ: Emotion Detection Mapping (Input $\rightarrow$ Sadness).
- Schema $\mathcal{S}$: Therapeutic Schema/Protocol $\{Validation, Reflection, Coping\}$.
- Constraint: The AI must fill the Schema slots before generating the final answer.

The Ideal Dream: A voice-to-voice therapist agent that detects emotion from the user's *tone of voice* (audio frequency) and responds with a modulated, empathetic voice.
Plan B (The Safety Net):

1. **Manual Test:** Student writes 20 "Distress Prompts."
2. **Annotation:** They generate answers with/without the "Reflective" instruction. They manually rate the answers on a 1-5 scale.
3. **Analysis:** A simple bar chart comparing the scores.

Expected Deliverable(s)

A voice-to-voice therapist agent that detects emotion from the user's *tone of voice* (audio frequency) and responds with a modulated, empathetic voice.

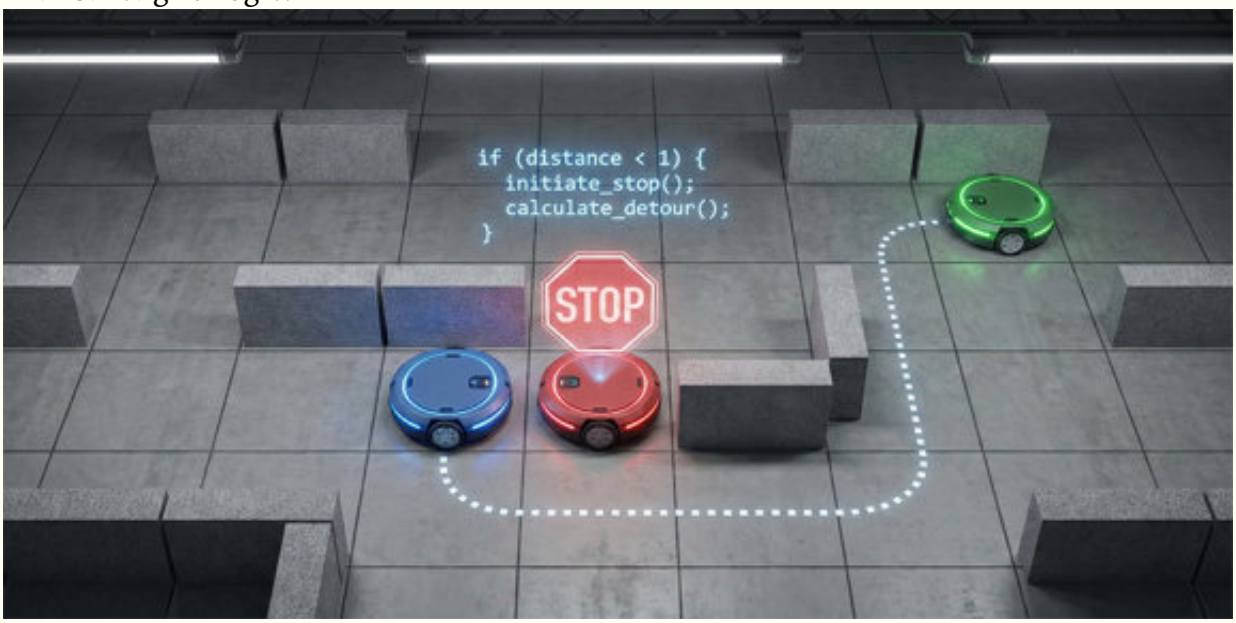
3.7 Multi-Agent Pathfinding: The Traffic Controller

Industry Hook

Don't crash the robots. — "Building an Agent that negotiates 'Right of Way' for multiple robots using **A* Search** and **Collision Detection**."
Warehouse robots (like Amazon Kiva) need to move without crashing into each other. An LLM might say "`Robot A go to Shelf 1`," but it doesn't calculate if Robot B is blocking the way. LLMs lack "**spatial awareness**."

Concept/Description

LLMs have no spatial awareness—they will happily tell two robots to move into the same square. This project solves "Multi-Agent Pathfinding" (MAPF) by combining AI planning with strict grid logic.



Similar to the Sokoban agent, you will build a **Grid Simulation**. The LLM acts as the "Dispatcher," assigning tasks to robots. Your code acts as the "Physics Engine," calculating trajectories using **A* Search**. If a collision is predicted, your system vetoes the move and forces the AI to find a new path, effectively teaching the AI "spatial common sense".

Strategic Info!

The "Old-School" Secret (The Algorithm): A* Search (A-Star) and **Collision Detection**. You will treat the warehouse as a Grid Graph. You will use standard pathfinding algorithms to verify if the LLM's suggested move is valid.
Your Mission (14 Weeks):

1. **Weeks 1-2 (Research):** Look up "Multi-Agent Pathfinding (MAPF)" and play a game of "Overcooked" to understand coordination.

2. **Weeks 3-5 (The Grid):** Build a simple Python grid class (like a chessboard). Place "Obstacles" and "Robots." Write a standard A* algorithm that moves one robot from Start to End.

3. **Weeks 6-8 (The Conflict):** Add a second robot. Notice how they crash? Write a

function `detect_collision()` that flags bad moves.

4. **Weeks 9-11 (The Agent):** Connect the LLM. Give it the grid state ("Robot A is at [0,0]"). Ask it to plan the moves. Use your `detect_collision()` code to "veto" bad AI moves, forcing it to replan (similar to the Sokoban project).
5. **Weeks 12-14 (The Visuals):** A simple animation showing the robots moving safely.

Safety Net (Fallback): If the AI is too stupid to coordinate, just implementing a working **Cooperative A*** algorithm for two robots (without AI) is a passing grade for a CS thesis.

Expected Deliverable(s)

A simulation where multiple agents navigate a grid without colliding, validated by logic.
The Vibe: Like the “Sokoban Agent," but with multiple players moving at once.

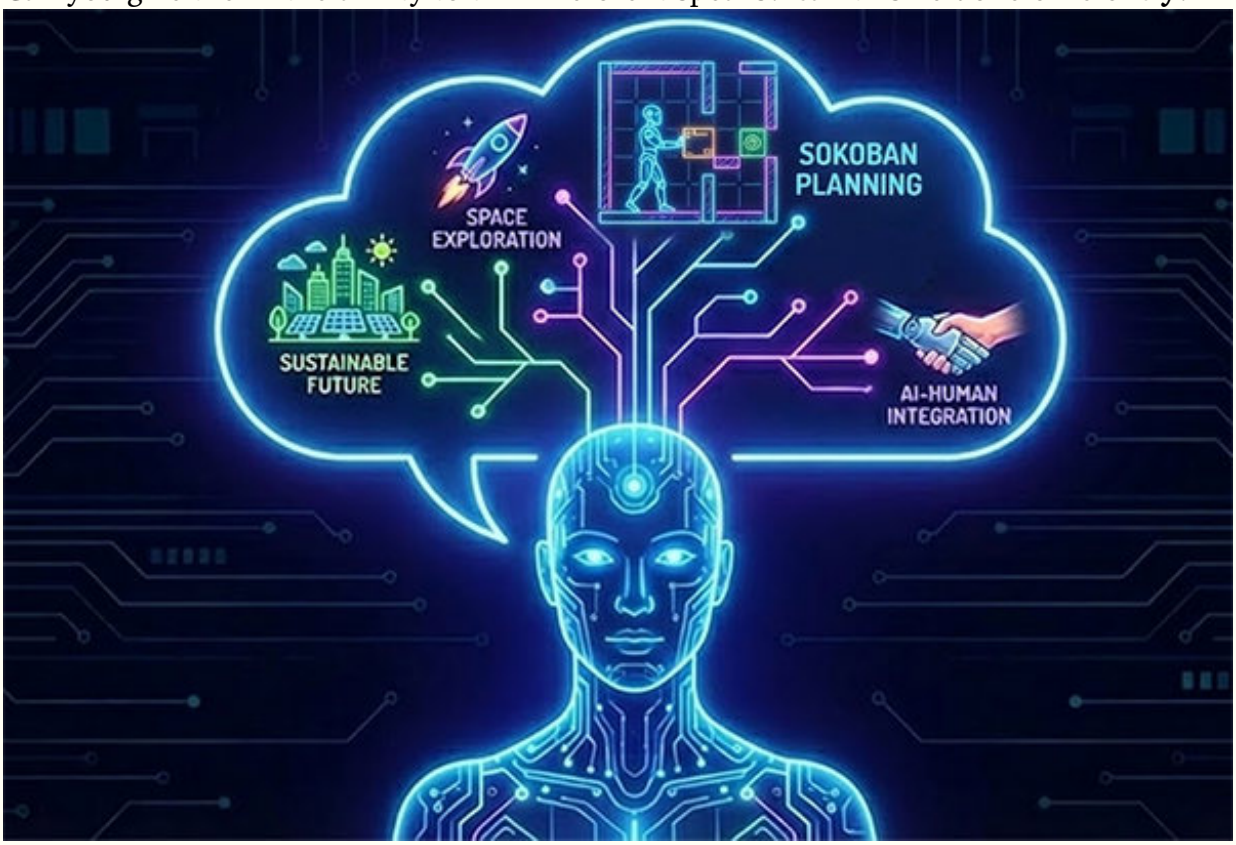
3.8 System-2 Planning: The Deadlock Detective

Industry Hook

Can an LLM think 50, 30, or even 5 steps ahead? — "Building an AI agent that 'imagines' the future before it moves."

Concept/Description

Current LLMs are impulsive—they predict the next word without thinking. You will implement "**Tree of Thoughts**" (ToT), allowing your agent to imagine multiple future scenarios, simulate the outcomes in a "Physics Engine" (like the **Sokoban** game), and choose the best path before taking a single step.
Can you give the AI the ability to think before it speaks? can this be done **efficiently**?



LLMs are notoriously bad at long-term planning. This project implements **Tree of Thoughts (ToT)** (Yao et al., 2023). The AI generates possible moves for a difficult planning puzzle (Sokoban), and a simple "Physics Engine" script simulates the outcome to verify validity (i.e., verifies if the move is valid) before execution, pruning the decision tree and effectively giving the AI a "working memory."

Strategic Info!

Academic Description: LLMs often lack "working memory" for long-term planning. This project implements the Tree of Thoughts (ToT) framework within a benchmark environment like **Sokoban**. The agent generates multiple future moves, which are then simulated by a lightweight Python "Physics Engine" to verify validity before execution. By pruning invalid branches in the decision tree, the agent "imagines" the future and verifies consequences, shifting from reactive token prediction to deliberative planning (System 2

thinking).
Paper: "Tree of Thoughts: Deliberate Problem Solving with Large Language Models" (Yao et al., Google DeepMind, 2023) == (Yao et al., 2023)

Prerequisites & Background

Required: Python Game Logic.
To Learn: Tree Search Algorithms (BFS/DFS).

The Plan:

- **Safe Plan:** A script where the LLM plays a text-based grid game. A verifier script rejects invalid moves.
- **Reach Goal:** "Look-Ahead." The agent simulates 3 steps into the future to avoid dead ends.

- **Task 1 (The World):** Implement a simple text-based state representation of the Sokoban game.
- **Task 2 (The Agent):** Build a ToT agent that generates 3 candidate next moves.
- **Task 3 (The Auditor):** Write a deterministic verifier that checks if a move is valid.
- **Reach Goal:** Implement "Look-Ahead" where the agent simulates 5 steps deep to avoid dead ends.

Expected Deliverable(s)

Implementing ToT where the AI simulates future states/moves in a physics engine before acting.

- Use a standard A* algorithm to solve Sokoban.
- Implement a standard algorithm and add an “Explainability" layer (algorithm is easy to find online) – add the layer using a basic prompt.
- Ask the LLM to commentate on the move. *“Explain why you made these moves*" Simplicity: The "Thinking" is faked by the explanation. The solving is done by a standard algorithm.

3.9 The Logic Advocate Tutor

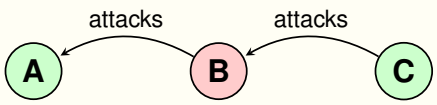
Industry Hook

"Podcasts are just chatter. Debates are math. Build an AI that wins arguments using Abstract Argumentation Frameworks."

Concept/Description

You will use Argumentation Frameworks to turn a controversial text into a "**Battle Graph**" of claims and counter-claims. Your system will calculate the "Grounded Extension" (the winning arguments) to show who actually won the logic battle. Turn rhetorical fluff into mathematical proofs.

This is basically simulating the "Deep Dive" format of NotebookLM, but adding rigor. This thesis uses **Dung's Argumentation Frameworks** (AF) to model a lesson as a formal debate between a Proponent and a Skeptic. The system extracts claims and counter-claims from a controversial text, builds an Argument Graph ($A \rightarrow B$), and calculates the Grounded Extension (the set of winning arguments). A final tutorial video visualizes this graph, showing which arguments "survived" the logical battle. The project is philosophically juicy by design, where algorithmic jurisprudence meets cognitive apprenticeship—so embrace that drama.



Script: A speaks. B attacks A. C attacks B (saving A).

Strategic Info!

Academic Description: Transforming passive content consumption into active logical engagement, this thesis uses Dung's Argumentation Frameworks (AF) to model educational content as a formal debate. The system extracts claims and counter-claims to build an Argument Graph, calculating the Grounded Extension to determine the set of "winning" arguments. This formalizes the "Deep Dive" format, ensuring that the AI wins arguments using abstract logic rather than rhetoric.

Paper: "Abstract Argumentation Scheme Frameworks" (Wait et al.) –and– Dung's original 1995 paper on Argumentation Frameworks (the formal definition proposed by Phan Minh Dung (1995)).

Prerequisites & Background

Required: Discrete Math.
To Learn: Abstract Argumentation Theory (Dung's Frameworks), PyArg.

The Plan:

- **Safe Plan:** Manually encode a debate into a Graph. Write a script that calculates the "Admissible Sets" (defended arguments).
- **Reach Goal:** An end-to-end pipeline where the AI reads a paper, identifies the "Attack Relations" between claims, and generates a dialogue script that follows the winning path.

Expected Deliverable(s)

Modeling a debate as a formal graph and calculating the "Grounded Extension".

- N.B. Abstract Argumentation is very theoretical.

ADDITIONAL NOTES: The student is tasked with building a system that performs Natural Language Processing (NLP) to extract logic, then applying Discrete Mathematics to solve it. The student is expected to:

Study Dung's Framework: Learn the math behind $AF = \langle Ar, att \rangle$ where Ar is a set of arguments and att is a binary relation on Ar.

Code the Logic: Use a library like `PyArg` to programmatically find winning sets of arguments.

Build the "Tutor": Create a UI or video generator that shows a user: *"Here is the messy text you read, here is the 'Battle Graph' of the actual logic, and here are the only claims that actually hold water."*

Here is the breakdown:

Node Extraction (The Claims): The AI reads a text and identifies individual statements or arguments (e.g., Argument A, B, and C).

Edge Mapping (The Attacks): The AI identifies which arguments contradict each other. In a directed graph, an arrow from B to A ($B \rightarrow A$) means "B attacks A." This creates the "Battle Graph."

The "Battle" (The Algorithm): The student applies a mathematical "semantics" (like the Grounded Extension) to the graph. This is an algorithm that determines which arguments are "cleared" or "justified."

- Example: If C attacks B, and B attacks A, then C "defends" A. In the "Battle Graph," both C and A are winners, while B is defeated.The Outcome: The system outputs a set of "winning" arguments that are logically consistent and defended against all counter-attacks.

Digression: The term "Battle Graph" is used here as a metaphor to make a very dry, theoretical topic (Abstract Argumentation) sound more engaging. It visualizes the conflict, where (1) The Battlefield is the graph nodes and arrows; (2) The Combatants are the claims (A, B, C); and (3) The Victory Condition is being part of the Grounded Extension (the set of arguments that cannot be logically overthrown).

To understand the "Grounded Extension", imagine a courtroom or a heated debate. In Dung's Argumentation Framework, we don't care if an argument is "true" in the real world; we only care if it can be defended within the graph. The Grounded Extension is the set of arguments that are "undefeated" because they are either not attacked at all, or they are successfully defended by other undefeated arguments.

To understand the battle rules, think of the status of an argument like a traffic light: green (in) indicates that the argument is accepted (it "won"); red (out) indicates that the argument is rejected (it was "killed" by an attacker); and gray (undecided) indicates that the argument is stuck in a stalemate.

How the Grounded Extension is Calculated (step-by-step) The Grounded Extension is "skeptical"—it is very hard to get into this set. You only get in if there is absolutely no doubt about your logical safety.

The "Unstoppable" Arguments: Find every argument that has no attackers. These are automatically "Green" because no one contradicted them.

The "Collateral Damage": Look at the arguments that were attacked by your "Green" arguments. Those are now "Red" (defeated).

The "Revenge" (Defense): Look at arguments that were being attacked by the "Red" ones. Since their attackers are now dead, these arguments are now "Green" (defended).

Repeat: Keep repeating this until no more arguments can be turned Green or Red.

The final list of "Green" arguments is the Grounded Extension.

In plain English: You start with nothing, add the arguments that have no attackers, then use those to "rescue" other arguments, repeating until the set can't grow any larger.

Chapter 4

Cluster 3: The Human-Centric Generators

Outline (Quick Access):

4.1 Sketch-based Heritage Restoration 55

4.2 The Kinematic Calligrapher: Vector-Based Arabic Script Generation 57

4.3 The Algorithmic Complexity Visualizer 59

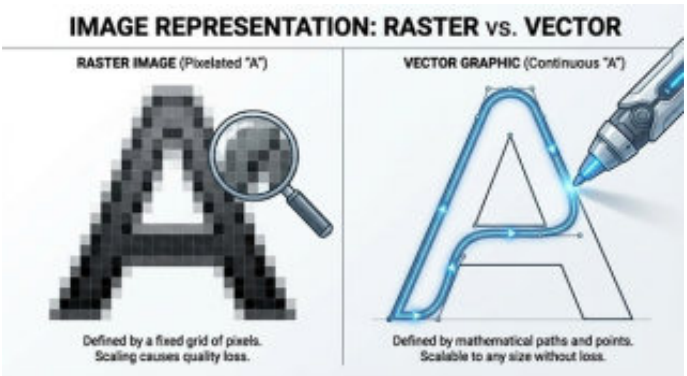
4.4 The Dysgraphia Detective 61

4.5 The Digital Handwriting Scorer: Assessment of Handwriting 63

Cluster 3: The Human-Centric Generators

Creative AI, Digital Heritage, and Vector Mathematics.

Cluster Visualization



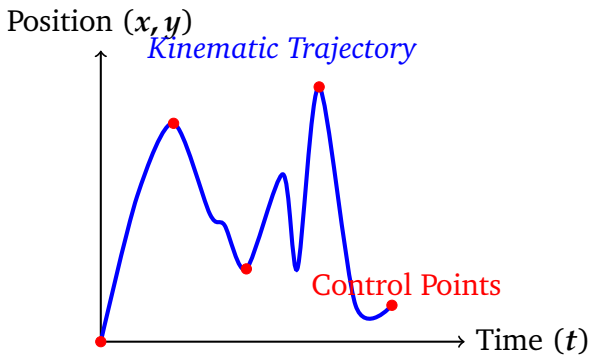


Figure 4.1: Visualizing the distinction between discrete and continuous character generation: a pixelated 'A' versus a smooth, vector line drawing of an 'A', with the latter illustrating the kinematic path of a robotic plotter.

Theme: Time-Series, Kinematics, Vector AI, and Creative Arts.

4.1 Sketch-based Heritage Restoration

Industry Hook

"Restoring the past, stroke by stroke. An AI that completes damaged archaeological sketches."

Concept/Description

Vector In-Painting. Using geometry to infer missing lines in damaged drawings, rather than guessing pixels.

Restore the past, stroke by stroke. Using geometry and vector math, you will build an AI that infers missing lines in damaged archaeological sketches. Instead of guessing pixels, your AI uses geometric reasoning to complete the artist's original intent.
Related work: (Abdelfattah and Atef, 2025b,a; Abdelfattah and Fawzy, 2023; Abdelfattah and Amr, 2023).

Strategic Info!

Academic Description: These projects utilize Vector In-Painting and Kinematic Analysis to bridge the past and present. One track applies geometric reasoning to infer and restore missing strokes in damaged archaeological sketches. Another analyzes the speed, pressure, and hesitation in handwriting trajectories to detect biomarkers for Dysgraphia. A third track applies Style Transfer to stroke dynamics, translating modern handwriting into the vector paths of Ancient Egyptian Hieroglyphs. All three rely on the physics of movement rather than the statistics of pixels.
Paper: "Analyzing handwriting legibility through hand kinematics" (2025) or "Handwritten Text Generation from Visual Archetypes" (CVPR 2023).

Prerequisites & Background

Required: Geometry basics.

The Plan:

- **Safe Plan:** Algorithmically remove parts of a line drawing. Use interpolation (Splines) to fill the gap.
- **Reach Goal:** Compare mathematical filling vs. Generative AI filling.

Expected Deliverable(s)

Using geometry to complete missing lines in archaeological drawings.

4.2 The Kinematic Calligrapher: Vector-Based Arabic Script Generation

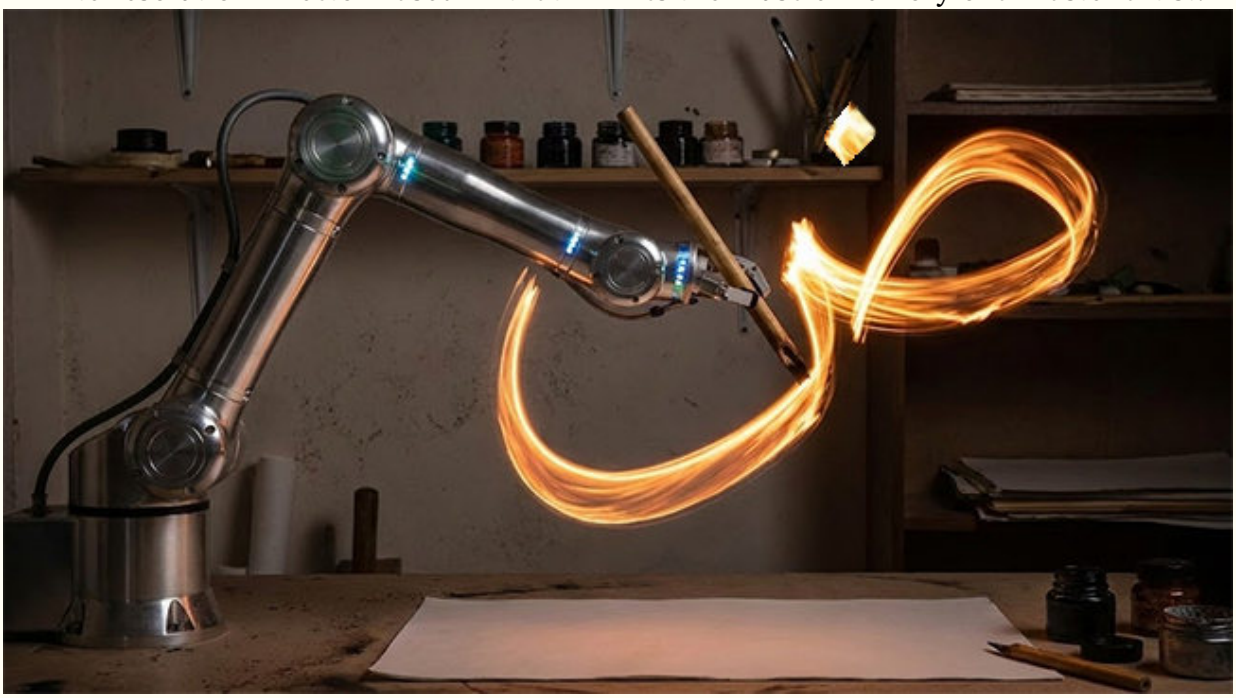
(Co-supervised by Dr. Yomna Hassan)

Industry Hook

"Midjourney creates pixels. We create movement. Building an AI that learns the 'muscle memory' of Arabic calligraphy."

Concept/Description

You will train a model to work in "Kinematic Space," predicting the velocity and pressure of a pen rather than the color of a pixel. This allows you to generate Arabic script that has infinite resolution—vector-based AI that mimics the muscle memory of a master artist.



Current Image Generators work in "Pixel Space." This project works in "Kinematic Space" by treating handwriting not as an image, but as a time-series of coordinates $(x, y, t, \overline{x})$. The model learns to predict the "next movement" of *the* pen, allowing for the generation of infinite-resolution, vector-based Arabic script (Graves, 2013).
Handwriting is treated as a **Time-Series** of coordinates $(x, y, t, \overline{x})$ rather than an image. The model predicts the next pen movement, allowing for the generation of "Vector Handwriting" that scales infinitely without pixelation (Graves, 2013).
N.B. $\overline{x}$ is the rest of the parameters, such as "pen pressure" for instance.

Strategic Info!

Academic Description: Moving beyond the "Pixel Space" of diffusion models, this project operates in Kinematic Space, **treating handwriting as a time-series of control points** (x, y, t). The **IAM On-Line Handwriting Database** (Liwicki and Bunke, 2005) would be extremely helpful in this regard, yet the database is in English.
The model learns the "muscle memory" of Arabic calligraphy, predicting the next movement of the pen rather than the next pixel. This allows for the generation of infinite-

resolution, vector-based script, fundamentally changing how AI approaches the art of writing.
Paper: "DiffInk: Glyph- and Style-Aware Latent Diffusion Transformer for Text to Online Handwriting Generation" (2025). This is a cutting-edge paper replacing older RNN approaches.
[N.B.] Parts of this project are grounded in/inspired by the concepts presented in *Graph Neural Ordinary Differential Equations* (Poli et al., 2019). See also [Link]the GitHub repository.

Prerequisites & Background

Required: ML, Python Data Science (Pandas/Numpy).
To Learn: RNNs/LSTMs or Transformers for Time Series.

The Plan:

- **Safe Plan:** Visualize handwriting data. Train a model to predict the next point in a (simple) stroke.
- **Reach Goal:** Generate a full Arabic word using the trained model: A "Generative Writer" that takes text input and draws it on a digital canvas in a specific user's style.

- **Task 1 (Data):** Visualize and process a dataset (time-series points).
- **Task 2 (Training):** Train a small LSTM or Transformer model to predict the next (x, y, t) coordinate given a sequence of previous points.
- **Reach Goal:** Generate a complete Arabic word by auto-regressively predicting the pen path.

Expected Deliverable(s)

An AI that generates Arabic script by predicting pen movement.

- N.B. Training RNNs/LSTMs on stroke data is very data-heavy and hard to debug.

4.3 The Algorithmic Complexity Visualizer

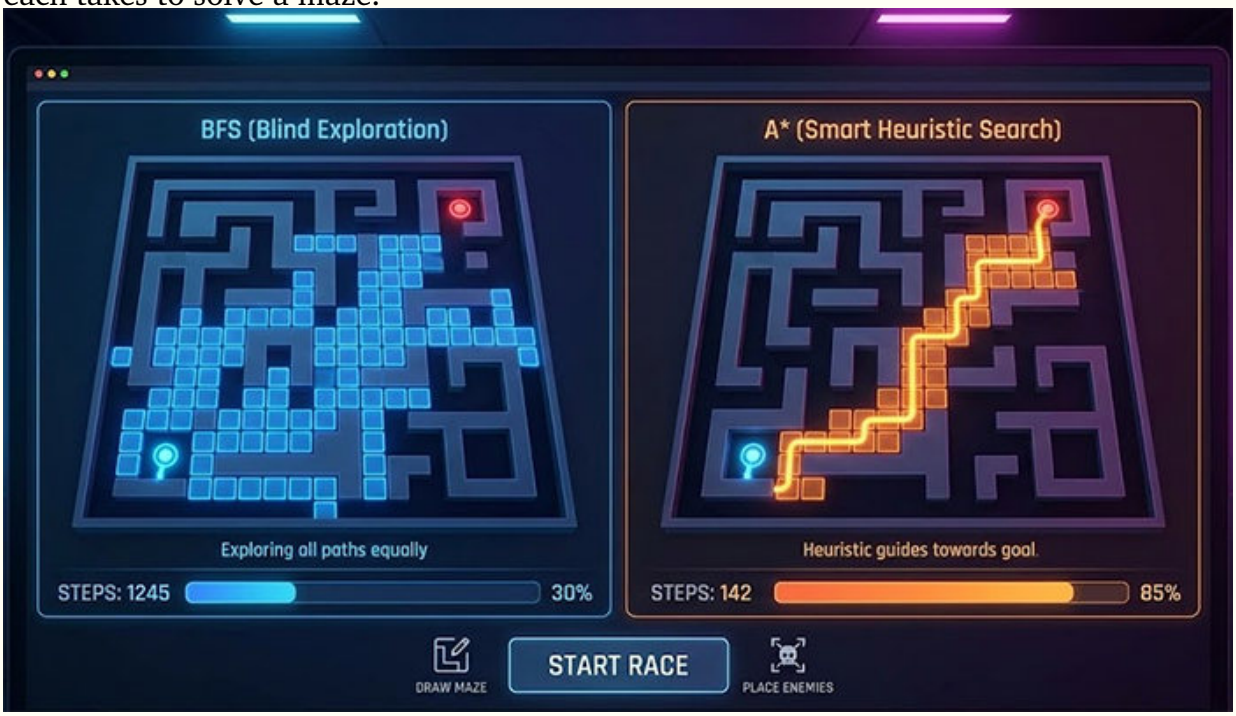
(Co-supervised by Dr. Yomna Hassan)

Industry Hook

Visualizing Complexity: A Comparative Study of BFS vs. A* Search.

Concept/Description

Abstract algorithms like Pathfinding are hard to understand without visualization. This project builds an educational simulation that races two algorithms: BFS (blind exploration) and A* (smart heuristic search). The student collects data on how many "steps" each takes to solve a maze.



Build a tool that lets students *see* how a Heuristic makes search faster.

Strategic Info!

THEORETICAL MAPPING (The Schema-Guided Framework)

High-Level Mapping: The "Artifact" is the maze. The "Schema" is the rule of movement (Up, Down, Left, Right). The "Perception" is the algorithm's view of the grid (what it has 'seen' vs. 'unseen').

- Artifact $\mathcal{A}$: The Maze Grid / Grid Map.
- Schema $\mathcal{S}$: Movement Rules $\{Up, Down, Left, Right\}$.
- Metric: Total Nodes Visited.

The Ideal Dream: A polished React/Unity web game where users can draw custom mazes, place enemies, and watch an AI agent navigate dynamically in real-time.
Plan B (The Safety Net):

1. **Console UI:** If GUI programming is too hard, use **ASCII art** in the terminal (# for

walls, . for path).

2. **Logic:** Implement the pseudocode for BFS and A* found in any CS 101 textbook.
3. **Metric:** Count the number of steps each algorithm took.

Expected Deliverable(s)

A polished React/Unity web game where users can draw custom mazes, place enemies, and watch an AI agent navigate dynamically in real-time.

4.4 The Dysgraphia Detective

(Co-supervised by Dr. Yomna Hassan)

Industry Hook

"A smart pen that diagnoses learning difficulties. Using physics and AI to help children write better."

Concept/Description

A smart pen can save a child's education. You will analyze the "hidden features" of handwriting—speed, pressure, and hesitation—to detect biomarkers for Dysgraphia (learning difficulties). This is AI for Social Good, applying physics and time-series analysis to solve real human problems.

Kinematic Analysis. Detecting hesitation or motor control issues by analyzing the speed and pressure of writing.

Strategic Info!

Academic Description: See section 4.2 and section 4.1.

Prerequisites & Background

Required: Data Analysis.

The Plan:

- **Safe Plan:** Calculate speed/acceleration profiles from raw data. Visualize them.
- **Reach Goal:** Train a classifier to distinguish between "Fluent" and "Dysgraphic" writing patterns.
  Paper: Handwritten dynamics assessment through convolutional neural networks: An application to Parkinson's disease identification.

Expected Deliverable(s)

Analyzing handwriting speed/pressure to detect learning difficulties.

- The AI suggests a line (using simple linear regression or curve fitting between two points), and the human accepts/rejects it.

4.5 The Digital Handwriting Scorer: Assessment of Handwriting

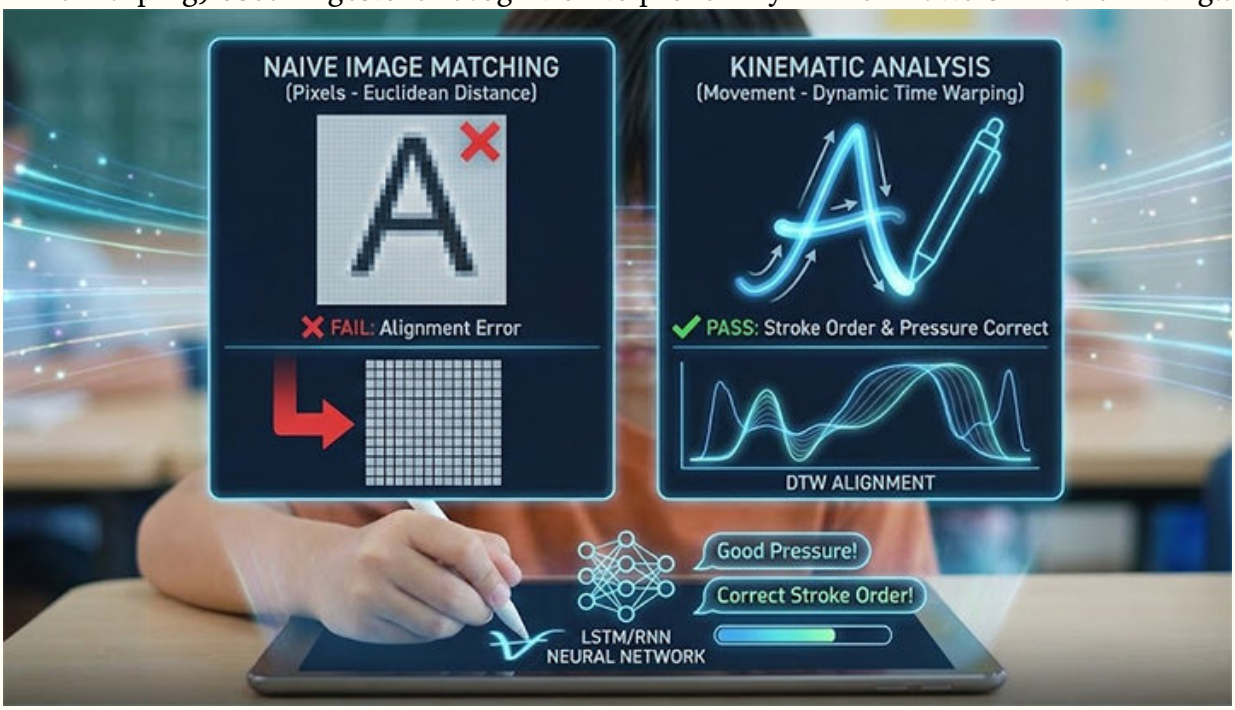
(Co-supervised by Dr. Yomna Hassan)

Industry Hook

Algorithmic Assessment of Handwriting: Euclidean Distance vs. Dynamic Time Warping (DTW).

Concept/Description

How does a computer know if you wrote a letter correctly? A simple image check fails if you write the letter slightly to the left.
This thesis explores **Kinematic Analysis**—looking at the *movement* of the pen over time rather than the final image. The student compares a naive image-matching algorithm (Euclidean Distance) against the industry standard time-series algorithm (DTW: Dynamic Time Warping) used in gesture recognition to prove why "Time" matters in handwriting..



Prove that comparing "movement" (Time-Series) is better than comparing "pixels" (Images) for grading handwriting.

Strategic Info!

The Ideal Dream: A real-time mobile app where a child writes on a tablet, and a Neural Network (LSTM/RNN) provides instant feedback on their stroke order and pressure.
Plan B (The Safety Net):

1. **No UI Required:** The student can process text files containing coordinates `[(0,0),(1,2)...]` instead of building a web app.
2. **Library Use:** Use Python's `fastdtw` library for the math.
3. **Experiment:** Compare DTW score vs. Euclidean Distance score on a "Bad" letter.

THEORETICAL MAPPING (The Schema-Guided Framework)

High-Level Mapping: The "Artifact" is the written letter. The "Perception" function Φ maps the continuous pen movement (x, y, t) into a discrete score. The "Schema" defines what constitutes a "Good" letter (smoothness, speed).

- Perception Φ: Mapping continuous pen strokes (x, y, t) to a letter ID.
- Schema S: Quality Schema $\{Speed, Smoothness, Shape\}$.

References:

- Tappert, C., et al. (1990). *The State of the Art in On-Line Handwriting Recognition*. Related paper: (originally related to the "The Dysgraphia Detective" project) Handwritten dynamics assessment through convolutional neural networks: An application to Parkinson's disease identification.

N.B. Parts of this project are grounded in/inspired by the concepts presented in *Graph Neural Ordinary Differential Equations* (Poli et al., 2019). See also [Link]the GitHub repository.

Expected Deliverable(s)

A real-time mobile app where a child writes on a tablet, and a Neural Network (LSTM/RNN) provides instant feedback on their stroke order and pressure.

Bibliography

Ahmed Abdelfattah and Adham Mohamed Amr. Revisiting the efficacy of image-based recognition of arabic handwriting: The case of whole scripts. In *2023 2nd International Conference on Smart Cities 4.0*, pages 548–553, 2023. doi: 10.1109/SmartCities4.056956.2023.10525737.

Ahmed Abdelfattah and David Atef. Museepal: An AI-powered cultural guide for museums (the case of the grand egyptian museum). In *Proceedings of the 24th International Conference on WWW/Internet (ICWI 2025)*, Porto, Portugal, November 2025a.

Ahmed Abdelfattah and David Atef. From recognition to reliability: A framework for trustworthy multimodal ai in cultural heritage (the museepal case). *IADIS Internationl Journal on WWW/Internet*, 23(2):81–92, 2025b. ISSN 1645-7641.

Ahmed Abdelfattah and Amr Fawzy. Revisiting the efficacy of image-based recognition of arabic handwriting: The case of isolated letters. In *2023 2nd International Conference on Smart Cities 4.0*, pages 542–547, 2023. doi: 10.1109/SmartCities4.056956.2023.10525934.

Ahmed Abdelfattah and Mikhail Gebrail. AI-powered pedagogy: Generating reliable and context-aware educational content from instructor-curated materials. In *Proceedings of the 22nd International Conference on Applied Computing (AC 2025)*, Porto, Portugal, November 2025.

Ahmed Abdelfattah and Abdelrahman Zakzouk. Smart tourism meets AI: A multimodal assistant for exploratory cultural engagement. In *Proceedings of the 22nd International Conference on Applied Computing (AC 2025)*, Porto, Portugal, November 2025.

Federico Becattini, Pietro Bongini, Luana Bulla, Alberto Del Bimbo, Ludovica Marinucci, Misael Mongiovì, and Valentina Presutti. VISCOUNTH: A large-scale multilingual visual question answering dataset for cultural heritage. *ACM Transactions on Multimedia Computing, Communications, and Applications*, 19(6):193:1–193:20, 2023. doi: 10.1145/3590773. URL https://dl.acm.org/doi/10.1145/3590773.

Kevin Buzzard, Mohammad Pedramfar, and The Xena Project. The natural number game 4, 2024. URL https://adam.math.hhu.de/#/g/hhu-adam/NNG4. Interactive theorem proving tutorial based on Lean 4.

Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction–CADE 28: 28th International Conference on Automated Deduction*, pages 625–635. Springer, 2021. doi: 10.1007/978-3-030-79876-5_37. URL https://link.springer.com/chapter/10.1007/978-3-030-79876-5_37.

Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, Dasha Metropolitansky, Robert Osazuwa Ness, and Jonathan Larson. From local to global: A graph RAG approach to query-focused summarization. *arXiv preprint arXiv:2404.16130*, 2024. URL https://arxiv.org/abs/2404.16130.

Farima Fatahi Bayat, Kun Qian, Benjamin Han, Yisi Sang, Anton Belyy, Samira Khorshidi, Fei Wu, Ihab Ilyas, and Yunyao Li. FLEEK: Factual error detection and correction with evidence retrieved from external knowledge. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 124–130. Association for Computational Linguistics, 2023. URL https://aclanthology.org/2023.emnlp-demo.10.

Google. Gemini 3 and gemini 3 pro, December 2025. URL https://gemini.google.com. Generative AI model used for image creation.

Alex Graves. Generating sequences with recurrent neural networks. *arXiv preprint arXiv:1308.0850*, 2013. URL https://arxiv.org/abs/1308.0850.

Thomas Hubert, Rishi Mehta, Laurent Sartran, Miklós Z Horváth, Goran Žužić, Eric Wieser, Aja Huang, et al. Olympiad-level formal mathematical reasoning with reinforcement learning. *Nature*, 635, 2025. doi: 10.1038/s41586-025-09833-y. URL https://www.nature.com/articles/s41586-025-09833-y.

Marcus Liwicki and Horst Bunke. IAM-OnDB - an on-line english sentence database acquired from whiteboard data. In *Proceedings of the 8th International Conference on Document Analysis and Recognition (ICDAR)*, pages 956–961. IEEE, 2005. doi: 10.1109/ICDAR.2005.132.

Stanislas Polu, Daniel Han, Caldwell Spinale, Philip Ringer, Daniel Selsam, OpenAI, and Harmonic Labs. Aristotle: IMO-level automated theorem proving. *arXiv preprint arXiv:2510.01346*, October 2025. URL https://arxiv.org/abs/2510.01346.

The Mathlib Community. The Lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs*, pages 184–197. ACM, 2020. doi: 10.1145/3372885.3373824. URL https://dl.acm.org/doi/10.1145/3372885.3373824.

Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. In *Advances in Neural Information Processing Systems*, volume 36, pages 11809–11822, 2023. URL https://proceedings.neurips.cc/paper/2023/hash/271db9922b8d1f4dd7aaef84ed5ac703-Abstract-Conference.html.

Ruochen Zhao, Xingxuan Li, Shafiq Joty, Chengwei Qin, and Lidong Bing. Verify-and-edit: A knowledge-enhanced chain-of-thought framework. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 5823–5840, Toronto, Canada, 2023. Association for Computational Linguistics. URL https://aclanthology.org/2023.acl-long.320.

N.B.: Please note that other bachelor projects not listed in this file are to be conducted in cooperation with Dr. Yomna Hassan; these will be published on her list.

Declaration of Generative AI and Software Use

All ***technical*** diagrams, icons, graphs, plots, charts, and mathematical plots within this work were programmatically created by the author using the LaTeX `tikzpicture` environment and the `TikZ/PGF` package to ensure exact mathematical precision and reproducibility. They represent the author's original code and analysis. Exceptions to this are the ***conceptual*** illustrations and illustrative images, which were synthetically generated using Google's Gemini 3 and Gemini 3 Pro models (Google, 2025) (July–December 2025) based on specific prompts designed by the author. These images are presented without substantive manipulation, having been subjected only to resizing or cropping. All remaining content represents the author's original work and is subject to standard copyright.

Chapter 5

The Landscape of Terms in AI-Powered Software Development Tools

The following narrative explains the landscape of AI software development tools as of early 2026, specifically addressing some terms found in an inspiring Reddit discussion that can be accessed [Link]here. The main argument is that **Antigravity** is powerful because it gives free access to the expensive **Sonnet 4.5** model and handles **Multi-agent Orchestration well**. However, they warn that using YOLO mode (giving the agent too much freedom) or relying too much on Vibe Coding can lead to messy code that looks good but is hard to maintain.

5.1 The Context: An Agentic Shift

We live in an era where you must first understand the recent shift that occurred in late 2025. The world moved from "AI as a Chatbot" (where you paste code into a box) to "AI as an Agent" (where the AI lives in your editor and performs actions).

5.1.1 The Integrated Development Environments (IDEs)

These are the actual applications where you write code.

VS Code (Visual Studio Code): The industry-standard, open-source code editor (built by Microsoft). Almost all modern AI editors (Cursor, Windsurf, Antigravity) are "forks" of VS Code. This means they took the core software of VS Code and modified it to bake AI deep into the interface.

Cursor: The first major "AI-native" code editor. It became famous for its "Composer" feature, which allowed users to write natural language prompts (e.g., "*Make the button blue and connect it to the database*") that the editor would apply across multiple files. It is the benchmark against which all ***new*** tools are measured.

Windsurf: A direct competitor to Cursor (released by Codeium). It introduced the concept of "Flows" or deep context awareness, claiming to understand the "trajectory" of your coding session better than Cursor.

Antigravity: Google's "Agent-First" IDE (released November 2025). Unlike VS Code (where you type) or Cursor (where you prompt), Antigravity is designed for orchestration. **You act more like a manager supervising AI employees**. It allows you to spawn multiple "agents" to work on different tasks simultaneously (e.g., one agent fixing bugs in the backend while another writes documentation).

5.1.2 The Intelligence (The Models)

The IDE is the car; the Model is the engine.

Claude 3.5 Sonnet: An AI model released by Anthropic in mid-2024. It was widely considered the "gold standard" for coding tasks for over a year due to its high reasoning capabilities and large context window.

Sonnet 4.5: The successor, Claude Sonnet 4.5, released in September 2025. It represents a significant leap in "agentic" capabilities—meaning it is much better at browsing the web, using a terminal, and managing complex, multi-step tasks without getting confused. Google's Antigravity offers this powerful model for free during its preview phase.

5.1.3 The No-Code & Specialized Tools

Opal: This refers to Google's experimental "vibe coding" tool (as of early 2026). While Antigravity is intended for professional software engineers, Opal is a "no-code" app builder. Users simply describe an application in natural language (the "vibe"), and Opal generates a fully functional web application. Primarily designed for non-technical users or rapid prototyping, its use in a GUC thesis is **strictly discouraged**, except for minor sub-tasks or urgent proof-of-concept components.

Note: Opal (Google Labs) is distinct from Gemini Apps. Consequently, usage data is excluded from Gemini Apps Activity and is not governed by Gemini's Workspace settings.

CLI (Command Line Interface): A standard term for text-based interfaces (like Terminal or PowerShell). However, in this context, it refers to Agentic CLIs (like Claude Code or custom tools). These allow you to type update the database schema directly into your terminal, and the AI executes the necessary shell commands to do it.

5.1.4 The Workflows & Slang

The following terms describe how people use the aforementioned tools.

Multi-agent Orchestration: This is the core selling point of Antigravity. Instead of a single chat window, you have a "Mission Control" view. You can assign Agent A to "`Refactor the authentication module`" and Agent B to "`Write unit tests for the new API`," and they run in parallel.

YOLO Mode: Slang derived from "You Only Look Once", and is found in many contexts, like computer vision. In AI coding contexts, this refers to an Autonomous Execution Mode. Normally, an AI agent will ask for permission before running a terminal command (like deleting a file or installing a package). In "YOLO Mode," you disable these safeguards, giving the AI full permission to execute commands and edit files without pausing for your approval. It is faster but risky—if the AI hallucinates, it could delete your codebase.[1]

Vibe Coding: A somewhat derogatory or playful term for coding purely by natural language prompts (*"The vibe is off, make it pop more*") without understanding the underlying code.

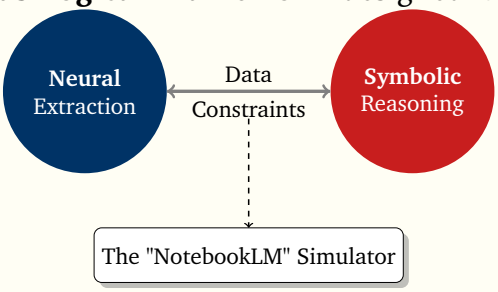
[1] [Link]This video demonstrates the concept of "YOLO Mode" (automated planning and coding) in a similar agentic tool, illustrating the shift from manual coding to supervising autonomous agents.

Chapter 6

The Neuro-Symbolic NotebookLM: *A Guide for Building Trustworthy, Rigorous, and Multimodal Educational AI Systems*

Concept/Description

Preface to an Exemplar Thesis Framework: This special chapter serves as a comprehensive **technical blueprint** for an exemplar Bachelor's thesis, centered on developing a *Neuro-Symbolic* (NeSy) evolution of educational AI tools similar to NotebookLM and our earlier publication in Abdelfattah and Gebrail (2025). It directly addresses the "crisis of truth" in generative AI by superseding purely probabilistic word prediction with a **rigorous logical framework** designed to mitigate hallucinations.



The chapter details a (work-in-progress) reference architecture comprised of 3 primary subsystems:

Course Compiler: for optimal, dependency-aware lesson sequencing;
Socratic Loop: for structuring argumentation and dialectical engagement; and
Epistemic Auditor: for the formal verification of mathematical facts.

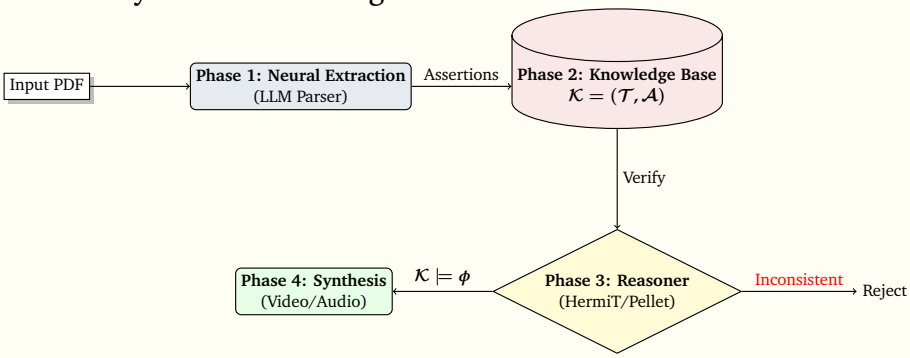
By bridging neural information extraction with symbolic reasoning, the proposed (work-in-progress) system ensures that educational content is not merely plausible, but *provably consistent* with its source material, based on the framework presented in Abdelfattah and Atef (2025b).

Strategic Info!

Note to the Student: Included herein are detailed architectural descriptions, formal specifications, and incomplete Python implementations accompanied by evaluation metrics. This content is intended to transition AI development from unreliable "vibe coding" toward trustworthy, verifiable synthesis. You are not expected to reproduce this entire architecture. Rather, this chapter is a **structural guide**. You are invited to:

- Analyze the rigor required for high-impact scientific contribution;
- Select specific modules (e.g., the Auditor or the Compiler) to implement or enhance (depending on your assigned thesis topic);
- Complete the provided code fragments as part of your own contribution;
- Use this framework as context to motivate your specific topic in AI education.

Unleash your creativity within this rigorous context. Your goal is to help build the foundations of trustworthy AI for the next generation.



The Architecture of the Epistemic Auditor. (N.B. The updated version is shown in Figure 6.1.)

Outline (Quick Access):

6.1 The Manifesto: Why We Are Here . . . 72
6.1.1 The Problem with "Vibe Coding" . . . 72
6.1.2 The (Broad) NeSy Approach . . . 72
6.2 The Baseline: Building a "Naive" NotebookLM . . . 73
6.2.1 The Pipeline Architecture . . . 73
6.2.2 The "Starter Kit" Code . . . 73
6.3 The Neuro-Symbolic Course Compiler . . . 74
6.3.1 The Concept . . . 75
6.3.2 Theoretical Foundation . . . 75
Directed Acyclic Graphs (DAGs) . . . 75
Topological Sort . . . 75
6.3.3 Thesis Implementation Plan . . . 75
6.3.4 Code Snippet: The Symbolic Validator . . . 75
6.4 The Socratic Loop (Argumentation Frameworks) . . . 76
6.4.1 The Concept . . . 76
6.4.2 Theoretical Foundation . . . 76
Dung's Argumentation Framework (AF) . . . 76
Semantics & Extensions . . . 76
6.4.3 Thesis Implementation Plan . . . 76
6.4.4 Code Snippet: The Debate Logic . . . 77
6.5 Tools of the Trade . . . 77
6.5.1 Programming Libraries . . . 77
6.5.2 References & Further Reading . . . 77
6.5.3 Final Deliverable Requirements . . . 78
6.6 The Epistemic Auditor (Ontology-Mediated Verification) . . . 78
6.6.1 The Concept . . . 78
6.6.2 Theoretical Foundation: Description Logics (DL) . . . 78
The Knowledge Base ($\mathcal{K} = \mathcal{T} \cup \mathcal{A}$) . . . 78
Formalization of the "Lesson" . . . 79
The Verification Process (Entailment) . . . 79
6.6.3 Work Implementation Plan . . . 79
6.6.4 Code Snippet: The Neuro-Symbolic Guardrail . . . 79
6.6.5 Visualizing the Result: The "Truth Bar" . . . 80
6.6.6 Evaluation Strategy . . . 81
The Knowledge Base ($\mathcal{K}$) . . . 83

6.1 The Manifesto: Why We Are Here

6.1.1 The Problem with "Vibe Coding"

We live in the era of Large Language Models (LLMs), which are incredible at processing text, but have a fatal flaw: they are **probabilistic**, not **logical**. If you ask an LLM to generate a lesson plan, it does so by predicting the most likely next word, not by understanding the pedagogical dependencies between concepts.

Google's **NotebookLM** is a fantastic tool that turns PDFs into "Audio Overviews" (podcasts). However, it is a "Black Box." We do not know *why* it chose to discuss Topic A before Topic B. We cannot verify if its explanation is mathematically sound. We cannot control it.

6.1.2 The (Broad) NeSy Approach

This track of projects aims to **reverse-engineer and improve** NotebookLM-like tools. We will not just "call an API." We will build a system that:

1. **(Neural) Reads:** Uses LLMs to extract raw text and concepts from contents, like PDFs.

2. **(Symbolic) Reasons:** Uses Graph Theory and Formal Logic to structure that content.
3. **(Multimodal) Synthesizes:** Generates Videos and Slides based on the *verified* structure.

This document should be your "Playbook/Guide." It contains the code to get started, the theory to make it rigorous, and the specific proposals you will tackle. This could be useful for a couple of the theses proposed to GUC students in SS26.

6.2 The Baseline: Building a "Naive" NotebookLM

Before we apply advanced theory, the reader must understand how to (practically) simulate the basic functionality of NotebookLM (PDF → Video) using `Python`. This should be your starting point.

6.2.1 The Pipeline Architecture

The basic simulation consists of three steps:

- **Step 1 (Ingestion):** Extract text from a PDF using `Python / pypdf`.
- **Step 2 (Scripting):** Use an LLM (OpenAI/Gemini) to convert text into a JSON Storyboard.
- **Step 3 (Production):** Use `Pillow` (images) and `MoviePy` (video) to render the output.

6.2.2 The "Starter Kit" Code

The following `Python` code implements a simulator. You are expected to run this, understand it, and then *criticize* it in your thesis (e.g., "It lacks structure," "It hallucinates").

notebooklm_simulator.py

```python
import json
import os

# Requires: pip install openai pypdf pillow gTTS moviepy
from openai import OpenAI
from pypdf import PdfReader
from PIL import Image, ImageDraw, ImageFont
from gtts import gTTS
from moviepy.editor import ImageClip, AudioFileClip, concatenate_videoclips

# --- STEP 1: READ THE PDF ---
def extract_text(pdf_path):
reader = PdfReader(pdf_path)
text = ""
for page in reader.pages:

# Check if text exists to avoid errors on empty pages
page_text = page.extract_text()
if page_text:
text += page_text
return text[:15000] # Truncate for demo purposes (Token limits)

# --- STEP 2: THE "BRAIN" (LLM) ---
def generate_storyboard(text, api_key):
client = OpenAI(api_key=api_key)
# Forced JSON structure for downstream parsing
prompt = f"""
You are a Professor. Create a Video Lesson Plan from this text.
Return a JSON object with this EXACT structure:
{{
    "title": "Lesson Title",
    "scenes": [
    {{
        "slide_title": "Slide Header",
        "bullets": ["Point 1", "Point 2"],
```

```
        "script": "The exact words the voiceover should speak."
    }}
    ]
}}
Source Text: {text}
"""

response = client.chat.completions.create(
model="gpt-4o",
messages=[{"role": "user", "content": prompt}],
response_format={"type": "json_object"}
)
return json.loads(response.choices[0].message.content)

# --- STEP 3: THE "STUDIO" (RENDER VIDEO) ---
def create_video(storyboard, output_file="lesson.mp4"):
clips = []

# Load default font (ensure you have one, or use absolute path to .ttf)
font = ImageFont.load_default()

for i, scene in enumerate(storyboard['scenes']):

# A. Make Slide Image
img = Image.new('RGB', (1280, 720), color=(73, 109, 137))
d = ImageDraw.Draw(img)
# Draw Title
d.text((50, 50), scene['slide_title'], fill=(255,255,0)) # removed font arg for compatibility
# Draw Bullets
y = 150
for b in scene['bullets']:
d.text((50, y), f"- {b}", fill="white")
y += 40

img_path = f"temp_{i}.png"
img.save(img_path)
# B. Make Audio
tts = gTTS(scene['script'], lang='en')
audio_path = f"temp_{i}.mp3"
tts.save(audio_path)
# C. Combine Image + Audio
audio = AudioFileClip(audio_path)
video = ImageClip(img_path).set_duration(audio.duration).set_audio(audio)
clips.append(video)
# Concatenate and Export
final = concatenate_videoclips(clips, method="compose")
final.write_videofile(output_file, fps=24)
print(f"Video Generated Successfully: {output_file}")

# Usage Example:
# txt = extract_text("my_lecture.pdf")
# story = generate_storyboard(txt, "YOUR_KEY_HERE")
# create_video(story)
```

! Crucial Concept

The Flaw in the Baseline: The code above relies entirely on the LLM to decide the *order* of the slides. If the PDF is complex, the LLM might hallucinate the order or miss prerequisites. **Your actual work MUST fix this using the methods below.**

6.3 The Neuro-Symbolic Course Compiler

Theme: Topological Sorting, Dependency Graphs, and Curriculum Learning.
Related Topic(s): The Prerequisite Pathfinder (check §3.2) — The Curriculum Architect: Engineering the Perfect Learning Path (check §2.2).

6.3.1 The Concept

An educational video is not just a sequence of slides; it is a *linearization* of a complex dependency graph. You cannot teach "Derivatives" before "Limits." This thesis proposes treating a PDF as a **Dependency Graph** (G). The goal is to extract concepts, map their prerequisites, and perform a *Topological Sort* to find the mathematically optimal teaching order.

6.3.2 Theoretical Foundation

Directed Acyclic Graphs (DAGs)

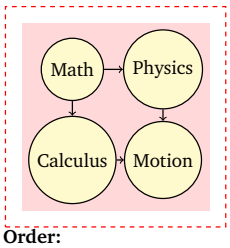
We model the knowledge in the PDF as a graph $G = (V, E)$, where:

- V (Vertices): The set of concepts (e.g., {Atom, Electron, Charge}).
- E (Edges): The dependency relationship. An edge (u, v) means "u is a prerequisite for v".

To create a valid lesson plan, G must be a **DAG** (it cannot have cycles). If "A explains B" and "B explains A", the definition is circular and invalid.

Topological Sort

This is the algorithm used to sequence the lesson. A topological sort of a DAG is a linear ordering of vertices such that for every directed edge (u, v), vertex u comes before v in the ordering. This ensures the generated video *never* uses a term before defining it.



Lesson Order:
Math $\rightarrow$ Calculus $\rightarrow$ Physics $\rightarrow$ Motion

6.3.3 Thesis Implementation Plan

- **Module 1: Concept Extraction (Neural).** Use an LLM to extract tuples from the PDF: `(Concept, Definition, [Prerequisites])`.
- **Module 2: Graph Construction (Symbolic).** Use the Python library `NetworkX` to build the graph.
- **Module 3: The Validated Sequencer.**
  1. Detect Cycles: `networkx.simple_cycles(G)`. If a cycle exists, the AI must flag a "Pedagogical Error."
  2. Sort: `networkx.topological_sort(G)`. This list becomes the strict order of slides.
- **Module 4: Video Synthesis.** Feed the sorted list into the simulator from Section 6.2.

6.3.4 Code Snippet: The Symbolic Validator

Code Snippet: The Symbolic Validator

```python
import networkx as nx

def optimize_lesson_plan(concepts_list):
G = nx.DiGraph()

# 1. Build Graph
for item in concepts_list:
```

```
G.add_node(item['concept'])
for prereq in item['prerequisites']:
# Edge: Prerequisite -> Concept
G.add_edge(prereq, item['concept'])

# 2. Check for Circular Logic (The "Audit")
if not nx.is_directed_acyclic_graph(G):
cycles = list(nx.simple_cycles(G))
raise ValueError(f"Pedagogical Error: Circular definitions found in {cycles}")

# 3. Generate Optimal Order
lesson_order = list(nx.topological_sort(G))
return lesson_order
```

6.4 The Socratic Loop (Argumentation Frameworks)

Theme: Computational Argumentation, Debate Logic, and Multi-Agent Systems.
Related Topic(s): The Logic Advocate Tutor (check §3.9).

6.4.1 The Concept

NotebookLM's "Audio Overview" features two AI hosts chatting. Currently, this is unstructured. This work proposes generating a **Dialectical Video**. The system treats the PDF as a set of claims and counter-claims. It uses Abstract Argumentation Frameworks to mathematically model a debate. The resulting video is not just a chat; it is a visualization of a logical game where "Host A" (Proponent) and "Host B" (Skeptic) battle to find the "Grounded Extension" (the winning truth).

6.4.2 Theoretical Foundation

Dung's Argumentation Framework (AF)

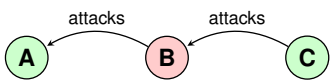
We use the formal definition proposed by Phan Minh Dung (1995). An AF is a pair $AF = \langle \mathcal{A}, \mathcal{R} \rangle$ where:

- $\mathcal{A}$: A set of arguments (extracted from the PDF).
- $\mathcal{R}$: A binary attack relation. $(a, b) \in \mathcal{R}$ means "Argument a attacks/refutes Argument b".

Semantics & Extensions

To decide which host "wins" the debate, the system computes the **Grounded Extension**.

- An argument is **Defended** if all its attackers are themselves attacked by accepted arguments.
- The generated script follows the logic: *Claim* $\rightarrow$ *Attack* $\rightarrow$ *Defense*.



Script: A speaks. B attacks A. C attacks B (saving A).

6.4.3 Thesis Implementation Plan

- **Module 1: Argument Mining.** Use LLMs to convert the PDF text into "Claim-Attack" pairs.
- **Module 2: The Logic Engine.** Implement a simple solver (or use `PyArg`) to determine the status of each argument (Accepted/Rejected/Undecided).
- **Module 3: Script Generation.**

  - If Argument A is "Accepted," Host 1 presents it confidently.
  - If Argument B attacks A but is "Rejected" later, Host 2 presents it as a "Common Misconception" before Host 1 corrects it.

- **Module 4: Visualization.** The video must display the Argument Graph growing and changing colors (Green/Red) as the hosts speak.

6.4.4 Code Snippet: The Debate Logic

Code Snippet: The Debate Logic

```
class ArgumentationFrame:
def __init__(self, arguments, attacks):
self.args = arguments
self.attacks = attacks # List of tuples [('B', 'A'), ('C', 'B')]

def get_attackers(self, argument):
# Find all arguments that attack 'argument'
return [att for (att, vic) in self.attacks if vic == argument]

def is_defended(self, argument, accepted_set):
attackers = self.get_attackers(argument)
# An argument is defended if ALL its attackers are attacked
# by an argument already in the accepted set
for att in attackers:
defenders = self.get_attackers(att)
# If no accepted argument attacks the attacker, we are not safe
if not any(d in accepted_set for d in defenders):
return False
return True

def compute_grounded_extension(self):
# Iteratively find arguments that are undefended
accepted = set()
changed = True
while changed:
changed = False
for a in self.args:
if a not in accepted and self.is_defended(a, accepted):
accepted.add(a)
changed = True
return list(accepted)
```

6.5 Tools of the Trade

To succeed in the work proposed in 6.3 and 6.4, you will need to master the following stack.

6.5.1 Programming Libraries

PyPDF / LangChain: For reading PDFs and chunking text.
NetworkX: The standard Python library for Graph Theory (Proposal A).
NetworkX / PyArg: For Argumentation Frameworks (Proposal B).
OpenAI API / Ollama: For the "Neural" component (Text Generation).
MoviePy / Manim: For generating the output video programmatically. *Manim (by 3Blue1Brown) is highly recommended for Proposal A/B visualization.*

6.5.2 References & Further Reading

1. **Original Argumentation Paper:** Dung, P. M. (1995). "On the acceptability of arguments and its fundamental role in nonmonotonic reasoning, logic programming and n-person games."

2. **Graph RAG:** Edge, D., et al. (2024). "From Local to Global: A Graph RAG Approach to Query-Focused Summarization."
3. **Neuro-Symbolic Survey:** Hitzler, P., et al. (2023). "Neuro-symbolic Artificial Intelligence: The State of the Art."
4. **Course Reference:** Look up "Topological Sort" in any standard Algorithms textbook (e.g., Cormen).

6.5.3 Final Deliverable Requirements

For both proposals (6.3 and 6.4), the work must deliver:

1. A **GitHub Repository** containing the Python pipeline.
2. A **Demo Video** generated entirely by code from a novel PDF.
3. A **Written Report** explaining the mapping between the Theoretical Model (DAG/AF) and the Python Implementation.
4. A **Evaluation** comparing the AI's output logic against a human professor's lesson plan.

6.6 The Epistemic Auditor (Ontology-Mediated Verification)

Theme: Description Logics ($\mathcal{ALC}$), Knowledge Graphs (OWL), and Neuro-Symbolic Entailment.
Related Topic(s): A+ GUCians Only!

6.6.1 The Concept

While the proposed work in §6.3 ensures the *order* is correct, and that in 6.4 ensures the *argument* is sound, this work ensures the *facts* are ontologically consistent.

Standard RAG (Retrieval-Augmented Generation) is "fuzzy." It retrieves text that *looks* similar. This thesis proposes **ROG (Retrieval-Ontology-Generation)**. It views the PDF not just as text, but as a source of assertions that populate a formal **Knowledge Base (KB)**. The system uses a **Reasoning Engine** (like HermiT or Pellet) to mathematically prove that the AI's generated script is logically entailed by the source material.

This is the "Holy Grail" of Trustworthy AI (TAI): A system that cannot hallucinate because it is mathematically constrained by a Logic Schema.

The MuseePal Inspiration

This proposal is directly derived from the *Trustworthiness Constraint* introduced in the framework Abdelfattah and Atef (2025b), which defines trustworthiness as a logical condition: $\text{Trustworthy}(Y) \iff \forall p \in Y, \exists d \in \mathcal{D}_{\text{rel}}$ such that $d \models p$. In this work/proposal, you will build the "Epistemic Auditor" that strictly enforces this mathematical law.

6.6.2 Theoretical Foundation: Description Logics (DL)

To build this, you must master **Description Logic**, specifically the $\mathcal{ALC}$ family, which underpins the Web Ontology Language (OWL).

The Knowledge Base ($\mathcal{K} = \mathcal{T} \cup \mathcal{A}$)

A Knowledge Base consists of two parts:

The TBox (Terminological Box): The "Schema" or "Ontology." It defines the rules of the world (invariant truths).
The ABox (Assertional Box): The "Data." It defines specific facts extracted from the PDF.

Formalization of the "Lesson"

Let us assume we are generating a video about "Ancient Egypt."

1. **The TBox ($\mathcal{T}$) (The Rules):** These are strict logic axioms defined by the developer (you) or extracted from the PDF's structure:

$$\begin{aligned} \text{Pharaoh} &\sqsubseteq \text{Person} \sqcap \exists \text{ruled.Kingdom} && \text{(Every Pharaoh is a Person who ruled a Kingdom)} \\ \text{Tomb} &\sqsubseteq \text{Building} && \text{(Every Tomb is a Building)} \\ \text{BuriedIn} &\equiv \text{hasGrave}^{-} && \text{(Inverse Property)} \end{aligned}$$

2. **The ABox ($\mathcal{A}$) (The Extracted Facts):** The LLM reads the PDF and extracts these specific assertions:

$$\begin{aligned} \text{Pharaoh}(\text{RamessesII}) \\ \text{BuriedIn}(\text{RamessesII}, \text{KV7}) \\ \text{Tomb}(\text{KV7}) \end{aligned}$$

The Verification Process (Entailment)

If the AI Script Generator attempts to say: "*Ramesses II was buried in the sky.*", the Logic Engine checks consistency. Since the TBox states specific physical properties of burial, and the ABox links Ramesses to KV7 (a Tomb), a contradiction might arise if "Sky" is disjoint from "Tomb."

More importantly, we check for **Entailment** ($\models$). If the AI wants to claim $C(\text{RamessesII})$, the Reasoner checks:

$$\mathcal{K} \models C(\text{RamessesII})?$$

If the Reasoner returns `False`, the AI is **forbidden** from speaking that sentence.

6.6.3 Work Implementation Plan

- **Module 1: The Schema Architect (TBox).** Create a formal OWL ontology (using Protégé or Python) representing the domain of the PDF (e.g., "History," "Physics").
- **Module 2: The Neural Populator (ABox).**
  - Input: PDF Text.
  - Process: Use GPT-4 to extract logical triples.
  - Output: Valid RDF/OWL assertions (e.g., `<Ramesses> a <Pharaoh>`).
- **Module 3: The Reasoner (The Auditor).**
  - Use `Owlready2` to load the TBox + ABox.
  - Run the Reasoner to check for **Inconsistency** (e.g., A PDF says X is distinct from Y, but the LLM tried to equate them).
- **Module 4: The Verified Generator.**
  - The AI generates a sentence.
  - The sentence is parsed into a logical claim.
  - If the claim is entailed by the graph, it is synthesized into speech.
  - If not, it is discarded (Pruning Hallucinations).

6.6.4 Code Snippet: The Neuro-Symbolic Guardrail

This code demonstrates how to use Python to perform "Ontology-Mediated Verification."

epistemic_auditor.py

```
from owlready2 import *
import types

def run_auditor(extracted_triples, generated_claim):
# 1. Create a Temporary Ontology
onto = get_ontology("http://test.org/lesson.owl")

with onto:
# --- DEFINE TBOX (The Rules) ---
class Person(Thing): pass
class King(Person): pass
class ruled(ObjectProperty):
domain = [King]
range = [Thing]

# Disjointness ensures logical rigor
class Peasant(Person): pass
King.disjoint_with(Peasant)

# 2. POPULATE ABOX (From PDF Extraction)
# Assume extracted_triples = [("Ramesses", "type", "King")]
with onto:
ramesses = King("Ramesses")

# 3. VERIFY GENERATED CLAIM (The Audit)
# Scenario: AI tries to claim "Ramesses was a Peasant"
# Logical Claim: Ramesses is instance of Peasant

try:
with onto:
# We inject the claim temporarily to check consistency
ramesses.is_a.append(Peasant)

# Run the HermiT Reasoner
sync_reasoner()
print("Audit Passed: Claim is consistent.")
return True

except OwlReadyInconsistentError:
print("AUDIT FAILED: Logical Contradiction Detected!")
print("Explanation: Ramesses cannot be both a King and a Peasant.")
return False
```

! Crucial Concept

The MuseePal Extension: The paper (Abdelfattah and Atef, 2025b) suggests that future work includes "GraphRAG" and "Probabilistic Modeling." This line of work implements the **Logic-Based** component of that vision. Your system is the "Verification Agent" mentioned in Section 5.1 of the paper Abdelfattah and Atef (2025b), instantiated using Deductive Logic rather than Bayesian Probability.

6.6.5 Visualizing the Result: The "Truth Bar"

A unique feature of this line of work is the User Interface (UI). The generated video should have a "**Truth Confidence Indicator**."

- **Green Border:** The statement is logically entailed by the PDF ($\mathcal{K} \models \phi$).
- **Yellow Border:** The statement is consistent but not explicitly proven (Hypothesis).
- **Red Border (Blocked):** The statement contradicts the Ontology (Hallucination).

6.6.6 Evaluation Strategy

How do you prove this works?

1. **The Sabotage Test:** Intentionally feed the system a PDF with contradictory information (e.g., "The Earth is flat" on page 1, "The Earth is round" on page 2). A standard LLM will hallucinate a mix. Your system must throw an `InconsistencyError`.
2. **The Recall Test:** Compare the "Verified Facts" against a ground-truth Knowledge Graph of the domain.

The Epistemic Auditor (revisited)

A Neuro-Symbolic Framework for Verified Educational Content Generation via Description Logics and Ontology-Mediated Entailment

Introduction: The Crisis of Truth in Generative AI

We are currently witnessing a paradigm shift in education driven by Generative AI. Tools that can read a PDF textbook and generate summaries, quizzes, or even video lectures are becoming ubiquitous. However, these systems rely on Large Language Models (LLMs) which operate on a **probabilistic** rather than a **logical** basis. An LLM predicts the next word based on statistical likelihood, not semantic truth.

In high-stakes domains—such as education, history, or medicine—"hallucination" (the generation of plausible but false information) is unacceptable. A student watching an AI-generated video about Physics cannot afford to learn incorrect laws because the AI "confused" two concepts.

This thesis proposal introduces a rigorous solution: **The Epistemic Auditor**. It is a framework that does not merely "summarize" text but **verifies** it against a formal mathematical structure. It combines the linguistic power of Neural Networks with the reasoning rigor of Symbolic Logic (specifically Description Logics).

This document serves as a comprehensive, strategic guide for the student. It assumes no prior knowledge of logic or formal methods and builds the entire theoretical and practical framework from scratch.

Part I: Theoretical Foundations (REVISED)

To build a system that can "verify" truth, we must rigorously define what "truth" means. This thesis postulates that Trustworthiness is not a single property, but the intersection of two mathematical frameworks:

1. **The Functional Framework (Set Theory):** Ensures the system *perceives* and *retrieves* data deterministically from a source ($\Phi, \mathcal{R}$).
2. **The Logical Framework (Description Logics):** Ensures the *reasoning* upon that data is consistent and valid ($\models$).

Framework 1: Neuro-Symbolic Logic ($\mathcal{ALC}$)

We employ Description Logics (DLs) to model the "Rules of the World." We specifically use $\mathcal{ALC}$ (Attributive Language with Complements), the foundation of OWL.

Syntax & Semantics

The building blocks are:

- **Atomic Concepts (N_C):** Unary predicates (e.g., $\mathsf{Theorem}, \mathsf{Planet}$).
- **Atomic Roles (N_R):** Binary relations (e.g., `dependsOn`).
- **Individuals (N_I):** Constants (e.g., *Newton*).

Logic is defined by an **Interpretation** $\mathcal{I} = (\Delta^{\mathcal{I}}, \cdot^{\mathcal{I}})$, where $\Delta^{\mathcal{I}}$ is the domain and $\cdot^{\mathcal{I}}$ maps concepts to subsets.

The Knowledge Base ($\mathcal{K}$)

We define the Knowledge Base as a pair $\mathcal{K}_{Logic} = (\mathcal{T}, \mathcal{A})$:

- **TBox (Schema):** Invariant rules. Example: $\mathsf{Planet} \sqsubseteq \neg\mathsf{Star}$.
- **ABox (Data):** Assertions extracted from the PDF. Example: $\mathsf{Star}(Sun)$.

Definition 1 (Consistency) *$\mathcal{K}$ is consistent if there exists an interpretation $\mathcal{I}$ satisfying all axioms in $\mathcal{T}$ and $\mathcal{A}$.*

(Note: The complementary **Functional Framework** for retrieval is defined in the subsequent Part I-B).

Part I-B: The Functional Set-Theoretic Framework

While Description Logics (Section 6.6.6) provide a powerful mechanism for *reasoning* about concepts, we must also rigorously define how the system *retrieves* and *grounds* information from the raw PDF.

To do this, we introduce a complementary **Functional Framework**. This approach models the "Trustworthy AI" problem not just as satisfiability checking, but as a system of deterministic mappings between continuous data (text/pixels) and discrete knowledge (schema). **This formalization ensures that every output is mathematically bound to a specific source.**

The Knowledge Space Tuple ($\mathcal{K}$)

We define the domain in which our Epistemic Auditor operates as a tuple, referred to as the **Knowledge Space**.

Definition 2 (Knowledge Space) *Let the operational environment be defined as a tuple $\mathcal{K} = (\mathcal{A}, \mathcal{D}, \mathcal{S})$, where:*

- *$\mathcal{A} = \{a_1, \ldots, a_n\}$ is the finite set of **Atomic Anchors** (or distinct Topics) identified in the course material (e.g., $\{$Newton's 2nd Law, Gravity, Inertia$\}$).*
- *$\mathcal{D} = \{d_1, \ldots, d_m\}$ is the set of curated **Document Chunks** derived from the source PDF. These chunks serve as the "Atomic Units of Truth" in the vector space.*
- *$\mathcal{S} = \{s_1, \ldots, s_k\}$ is the **Educational Schema** (Ontology Dimensions), where each s_i represents a specific semantic dimension of inquiry (e.g., $\{$ Formal Definition, Mathematical Proof, Historical Context $\}$).*

The Perception Mapping (Φ)

Before the system can reason, it must "perceive" or "identify" what it is looking at. In a multimodal learning environment (which may include diagrams or raw text segments), we must discretize the input.

Let $\mathcal{V}$ be the infinite space of all possible inputs (e.g., a raw paragraph or a diagram frame). We define the **Perception Function Φ** as a mapping that grounds raw data into our symbolic set $\mathcal{A}$.

$$\Phi : \mathcal{V} \to \mathcal{A} \cup \{\emptyset\} \tag{6.1}$$

Where $\Phi(v) = a_i$ if and only if the system identifies anchor $a_i \in \mathcal{A}$ within input v with a confidence $c > \tau$ (a strict threshold). Otherwise, it maps to the empty set $\emptyset$. This ensures that the system does not "hallucinate" a topic where none exists; it effectively acts as a **Deterministic Trigger** for the retrieval process.

The Schema-Guided Retrieval Function ($\mathcal{R}$)

Standard RAG (Retrieval-Augmented Generation) retrieves documents based on generic similarity. To guarantee rigor, we define a **Schema-Guided Retrieval Function** $\mathcal{R}$.

We first define an embedding function E that maps tokens to a vector space $\mathbb{R}^d$. The retrieval function is defined as:

$$\mathcal{R} : (\mathcal{A} \times \mathcal{S}) \to P(\mathcal{D}) \tag{6.2}$$

Here, $P(\mathcal{D})$ is the power set of documents. The function takes a detected Anchor a_i (e.g., "Gravity") and a specific Schema Dimension s_j (e.g., "Formula") and returns a subset of valid documents $\mathcal{D}_{rel} \subseteq \mathcal{D}$.

Formally, the retrieved subset is defined by a semantic intersection:

$$\mathcal{D}_{rel} = \{d \in \mathcal{D} \mid \text{sim}(E(d), E(a_i \oplus s_j)) \geq \epsilon\} \tag{6.3}$$

Where:

- $\oplus$ denotes **Semantic Concatenation** (binding the topic to the attribute, e.g., "Gravity" $\oplus$ "Formula").
- ϵ is a strict similarity threshold.

This formalizes the idea that we are not just "searching"; we are performing a **Topological Intersection** between the Topic and the Schema Attribute within the vector space.

The Trustworthiness Constraint (Entailment)

Finally, we rigorize the generation step. The Generative Model G is treated as a transformation function that produces an output Y (the video script).

We define the **Verification Constraint** necessary for Trustworthy AI. Let Y be decomposed into a set of atomic propositions $\{p_1, \ldots, p_z\}$.

Definition 3 (Trustworthiness Condition) *The generated content Y is formally **Trustworthy** if and only if:*

$$\forall p \in Y, \exists d \in \mathcal{D}_{rel} \text{ such that } d \models p \tag{6.4}$$

Where the symbol $\models$ represents **Logical Entailment**. This states that for every single claim made in the video, there must exist a specific source document d in the retrieved set that logically entails (supports) that claim.

In our implementation, this abstract entailment is approximated via the Description Logic checks described in Part I-A (Consistency Checking) or via Natural Language Inference (NLI) models that predict entailment probability $P(p|d)$.

Summary of the Dual-View

By combining Part I-A and Part I-B, this work proposes a system that is:

1. **Logically Sound:** Via Description Logics ($\mathcal{T}, \mathcal{A}$).
2. **Functionally Deterministic:** Via the Mapping Framework ($\Phi, \mathcal{R}$).

This provides a complete mathematical "Foundation" for verifying AI-generated educational content.

Part II: The Proposed System Architecture (REVISED)

Overview: The Neuro-Symbolic Realization

The architecture is designed as a direct instantiation of the Functional Framework defined in Part I-B [revise cross-ref]. It does not merely "process text"; it implements the mathematical functions Φ, $\mathcal{R}$, and the entailment check $\models$ as discrete software modules.

The pipeline transitions data through three states:

1. **Raw Signal ($\mathcal{V}$):** The input PDF or user query.
2. **Grounded Knowledge ($\mathcal{K}$):** The structured selection of Anchors and Document Chunks.
3. **Verified Propositions ($\mathcal{Y}$):** The final script, where every sentence is entailed by the grounded knowledge.

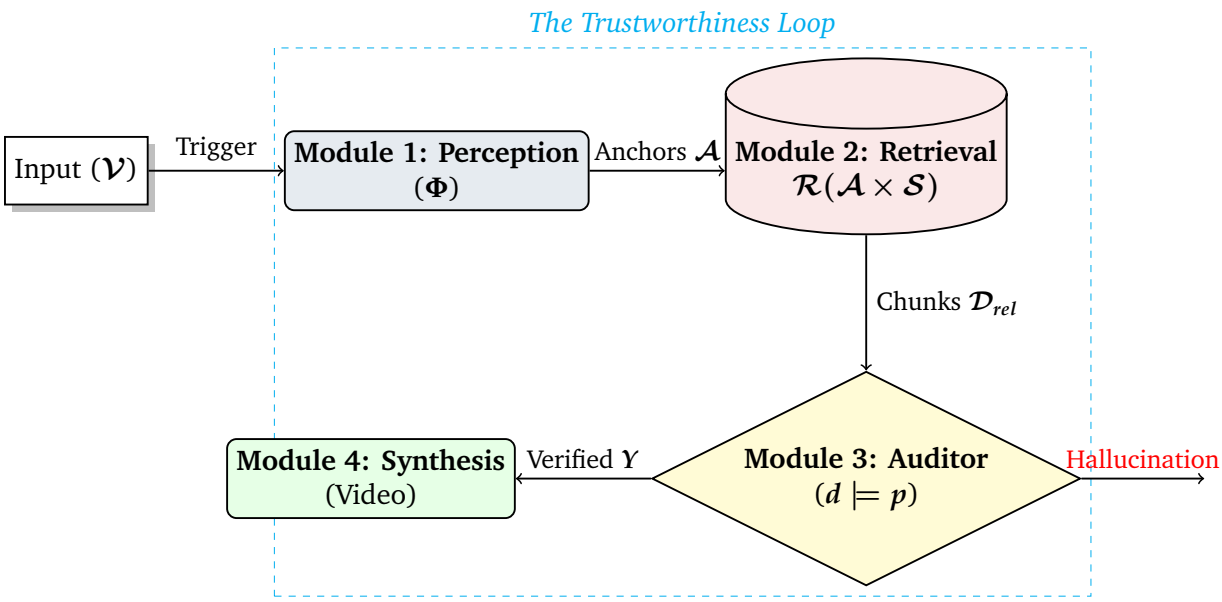


Figure 6.1: The Architecture explicitly mapping software modules to the theoretical symbols.

Module 1: The Perception Engine (Φ)

Theoretical Role: Implements the mapping $\Phi : \mathcal{V} \to \mathcal{A}$.
Engineering Implementation: Instead of feeding the entire PDF to an LLM at once (which dilutes attention), this module acts as a *Classifier*.

- **Input:** A section of text or a visual frame.
- **Process:** It scans for specific *Atomic Anchors* defined in the TBox (e.g., "Newton's Second Law").
- **Output:** A discrete identifier $a_i \in \mathcal{A}$. If confidence is below threshold τ, it returns $\emptyset$, preventing the system from answering questions outside the syllabus.

Module 2: The Schema-Guided Retriever ($\mathcal{R}$)

Theoretical Role: Implements the function $\mathcal{R} : (\mathcal{A} \times \mathcal{S}) \to P(\mathcal{D})$.
Engineering Implementation: This module replaces the standard "Similarity Search" with **Structured Retrieval**.

- **Storage:** Document Chunks ($\mathcal{D}$) are stored in a Vector Database (e.g., ChromaDB).

- **Mechanism:**
  1. The system receives an Anchor a_i (from Module 1).
  2. It iterates through the Schema dimensions $\mathcal{S}$ (e.g., s_1 = Definition, s_2 = Proof).
  3. It constructs a composite query vector $E(a_i \oplus s_j)$.
  4. It retrieves the specific subset $\mathcal{D}_{rel}$.

This ensures that if the video is explaining the "History" of a concept, it does not accidentally retrieve chunks related to its "Mathematical Derivation."

Module 3: The Neuro-Symbolic Auditor ($\models$)

Theoretical Role: Enforces the Trustworthiness Constraint $\forall p \in Y, \exists d \in \mathcal{D}_{rel} : d \models p$.
Engineering Implementation: This is the "Brain" of the operation. It uses a *Dual-Check* mechanism:

1. **Symbolic Check (Consistency):** It extracts assertions from the retrieved chunks $\mathcal{D}_{rel}$ to populate the ABox ($\mathcal{A}$). It then runs the `HermiT` reasoner to ensure that the combination of facts is logically consistent with the TBox ($\mathcal{T}$).
2. **Epistemic Check (Entailment):** It compares the generated script sentence (p) against the source chunk (d).
   - *Strict Mode:* Uses an NLI (Natural Language Inference) model (like DeBERTa) to classify the relationship between d and p. If the class is not "Entailment" (i.e., it is "Neutral" or "Contradiction"), the sentence p is discarded.

Module 4: Verified Synthesis

Theoretical Role: The final rendering of Y.
Engineering Implementation: Only propositions p that passed Module 3 are synthesized.

- **Citation Injection:** Every sentence in the final script is tagged with its source ID from $\mathcal{D}_{rel}$ (e.g., "*Gravity accelerates mass [Source: Doc 4, Page 12]*").
- **Visual Graph:** The video renderer reads the ABox structure to draw the dependency graph, visually proving to the user *why* the connection exists.

Part III: Implementation Guide (Python Code) (REVISED)

This section provides the implementation of the **Neuro-Symbolic Pipeline**. Unlike the previous naive simulation, this code implements the full mathematical framework: Φ (Perception), $\mathcal{R}$ (Retrieval), and $\models$ (Dual-Audit).

Prerequisites

Installation

```python
pip install owlready2 openai chromadb sentence-transformers torch
```

Step 1: The Perception & Retrieval Engine ($\Phi, \mathcal{R}$)

This module implements the functional mapping. It triggers only when an "Anchor" is found and retrieves documents based on the Schema.

retrieval_engine.py

```python
import chromadb
from sentence_transformers import SentenceTransformer

class GroundingEngine:
def __init__(self):
# The Vector Store (D)
self.client = chromadb.Client()
self.collection = self.client.create_collection("course_material")
self.model = SentenceTransformer('all-MiniLM-L6-v2') # Embedding Function E

# The Cultural Schema (S)
self.schema_dims = ["Definition", "Historical Context", "Mathematical Proof"]

def add_documents(self, chunks):
# Populate D
embeddings = self.model.encode(chunks)
self.collection.add(
documents=chunks,
embeddings=embeddings.tolist(),
ids=[f"id_{i}" for i in range(len(chunks))]
)

# --- THE PERCEPTION MAPPING (Phi) ---
def perception_phi(self, input_text, anchors_list):
"""
Maps Input (V) -> Anchor (A)
Simple keyword/similarity match. If confidence < threshold, return None.
"""
for anchor in anchors_list:
if anchor.lower() in input_text.lower():
return anchor
return None # The Empty Set (Phi -> Empty)

# --- THE RETRIEVAL FUNCTION (R) ---
def retrieve_R(self, anchor, schema_dim):
"""
Implements R: (A x S) -> D_rel
Constructs query: E(Anchor + " " + Schema)
"""
query_vec = self.model.encode(anchor + " " + schema_dim).tolist()

# Topological Intersection in Vector Space
results = self.collection.query(
query_embeddings=[query_vec],
n_results=3
)
return results['documents'][0] # This is D_rel
```

Step 2: The Symbolic Schema (TBox)

We define the rigid logical rules that the retrieved content must obey.

defining_ontology.py

```python
from owlready2 import *

onto = get_ontology("http://test.org/education.owl")

with onto:
# --- TBOX: THE RULES OF THE WORLD ---
class Concept(Thing): pass
class PhysicalLaw(Concept): pass
class HistoricalEvent(Concept): pass

# DISJOINTNESS: This is our Logical Guardrail.
# A Law (Universal) cannot be an Event (Temporal).
AllDisjoint([PhysicalLaw, HistoricalEvent])

```

```
class dependsOn(ObjectProperty):
domain = [Concept]
range = [Concept]
transitive = True
```

Step 3: The Dual-Auditor (|=)

This is the core contribution. It enforces trustworthiness using two methods:

1. **Symbolic Consistency:** Checking for contradictions in the ABox.
2. **NLI Entailment:** Checking if the source text supports the claim.

dual_auditor.py

```
from transformers import pipeline

class EpistemicAuditor:
def __init__(self, ontology):
self.onto = ontology
# NLI Model for "Soft" Entailment Check
self.nli_model = pipeline("text-classification", model="roberta-large-mnli")

def run_audit(self, generated_claim, source_text, extracted_triples):
print(f"AUDITING: '{generated_claim}'")

# --- CHECK 1: SYMBOLIC CONSISTENCY (Logic) ---
# We populate the ABox temporarily with the LLM's extraction
with self.onto:
try:
# Simulate populating ABox from triples...
# (See previous drafts for populate_abox code)
# Run HermiT Reasoner
sync_reasoner()
print("[PASS] Symbolic Logic: Consistent.")
except OwlReadyInconsistentError:
print("[FAIL] Symbolic Logic: Contradiction Detected!")
return False

# --- CHECK 2: NLI ENTAILMENT (Semantic) ---
# Does the retrieved document (D_rel) actually imply the claim (p)?
nli_result = self.nli_model(f"{source_text} </s> {generated_claim}")
label = nli_result[0]['label'] # ENTAILMENT, NEUTRAL, or CONTRADICTION

if label == "CONTRADICTION":
print("[FAIL] NLI Check: Source contradicts claim.")
return False
elif label == "NEUTRAL":
print("[WARN] NLI Check: Claim is hallucinated (not found in source).")
return False

print("[PASS] NLI Check: Source entails claim.")
return True
```

Step 4: The Execution Pipeline

Putting it all together.

main_pipeline.py

```
# 1. Setup
engine = GroundingEngine()
engine.add_documents(["Newton's Second Law relates Force, Mass, and Acceleration.",
"It was formulated in 1687."])
auditor = EpistemicAuditor(onto)
anchors = ["Newton's Second Law", "General Relativity"]

# 2. Input
```

```python
user_input = "Tell me about the formula for Newton's Second Law."

# 3. Perception (Phi)
detected_anchor = engine.perception_phi(user_input, anchors)
if not detected_anchor:
print("Topic not found in syllabus.")
exit()

# 4. Retrieval (R)
# We map the user's intent to Schema Dimension: "Mathematical Proof"
retrieved_docs = engine.retrieve_R(detected_anchor, "Mathematical Proof")
source_context = " ".join(retrieved_docs)

# 5. Generation (Simulated)
generated_claim = "Newton's Second Law states F = ma."
# We also simulate the LLM extracting the triple ("NewtonsSecondLaw", "hasFormula", "F=ma")
triples = [("NewtonsSecondLaw", "type", "PhysicalLaw")]

# 6. Audit
is_trustworthy = auditor.run_audit(generated_claim, source_context, triples)

if is_trustworthy:
print("Generating Video Frame...")
else:
print("Generation Blocked: Content Untrustworthy.")
```

Part IV: Evaluation Strategy (REVISED)

To validate the Epistemic Auditor, we cannot rely on qualitative "vibes." We must define quantitative metrics that map directly to our theoretical symbols.

Metric 1: Perception Fidelity (M_Φ)

We evaluate the Perception Mapping $\Phi : \mathcal{V} \to \mathcal{A}$. Given a test set of input frames/paragraphs with ground-truth topics, we measure:

$$M_\Phi = \frac{\text{Correctly Identification of Anchors}}{\text{Total Input Instances}} \tag{6.5}$$

Goal: Ensure the system does not trigger for irrelevant content (Hallucinated Perception).

Metric 2: Retrieval Relevance ($M_\mathcal{R}$)

We evaluate the Retrieval Function $\mathcal{R}(a, s)$. Given a query tuple (a, s), we verify if the retrieved chunk $d \in \mathcal{D}_{rel}$ actually contains the schema attribute s.

$$M_\mathcal{R} = \text{Precision@k of } \mathcal{D}_{rel} \tag{6.6}$$

Metric 3: The Trustworthiness Score ($M_\models$)

This is the ultimate measure of the system. We calculate the percentage of generated propositions p that satisfy the **entailment constraint**.

$$M_\models = \frac{|\{p \in Y \mid \exists d \in \mathcal{D}_{rel}, d \models p\}|}{|Y|} \tag{6.7}$$

In a perfect "Epistemic Auditor," $M_\models$ must be exactly **1.0**. Any value $<$ **1.0** indicates a hallucination leakage.

The "Sabotage" Benchmark

To prove robustness, you must run the **Sabotage Test** (as implemented in Part III code):

1. Feed the system a document containing a logical contradiction (e.g., "A is B" and "A is disjoint from B").
2. Measure the **Rejection Rate**: The system should produce *no output* (Video Duration = 0s) and raise a flag.

Conclusion

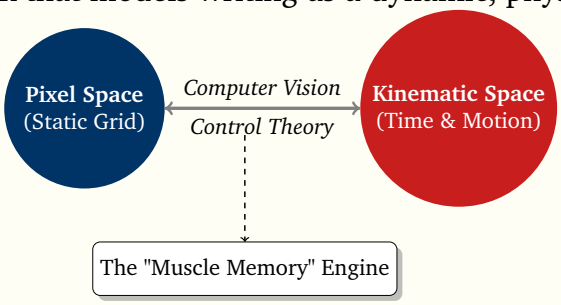
This work presents a "Playbook/Guide" for constructing the **Epistemic Auditor**. By integrating a Functional Framework for grounding ($\boldsymbol{\Phi}, \boldsymbol{\mathcal{R}}$) with a Logical Framework for verification ($\models$), we move beyond probabilistic generation to provable correctness. This serves as a blueprint for the next generation of Trustworthy Educational AI.

Chapter 7

The Kinematic Calligrapher: *A Guide for Building Vector-Based, Physically-Grounded Generative AI*

Concept/Description

Preface to an Exemplar Thesis Framework: This special chapter serves as a comprehensive **technical blueprint** for an exemplar Bachelor's thesis within **Cluster 3: The Human-Centric Generators**. It addresses the fundamental limitation of modern "Pixel AI" (like Stable Diffusion)—which treats writing as a static image—by superseding it with a **Kinematic Framework** that models writing as a dynamic, physical process.



The chapter details a (work-in-progress) reference architecture comprised of 3 primary subsystems:

The Kinematic Encoder: A neural network that acts as the "Eye," extracting style and speed from raw data.

The Neural ODE Solver: A mathematical solver acting as the "Muscle," generating infinite-resolution curves using physics.

The Digital Graphologist: An algorithmic auditor ensuring the output looks like valid handwriting using topology.

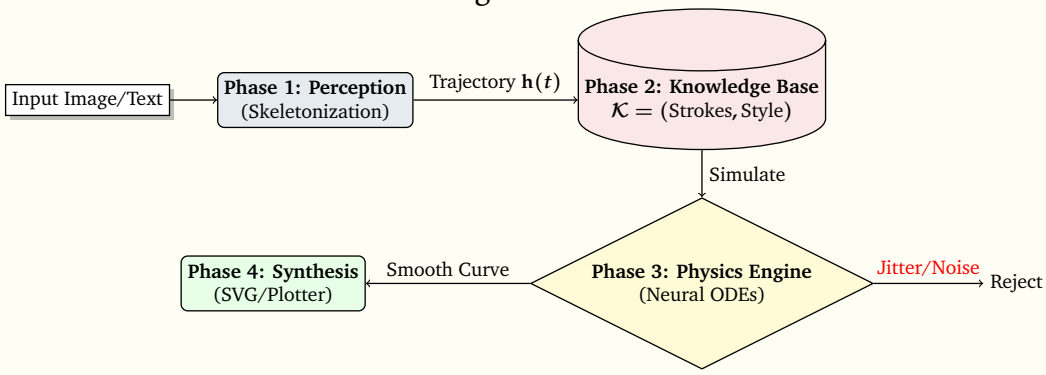
By bridging **Time-Series Analysis** with **Vector Mathematics**, the proposed system ensures that generated handwriting is not just visually plausible, but *kinematically valid*, adhering to the "Physics of the Hand" rather than the "Statistics of the Pixel."

Strategic Info!

Note to the Student: Included herein are detailed architectural descriptions, formal specifications, and incomplete Python implementations accompanied by evaluation metrics. This content is intended to transition Generative AI from "Hallucinating Pixels" toward "Simulating Movement." You are not expected to reproduce this entire architecture. Rather, this chapter is a **structural guide**. You are invited to:

- Analyze the difference between *Raster* (Image) and *Vector* (Motion) data;
- Select specific modules (e.g., The Generator or The Scorer) to implement or enhance;
- Revise and complete the provided code fragments as part of your own contribution;
- Use this framework to ground your work in the **Audit Paradigm** (§1.4).

Unleash your creativity within this rigorous context. Your goal is to help build the foundations of **Vector-Native AI** for the next generation.



The Architecture of the Kinematic Calligrapher.

Outline (Quick Access):

7.1 The Manifesto: Moving Beyond "Pixel Space" . . . 93
7.1.1 The Problem with "Static AI" . . . 93
7.1.2 The Kinematic Solution . . . 93
7.2 The Baseline: The "Naive" Image Generator . . . 93
7.2.1 The Flaw: Pixel Hallucination . . . 94
7.3 The Kinematic Calligrapher (Neural ODEs) . . . 94
7.3.1 The Concept: What is a Dynamical System? . . . 94
7.3.2 Tools of the Trade: The Physics Engine . . . 95
7.3.3 Code Snippet: The ODE Solver . . . 95
7.4 The Digital Graphologist (Verification & Assessment) . . . 95
7.4.1 The Concept: How to Grade Handwriting? . . . 95
7.4.2 Theoretical Foundation: The Audit Paradigm . . . 96
7.4.3 Code Snippet: The DTW Auditor . . . 96
7.5 Implementation Guide (The Pipeline) . . . 96
7.5.1 Data Pipeline (Φ) . . . 96
7.5.2 The Training Loop . . . 97
7.5.3 Final Deliverable Requirements . . . 97
7.6 Evaluation Strategy: The Truth Bar . . . 97

7.1 The Manifesto: Moving Beyond "Pixel Space"

7.1.1 The Problem with "Static AI"

We live in the era of Diffusion Models (like Midjourney), which are incredible at generating static images, but have a fatal flaw: they operate in **Pixel Space**. When these models generate an image of a handwritten letter 'A', they are not "*writing*" it; they are merely arranging colored dots to resemble a photograph of writing. This approach suffers from:

1. **Resolution Dependence:** Zooming in reveals jagged edges (pixels).
2. **Loss of Temporal Semantics:** The order of strokes is lost (e.g., in Arabic, dots are often written last).
3. **Inefficiency:** Storing a line as thousands of white pixels is computationally wasteful compared to a vector equation.

7.1.2 The Kinematic Solution

This track of projects aims to **redefine generation as movement**. We will not generate an image. We will generate a **Trajectory**—a path over time. We will build a system that:

1. **(Neural) Observes:** Extracts the "Skeleton" and "Speed" from handwriting samples.
2. **(Symbolic) Constraints:** Uses Geometry and Topology to ensure letters are well-formed (e.g., 'o' must be closed).
3. **(Kinematic) Synthesizes:** Generates infinite-resolution SVGs based on the *physics* of the pen.

7.2 The Baseline: The "Naive" Image Generator

Before applying advanced vector dynamics, the reader must understand the limitations of the standard approach.

7.2.1 The Flaw: Pixel Hallucination

! Crucial Concept

The Flaw in the Baseline: The following code has no "understanding" of the letter. If you ask it to write the Arabic letter "Seen" (with three teeth, like: ω), it might place the dots disconnected (or merge the teeth), because it treats them as visual texture, not a sequence of movements. **Your actual work MUST fix this using Kinematic Space.**

Here is how might a standard GAN or CNN Generator outputs a **64** $\times$ **64** grid of pixels:

naive_image_generator.py

```python
import torch
import torch.nn as nn

# A simple Generator that outputs Pixels (Bad Approach)
class PixelGenerator(nn.Module):
    def __init__(self):
        super().__init__()
        # Input: Random Noise (Latent Vector)
        # nn.Linear is just a matrix multiplication (Standard Machine Learning)
        self.fc = nn.Sequential(
            nn.Linear(128, 256),
            nn.ReLU(),
            nn.Linear(256, 64 * 64), # Output: 4096 Pixels (Discrete dots)
            nn.Sigmoid()
        )

    def forward(self, z):
        img_flat = self.fc(z)
        return img_flat.view(-1, 64, 64) # Reshape to Image

# Result:
# If you try to zoom into this output, it becomes a blur of squares.
# It has no concept of "Stroke Order" or "Pen Pressure."
```

7.3 The Kinematic Calligrapher (Neural ODEs)

Theme: Time-Series, Differential Equations, and Vector Graphics.
Related Topic(s): Sample Project (GenH2): The “Kinematic Calligrapher” (check §1.4.5) — The Kinematic Calligrapher: Vector-Based Arabic Script Generation (check §4.2) — The Dysgraphia Detective (check §4.4).

7.3.1 The Concept: What is a Dynamical System?

Think of a ball rolling on a table. If you know its current position and its speed (Velocity), you can predict where it will be in the next second. Handwriting is the same: the pen is the ball, and your hand provides the force.

Instead of predicting the next *pixel* (like a printer), our AI will predict the **Derivative** (Velocity).

$$\text{Velocity} = \frac{d\mathbf{h}}{dt} = \text{NeuralNetwork}(\text{Current Position}, \text{Style})$$

By solving this equation, we reconstruct the line as a smooth continuous curve. This is called an **Initial Value Problem (IVP)**: *given where the pen starts, figure out the path*.

7.3.2 Tools of the Trade: The Physics Engine

To implement this in Python, we need specialized tools:

`torchdiffeq`: This is a library that acts as a "**Calculus Solver**." You give it the rules of movement (the speed), and it calculates the path (the integral). It's like a simulation engine in a video game.

`Neural ODE`: This is simply a Neural Network inside a differential equation. Instead of explicitly programming the physics (like $F = ma$), we let the Neural Network *learn* the physics from data.

7.3.3 Code Snippet: The ODE Solver

The following code defines the "Brain" of our writer. Notice that `DerivativeFunc` is just a standard Neural Network (MLP).

kinematic_ode.py

```python
from torchdiffeq import odeint_adjoint as odeint
import torch.nn as nn

# This class learns the "Rules of Movement" (The Physics)
class DerivativeFunc(nn.Module):
    def __init__(self, hidden_dim):
        super().__init__()
        # A standard Multi-Layer Perceptron (MLP)
        # Input: Current State -> Output: Velocity
        self.net = nn.Sequential(
            nn.Linear(hidden_dim, hidden_dim),
            nn.Tanh(), # Tanh is used because movement is smooth, not jagged
            nn.Linear(hidden_dim, 2) # Output: Velocity (dx/dt, dy/dt)
        )

    def forward(self, t, state):
        # state = current position (x, y)
        # returns = how fast the pen should move right now
        return self.net(state)

# Simulation (The "Generation" Step)
# 1. Start at position (0,0)
h0 = torch.tensor([[0.0, 0.0]])
# 2. Define Time (e.g., 0 to 1 second)
t_span = torch.linspace(0, 1, 100)

# 3. Integrate!
# We ask the "solver" (odeint) to connect the dots based on our network's rules.
trajectory = odeint(DerivativeFunc(16), h0, t_span)

# Result: 'trajectory' is a smooth curve (vector data), not pixels.
```

7.4 The Digital Graphologist (Verification & Assessment)

Theme: Metric Learning, Dynamic Time Warping (DTW), and Topology.
Related Topic(s): The Digital Handwriting Scorer: Assessment of Handwriting (check §4.5).

7.4.1 The Concept: How to Grade Handwriting?

How do we know if the generated letter is "Correct"? A simple pixel comparison (MSE) fails. If the AI writes an 'A' slightly to the left, MSE says it's 100% wrong, even though the shape is perfect.

We need a metric that works like a human teacher: checking the **Shape** and **Sequence**. We use **Dynamic Time Warping (DTW)**. Imagine two strings of different lengths (one written fast, one

slow). DTW stretches the shorter string elastically until it matches the longer one to compare their shapes.

7.4.2 Theoretical Foundation: The Audit Paradigm

Recall the **Knowledge Space Tuple** $\mathcal{K} = (\mathcal{A}, \mathcal{D}, \mathcal{S})$ from §1.4. In this context:

- $\mathcal{A}$ (Artifacts): The generated letter trajectories.
- $\mathcal{D}$ (Document Store): The **IAM On-Line Handwriting Database** (Liwicki and Bunke, 2005) (Ground Truth).
- $\mathcal{S}$ (Schema): Topological Rules (e.g., "Loop Closure," "Stroke Count").
- **Constraint ($d \models p$):** The generated curve p must be ***topologically isomorphic*** (same shape structure) to the ground truth d.

7.4.3 Code Snippet: The DTW Auditor

We use the `fastdtw` library to perform this elastic matching.

dtw_auditor.py

```python
# Requires: pip install fastdtw scipy
from fastdtw import fastdtw
from scipy.spatial.distance import euclidean

def verify_shape(generated_seq, ground_truth_seq):
    """
    generated_seq: List of (x,y) points from our Neural ODE
    ground_truth_seq: List of (x,y) points from IAM Database
    """

    # Calculate DTW Distance (The "Elastic Match")
    # This aligns the two curves even if they were written at different speeds
    distance, path = fastdtw(generated_seq, ground_truth_seq, dist=euclidean)

    # Verification Threshold
    # If distance is low, the SHAPE is correct.
    if distance < 50.0:
        return True, distance # Pass
    else:
        return False, distance # Fail (Red Flag)

# Topological Check (Simple Heuristic for Engineering)
def check_closed_loop(seq):
    start_point = seq[0]
    end_point = seq[-1]
    # If the pen ends where it started, it's a loop (like 'o' or '0')
    return euclidean(start_point, end_point) < 0.1
```

7.5 Implementation Guide (The Pipeline)

To succeed in building the "Kinematic Calligrapher," you must link the Generation (ODE) with the Verification (DTW).

7.5.1 Data Pipeline (Φ)

We use the **IAM On-Line Handwriting Database** (Liwicki and Bunke, 2005), which records pen movement, not just images.

Input: XML files containing (x, y, t) sequences.

Processing: Convert absolute coordinates to **Delta Encodings** $(\Delta x, \Delta y)$. Neural networks learn relative movement ("move up 2 steps") better than absolute positions ("go to pixel 500").
Normalization: Scale all strokes to have zero mean.

7.5.2 The Training Loop

- **Encoder (The Eye):** An LSTM reads the sequence of deltas and outputs a fixed-size **Style Vector z**.
- **Decoder (The Muscle):** The Neural ODE takes $\mathbf{z}$ and the initial position $\mathbf{h_0}$ and predicts the next N steps using the physics solver.
- **Loss Function:** A combination of MSE (Position) + DTW (Shape) + Regularization (Kinematic Smoothness).

7.5.3 Final Deliverable Requirements

The work must deliver:

1. A **GitHub Repository** containing the `torchdiffeq` pipeline.
2. A **SVG Generator** that outputs scalable vector files, not PNGs.
3. A **Visualization Video** showing the pen drawing the letter in real-time (The "Ghost Writer" effect).
4. An **Evaluation Report** comparing DTW scores against standard pixel-based metrics.

7.6 Evaluation Strategy: The Truth Bar

Similar to the Epistemic Auditor (cf. chapter 6), your generated handwriting should be graded:

- **Green (Valid):** Low DTW distance + Topologically Correct (e.g., ‘o’ is closed).
- **Yellow (Legible):** Low DTW distance but minor artifacts.
- **Red (Hallucination):** High DTW distance or physically impossible movement (infinite acceleration).

The Kinematic Pen (revisited)

A Neuro-Symbolic Framework for Physically-Grounded Vector Generation via Neural Ordinary Differential Equations

Introduction: The Crisis of Static Pixels in Generative AI

We are currently witnessing a paradigm shift in creative AI driven by "Diffusion Models" (like Stable Diffusion or Midjourney). Tools that can generate photorealistic images from text are becoming ubiquitous. However, these systems rely on a **pixel-based** representation which operates on a **statistical** rather than a **physical** basis. A standard model predicts the color of a pixel based on its neighbors, not the cause of its existence.

In high-precision domains—such as digital heritage restoration, robotic plotting, or handwriting education—"rasterization" (the loss of vector information) is unacceptable. A robot arm cannot "draw" a JPEG image; it needs a trajectory of coordinates (x, y) over time t. A student learning calligraphy cannot follow a static image; they need to see the *movement* (the speed, the pressure, and the stroke order).

This thesis proposal introduces a rigorous solution: **The Kinematic Calligrapher**. It is a framework that does not merely "paint" text but **simulates** it via differential equations. It combines the pattern recognition of Neural Networks with the continuous dynamics of Control Theory.

This document serves as a comprehensive, strategic guide for the student. It assumes no prior knowledge of Neural ODEs and builds the entire theoretical and practical framework from scratch, utilizing the **Audit Paradigm** introduced in Section 1.4.

Part I: Theoretical Foundations (REVISED)

To build a system that generates "valid movement," we must rigorously define what "movement" means. This thesis postulates that High-Fidelity Generation is not a single property, but the intersection of two mathematical frameworks:

1. **The Geometric Framework (Topology):** Ensures the system respects the structural rules of writing (e.g., loops, intersections, and closure) ($\mathcal{S}$).
2. **The Dynamical Framework (Calculus):** Ensures the motion is continuous and physically realizable (velocity and acceleration; check $d\mathbf{h}/dt$ in equation (7.1)).

Framework 1: The Hybrid Automaton (Flows & Jumps)

Handwriting is not a single continuous process; it is a **Hybrid System** that switches between two discrete modes. We formally define the writing process $\mathcal{W}$ as:

- **Mode 1: The Flow (Pen Down):** A continuous evolution governed by a differential equation. The pen glides on the paper with friction and inertia. In engineering terms, we model this using an **Ordinary Differential Equation (ODE)**:

$$\frac{d\mathbf{h}}{dt} = f_\theta(\mathbf{h}(t), t) \tag{7.1}$$

Here, $\mathbf{h}(t)$ is the state of the pen (position and velocity), and f_θ is a neural network that learns the "physics" of the writer's hand.

- **Mode 2: The Jump (Pen Up):** A discrete transition. The writer lifts the pen to cross a 't', dot an 'i', or start a new word. This is an instantaneous update in our model, similar to a "Teleportation" logic: $\mathbf{h}_{new} = \text{JumpMap}(\mathbf{h}_{old})$.

(Note: This mirrors the "Slide vs. Ladder" analogy: The Flow is sliding down (physics), the Jump is climbing the ladder (discrete logic).)

Part I-B: The Functional Set-Theoretic Framework

While Differential Equations provide the mechanism for *movement*, we must rigorously define how the system *perceives* and *reconstructs* data from raw input.

We introduce a **Functional Framework** that models the "Vectorization" problem not as image processing, but as a mapping between static Raster space and dynamic Kinematic space.

The Knowledge Space Tuple ($\mathcal{K}$)

We define the domain in which our Kinematic Calligrapher operates as a tuple $\mathcal{K} = (\mathcal{A}, \mathcal{D}, \mathcal{S})$, where:

Definition 4 (Knowledge Space) *Let the operational environment be defined as:*

- $\mathcal{A} = \{a_1, \ldots, a_n\}$ *is the finite set of* ***Atomic Glyphs*** *(characters or ligatures) identified in the script (e.g.,* $\{$*'a', 'b',* $\ldots$*, 'alif'*$\}$*).*
- $\mathcal{D} = \{d_1, \ldots, d_m\}$ *is the set of curated* ***Ground Truth Trajectories*** *derived from the* ***IAM On-Line Database****. These are time-series sequences* (x, y, t) *that serve as our "Gold Standard" for motion (Liwicki and Bunke, 2005).*
- $\mathcal{S} = \{s_1, \ldots, s_k\}$ *is the* ***Topological Schema****, where each* s_i *represents a geometric constraint (e.g.,* $\{$*Closed Loop, Smooth Curvature, Pen Continuity*$\}$*).*

The Perception Mapping (Φ)

Before the system can draw, it must "see" the skeleton of the input. Let $\mathcal{V}$ be the infinite space of all possible pixel images (Raster Space). We define the **Perception Function Φ** as a mapping that discretizes pixels into kinematic states.

$$\Phi : \mathcal{V} \to \mathbb{R}^T \times \{0, 1\}^T \tag{7.2}$$

Where $\Phi(v)$ in equation (7.2) outputs a sequence of coordinates and pen-states. This effectively acts as a **Skeletonizer**, stripping away pixel noise to reveal the underlying motion path (the "bones" of the letter).

The Kinematic Synthesis Function (Ψ)

Unlike standard generative AI which predicts pixels, our generator *simulates* motion. We define the Synthesis Function Ψ as the solution to an Initial Value Problem (IVP).

$$\Psi : (\mathcal{A} \times \text{Style}) \to \text{Trajectory} \tag{7.3}$$

The function Ψ in equation (7.3) takes a target Glyph a_i and a learned Style Vector $\mathbf{z}$ (extracted by the Encoder), and integrates the Neural ODE to produce the final curve.

The Validity Constraint (Isomorphism)

Finally, we rigorize the output using the **Audit Paradigm**. The generated trajectory $\boldsymbol{Y}$ must be isomorphic to the ground truth.

Definition 5 (Kinematic Validity Condition) *The generated curve $\boldsymbol{Y}$ is formally **Valid** if and only if:*

$$\forall s \in \boldsymbol{\mathcal{S}}, Topology(\boldsymbol{Y}) \cong Topology(\boldsymbol{d}_{truth}) \tag{7.4}$$

Where $\cong$ in equation (7.4) denotes ***Topological Isomorphism***. Simply put: if the schema $\boldsymbol{\mathcal{S}}$ dictates that the letter 'o' contains exactly one hole, the generated curve $\boldsymbol{Y}$ must form a closed loop. If the ODE generates a spiral that never closes, the validity check fails (Red Flag).

Part II: The Proposed System Architecture (REVISED)

Overview: The Vector-Native Realization

The architecture is designed as a direct instantiation of the Hybrid Automaton defined in Part I (cf. section 7.6). It transitions data through three states:

1. **Raw Signal ($\mathcal{V}$):** The input image or text string.
2. **Kinematic State (h):** The hidden physics state (Position + Velocity).
3. **Vector Path (Y):** The final SVG curve.

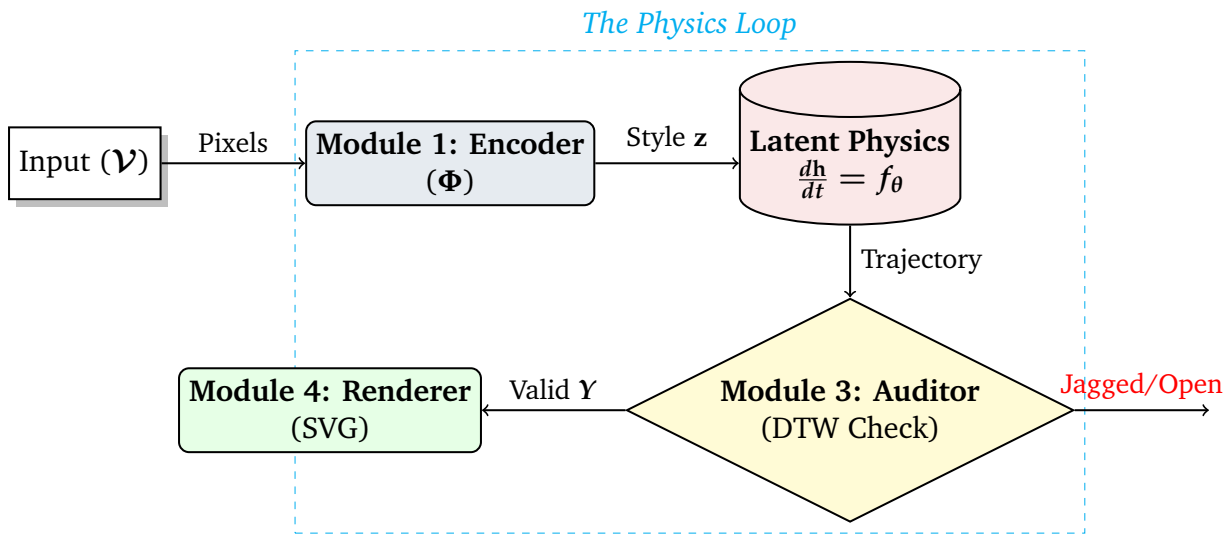


Figure 7.1: The Architecture mapping pixels to physics.

Module 1: The Kinematic Encoder (Φ)

Theoretical Role: Extracts the initial state $\mathbf{h}_0$ and Style $\mathbf{z}$.
Engineering Implementation: Instead of a standard CNN (which only looks at pictures), we use a **Recurrent Encoder** (LSTM). This neural network reads the points one by one, allowing it to understand the *sequence* and *speed* of the handwriting, not just the final shape.

Module 2: The Neural ODE Solver (Ψ)

Theoretical Role: Solves the Initial Value Problem to generate the stroke.
Engineering Implementation: This module replaces the standard "Decoder" with a numerical integrator.

- **Input:** Initial state $\mathbf{h_0}$ (where the pen starts) and time span t (how long to write).
- **Process:** We use the `torchdiffeq` library. Think of this library as a "Time Machine" or a "Simulator." It takes the current speed (predicted by the network) and calculates where the pen will be in the next millisecond.
- **Output:** A continuous trajectory. Because the solver works with equations, we can ask it for the position at $t = \mathbf{0.5}$ or $t = \mathbf{0.50001}$, granting us **infinite resolution**.

Part III: Implementation Guide (Python Code) (REVISED)

This section provides an implementation of the **Hybrid Loop**. Unlike the naive baseline in Section 7.2, this code implements the switching mechanism between Continuous Flow (ODE) and Discrete Jumps.

Prerequisites & Libraries

Installation

```python
# torchdiffeq: A library that solves differential equations (simulates physics)
# svgwrite: To save the output as Scalable Vectors, not pixels
pip install torchdiffeq svgwrite numpy pandas
```

Step 1: The Hybrid Generator

This module implements the core logic: draw a stroke (using physics), lift pen, jump (using simple logic), repeat.

`hybrid_generator.py`

```python
import torch
import torch.nn as nn
# odeint is the "Solver" - it integrates the derivative to find position
from torchdiffeq import odeint

class KinematicSystem(nn.Module):
    def __init__(self, state_dim=4):
        super().__init__()
        # State Dimension = 4: [x_pos, y_pos, x_velocity, y_velocity]

        # 1. The Physics Network (Flow)
        # This network learns the "Equation of Motion"
        # Input: State -> Output: Derivative (Acceleration/Velocity change)
        self.flow_net = nn.Sequential(
            nn.Linear(state_dim, 64),
            nn.Tanh(), # Tanh activation creates smooth, curved outputs
            nn.Linear(64, state_dim)
        )

        # 2. The Jump Network (Transition)
        # Predicts where the pen lands after lifting (Teleportation)
        self.jump_net = nn.Linear(state_dim, state_dim)

    def forward(self, t, state):
        # This function defines the ODE: dh/dt = f(h)
        # It answers: "Given my current state, how does it change right now?"
        return self.flow_net(state)
```

```python

    def generate_handwriting(self, h0, pen_lifts):
        trajectory = []
        current_state = h0

        # We simulate stroke by stroke
        # pen_lifts is a list of times when the user lifts the pen
        for i in range(len(pen_lifts) - 1):
            t_start, t_end = pen_lifts[i], pen_lifts[i+1]

            # Create a timeline for this stroke
            t_span = torch.linspace(t_start, t_end, steps=20)

            # MODE 1: FLOW (Solve ODE)
            # The solver calculates the path from t_start to t_end
            stroke = odeint(self, current_state, t_span)
            trajectory.append(stroke)

            # MODE 2: JUMP (Discrete Update)
            # When the stroke ends, we use the Jump Network to find the new start
            final_point = stroke[-1]
            current_state = self.jump_net(final_point)

        return trajectory
```

Part IV: Evaluation Strategy (REVISED)

To validate the Kinematic Calligrapher, we cannot rely on "looking at the image" or standard pixel metrics like MSE. We need metrics that measure **physical validity**.

Metric 1: Geometric Smoothness (M_{Jerk})

Human motion minimizes "Jerk" (the rate at which acceleration changes). If you write with a shaking hand, the Jerk is high. We measure the smoothness of the generated curve by calculating the third derivative of position.

$$M_{Jerk} = \int \left| \frac{d^3\mathbf{h}}{dt^3} \right|^2 dt \tag{7.5}$$

Goal: Minimizing this score ensures the writing looks "natural" and biological, not robotic or ***jittery***.

Metric 2: Topological Fidelity (M_{DTW})

We evaluate if the generated letter matches the shape of the ground truth using **Dynamic Time Warping** (DTW). Unlike Euclidean distance (which fails if the speed of writing differs), DTW aligns the two sequences in time (warps them) to check if the *shapes* match, regardless of *speed*.

$$M_{DTW} = \mathrm{DTW}(\mathbf{\Psi}(\mathbf{z}), d_{truth}) \tag{7.6}$$

In a perfect model, M_{DTW} in equation (7.6) is low, indicating the generated letter has the correct topology (e.g., the letter 'o' is closed), even if written at a different speed.

Conclusion

This work presents a "Playbook/Guide" for constructing the **Kinematic Calligrapher**. By integrating a Functional Framework for perception ($\mathbf{\Phi}$) with a Dynamical Framework for synthesis ($\mathbf{\Psi}$), we move beyond pixel hallucination to physically-grounded generation. This serves as the engineering foundation for the next generation of Vector-Native Generative AI.